

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống tử gửi đồ tự động tại điểm công cộng

Bùi Xuân Nhất

Nhat.bx225897@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: Ths. Lê Bá Vui

Khoa: Kỹ thuật Máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống gửi đơn tự động tại điểm công cộng

Bùi Xuân Nhất

Nhat.BX225897@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: Ths. Lê Bá Vui _____

Chữ kí GVHD

Khoa: Kỹ thuật Máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Hoàn thành đồ án tốt nghiệp này là kết quả của một chặng đường dài, và em không thể đi đến cuối nếu thiếu sự đồng hành của nhiều người. Trước hết, em xin gửi lời cảm ơn sâu sắc đến thầy Lê Bá Vui — người đã trực tiếp hướng dẫn, chỉ ra những thiếu sót và giúp em nhìn nhận vấn đề một cách nghiêm túc hơn trong suốt quá trình thực hiện đồ án. Em cũng xin cảm ơn quý thầy cô trong Trường Công nghệ thông tin và Truyền thông đã trang bị cho em nền tảng kiến thức quý báu. Cảm ơn gia đình đã luôn là chỗ dựa vững chắc, để em yên tâm tập trung vào việc học. Cảm ơn bạn bè đã cùng nhau trao đổi, động viên trong những lúc bế tắc. Và cảm ơn người luôn ở bên, kiên nhẫn lắng nghe những áp lực của em suốt thời gian qua. Cuối cùng, em muốn cảm ơn chính bản thân mình vì đã không bỏ cuộc, đã kiên trì và cố gắng đến phút cuối để đạt được kết quả này.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh nhu cầu gửi, nhận và quản lý đồ dùng cá nhân tại trường đại học, ký túc xá và các không gian công cộng như sân bay, tòa chung cư ngày càng tăng, các phương thức quản lý tủ truyền thống như sử dụng khóa thủ công hoặc phụ thuộc vào nhân sự quản lý trực tiếp tại trạm tủ bộc lộ nhiều hạn chế. Những phương thức này khó theo dõi trạng thái tủ theo thời gian thực, dễ phát sinh tình trạng mất chìa khóa, thiếu minh bạch về thời gian sử dụng và chưa thuận tiện cho người dùng khi cần thuê hoặc mở tủ ngoài giờ hành chính. Một số hướng tiếp cận khác như tủ khóa điện tử độc lập, hệ thống mã PIN cố định hoặc nền tảng quản lý tập trung đã được áp dụng. Tuy nhiên, các giải pháp này vẫn chưa tích hợp đầy đủ giữa thiết bị phần cứng, ứng dụng cho người dùng, trang quản trị cho người quản lý và cơ chế giám sát trạng thái theo thời gian thực.

Xuất phát từ thực tế đó, đồ án hướng tới việc xây dựng một hệ thống tủ gửi và nhận đồ tự phục vụ, trong đó người dùng có thể thuê tủ thông qua ứng dụng trên điện thoại hoặc thao tác trực tiếp tại trạm tủ. Đồng thời, quản trị viên có thể điều phối, giám sát hệ thống và theo dõi các số liệu vận hành thông qua một dashboard riêng. Hướng tiếp cận này phù hợp với các môi trường công cộng như trường học, sảnh chung cư và sân bay, nơi cần một giải pháp dễ sử dụng, thao tác nhanh và có khả năng tự vận hành mà không phụ thuộc vào nhân viên trực.

Hệ thống được xây dựng với bốn thành phần chính: backend đảm nhận xử lý nghiệp vụ và quản lý dữ liệu; ứng dụng tại trạm tủ chạy trên Raspberry Pi để điều khiển phần cứng và hiển thị giao diện cho người dùng thao tác trực tiếp; dashboard theo dõi thời gian thực dành cho quản trị viên; và ứng dụng Android dành cho người dùng cuối. Các thành phần này được kết nối và trao đổi dữ liệu với nhau theo thời gian thực. Kết quả của đồ án là một hệ thống có khả năng quản lý đầy đủ vòng đời của một phiên thuê tủ, xác thực mở tủ bằng mã PIN hoặc mã QR được cấp sẵn, ghi nhận nhật ký hoạt động và hỗ trợ ghép nối thêm các ngăn tủ hoặc trạm tủ mới vào hệ thống. Hệ thống cũng đặt nền móng cho việc mở rộng các chức năng trong tương lai.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

In the context of the growing demand for storing, receiving, and managing personal belongings at universities, dormitories, and other public places such as airports and apartment buildings, traditional locker management methods still reveal many limitations. Conventional approaches, such as manual locker keys or direct supervision by on-site staff, make it difficult to monitor locker status in real time, may lead to key loss, lack transparency in usage duration, and are inconvenient for users who need to rent or open lockers outside working hours. Other approaches, including standalone electronic lockers, fixed PIN-based systems, and centralized management platforms, have also been introduced. However, these solutions have not fully integrated hardware devices, user applications, administrator interfaces, and real-time monitoring mechanisms.

Based on these practical issues, this graduation project aims to develop a self-service locker system for storing and receiving items. The system allows users to rent lockers directly through a mobile application or at locker stations, while administrators can manage, coordinate, and monitor system data through a dedicated dashboard. This approach is selected because it is suitable for public environments such as schools, apartment lobbies, and airports, where a user-friendly, fast, and self-operating solution is required without depending on on-site staff.

The proposed system consists of four main components: a backend service responsible for business logic and data processing, a locker-station application running on Raspberry Pi to control hardware and provide an interface for direct user interaction, a real-time monitoring dashboard for administrators, and an Android application for end users. These components are connected and communicate with one another in real time. The final result of the project is a complete system that manages the full life cycle of a locker rental session, supports locker unlocking through PIN codes or pre-issued QR codes, records activity logs, and provides a foundation for expanding the system with additional locker compartments and locker stations in the future.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	3
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	5
2.1 Khảo sát thực tế.....	5
2.1.1 Hệ thống SantaPocket.....	5
2.1.2 Hệ thống ViettelPost SmartBox	6
2.2 Tổng quan chức năng	7
2.2.1 Biểu đồ use case tổng quát	8
2.2.2 Biểu đồ use case phân rã Thuê tủ.....	10
2.2.3 Biểu đồ use case phân rã Đóng mở tủ.....	11
2.2.4 Biểu đồ use case phân rã Quản lý thông tin cá nhân.....	12
2.2.5 Biểu đồ use case phân rã Quản lý tủ và ngăn.....	13
2.2.6 Biểu đồ use case phân rã Quản lý gói giá.....	14
2.2.7 Biểu đồ use case phân rã Giám sát và xử lý sự cố	15
2.2.8 Quy trình nghiệp vụ	15
2.3 Đặc tả chức năng	21
2.3.1 Đặc tả use case Thuê qua ứng dụng.....	21
2.3.2 Đặc tả use case Mở/đóng tủ thông qua PIN hoặc QR	21
2.3.3 Đặc tả use case Ghép nối tủ mới.....	22
2.3.4 Đặc tả use case Thanh toán	23
2.3.5 Đặc tả use case Quản lý tủ và ngăn	24

2.3.6 Đặc tả use case Giám sát và xử lý sự cố.....	25
2.4 Yêu cầu phi chức năng	26
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	27
3.1 Thiết bị phần cứng	27
3.1.1 Raspberry Pi.....	27
3.1.2 Màn hình cảm ứng	28
3.1.3 Module mở rộng MCP23017.....	29
3.1.4 Relay và khóa điện từ	29
3.1.5 Camera	30
3.2 Giao thức giao tiếp.....	31
3.2.1 HTTP.....	31
3.2.2 Giao thức MQTT	32
3.2.3 Chuẩn giao tiếp I2C	33
3.3 Công nghệ phần mềm	34
3.3.1 Node.js, Express và Prisma cho máy chủ backend	34
3.3.2 Python và PySide6 cho ứng dụng tại tủ	35
3.3.3 React và Socket.io cho trang quản trị	36
3.3.4 React Native cho ứng dụng di động.....	37
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	38
4.1 Kiến trúc hệ thống	38
4.1.1 Kiến trúc tổng quan.....	38
4.1.2 Kiến trúc phần cứng	40
4.1.3 Kiến trúc phần mềm.....	42
4.1.4 Kiến trúc chi tiết backend	43

4.2 Thiết kế chi tiết.....	45
4.2.1 Thiết kế giao diện	45
4.2.2 Thiết kế lớp	47
4.2.3 Thiết kế cơ sở dữ liệu	52
4.3 ‘Xây dựng ứng dụng’	57
4.3.1 Thư viện và công cụ sử dụng.....	57
4.3.2 Minh họa các chức năng chính	61
4.3.3 Mô hình tử gửi đồ	66
4.4 Kiểm thử.....	67
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT.....	71
5.1 Mở tử an toàn bằng mã PIN và mã QR.....	71
5.1.1 Giới thiệu bài toán.....	71
5.1.2 Giải pháp	71
5.1.3 Kết quả	72
5.2 Đồng bộ trạng thái tử theo thời gian thực	72
5.2.1 Giới thiệu bài toán.....	72
5.2.2 Giải pháp	73
5.2.3 Kết quả	73
5.3 Tự động ghép nối kiosk với hệ thống	73
5.3.1 Giới thiệu bài toán.....	73
5.3.2 Giải pháp	74
5.3.3 Kết quả	74
5.4 Tự động thu hồi tử khi hết hạn thuê.....	74
5.4.1 Giới thiệu bài toán.....	74
5.4.2 Giải pháp	75
5.4.3 Kết quả	75

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	76
6.1 Kết luận	76
6.2 Hướng phát triển.....	77
TÀI LIỆU THAM KHẢO	78
A. PHỤ LỤC	79
A.1 Khai báo sử dụng công cụ trí tuệ nhân tạo	79

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống	8
Hình 2.2	Biểu đồ use case phân rã chức năng Thuê tủ	10
Hình 2.3	Biểu đồ use case phân rã chức năng Đóng mở tủ	11
Hình 2.4	Biểu đồ use case phân rã chức năng Quản lý thông tin cá nhân	12
Hình 2.5	Biểu đồ use case phân rã chức năng Quản lý tủ và ngăn	13
Hình 2.6	Biểu đồ use case phân rã chức năng Quản lý gói giá	14
Hình 2.7	Biểu đồ use case phân rã chức năng Giám sát và xử lý sự cố	15
Hình 2.8	Biểu đồ hoạt động quy trình tạo đơn thuê tủ	17
Hình 2.9	Biểu đồ hoạt động quy trình sử dụng tủ	18
Hình 2.10	Biểu đồ hoạt động quy trình ghép nối tủ mới	20
Hình 3.1	Raspberry Pi 3 Model B (Nguồn: raspberrypi.com)	27
Hình 3.2	Màn hình cảm ứng DSI 5 inch (Nguồn: waveshare)	28
Hình 3.3	Module mở rộng GPIO MCP23017 (Nguồn: nshopvn.com)	29
Hình 3.4	Module relay 5V (Nguồn: thegioiic.com)	30
Hình 3.5	Khóa điện từ XG-07C 5V (Nguồn: lktronics.com)	30
Hình 3.6	Camera CSI cho Raspberry Pi (Nguồn: raspberrypi.com)	31
Hình 3.7	Mô hình publish/subscribe của MQTT (Nguồn: 200Lab)	33
Hình 3.8	Sơ đồ bus I2C với một master và nhiều slave	34
Hình 3.9	Tổng quan công nghệ phía backend	35
Hình 3.10	So sánh cơ chế giao tiếp giữa HTTP và WebSocket	37
Hình 4.1	Mô hình kiến trúc Client-Server	38
Hình 4.2	Kiến trúc tổng quan hệ thống	39
Hình 4.3	Sơ đồ khối hệ thống OmniBox	40
Hình 4.4	Sơ đồ nguyên lý mạch điều khiển khóa tủ	41
Hình 4.5	Kiến trúc CSR trong máy chủ backend	43
Hình 4.6	Sơ đồ chi tiết các gói trong backend	44
Hình 4.7	Wireframe màn hình chính trên kiosk	45
Hình 4.8	Wireframe nhập mã PIN trên kiosk	45
Hình 4.9	Wireframe quét mã QR trên kiosk	45
Hình 4.10	Wireframe màn hình trang chủ trên ứng dụng di động	46
Hình 4.11	Wireframe màn hình Dashboard quản trị viên	47
Hình 4.12	Thiết kế lớp LockerService	48
Hình 4.13	Thiết kế lớp RentalService	48
Hình 4.14	Thiết kế lớp PaymentService	49

Hình 4.15	Biểu đồ trình tự luồng thuê tủ qua ứng dụng di động	50
Hình 4.16	Biểu đồ trình tự luồng mở khoá tại kiosk	51
Hình 4.17	Biểu đồ thực thể liên kết của cơ sở dữ liệu	52
Hình 4.18	Đăng nhập	61
Hình 4.19	Đăng ký tài khoản	61
Hình 4.20	Trang chủ	61
Hình 4.21	Chọn kích thước tủ	62
Hình 4.22	Chọn gói thuê	62
Hình 4.23	Thanh toán	62
Hình 4.24	Thuê tủ thành công	63
Hình 4.25	Chi tiết phiên thuê	63
Hình 4.26	Màn hình chính kiosk	64
Hình 4.27	Nhập mã PIN tại kiosk	64
Hình 4.28	Quét mã QR tại kiosk	64
Hình 4.29	Thanh toán tại kiosk	64
Hình 4.30	Thuê tủ thành công tại kiosk	64
Hình 4.31	Mở khóa thành công tại kiosk	64
Hình 4.32	Dashboard quản trị	65
Hình 4.33	Quản lý tủ và ngăn tủ	65
Hình 4.34	Quản lý lượt thuê	66
Hình 4.35	Mô hình phần cứng của thiết bị tủ ở trạng thái ban đầu	66
Hình 4.36	Mô hình phần cứng của thiết bị tủ ở trạng thái mở ngăn	67

DANH MỤC BẢNG BIỂU

Bảng 2.1	Đặc tả use case Thuê qua ứng dụng	21
Bảng 2.2	Đặc tả use case Mở/đóng tủ thông qua PIN hoặc QR	22
Bảng 2.3	Đặc tả use case Ghép nối tủ mới	23
Bảng 2.4	Đặc tả use case Thanh toán	24
Bảng 2.5	Đặc tả use case Quản lý tủ và ngăn	25
Bảng 2.6	Đặc tả use case Giám sát và xử lý sự cố	25
Bảng 4.1	Cấu tạo bảng admins	52
Bảng 4.2	Cấu tạo bảng users	53
Bảng 4.3	Cấu tạo bảng locations	53
Bảng 4.4	Cấu tạo bảng cabinets	54
Bảng 4.5	Cấu tạo bảng compartments	54
Bảng 4.6	Cấu tạo bảng price_plans	54
Bảng 4.7	Cấu tạo bảng rentals	55
Bảng 4.8	Cấu tạo bảng payments	56
Bảng 4.9	Cấu tạo bảng locker_logs	56
Bảng 4.10	Cấu tạo bảng audit_logs	57
Bảng 4.11	Cấu tạo bảng notifications	57
Bảng 4.12	Danh sách thư viện và công cụ sử dụng ở phía backend	58
Bảng 4.13	Danh sách thư viện và công cụ sử dụng trong ứng dụng Android	59
Bảng 4.14	Danh sách thư viện và công cụ sử dụng trong ứng dụng tại tủ	60
Bảng 4.15	Các kịch bản kiểm thử chức năng của hệ thống	69
Bảng 4.16	Kết quả kiểm thử trên các thành phần của hệ thống	70
Bảng A.1	Khai báo sử dụng các công cụ trí tuệ nhân tạo trong đồ án	79

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

API	Giao diện lập trình ứng dụng (Application Programming Interface)
CSR	Mô hình kiến trúc Controller-Service-Repository
ERD	Biểu đồ thực thể liên kết dùng trong thiết kế cơ sở dữ liệu (Entity Relationship Diagram)
GPIO	Các chân vào/ra đa dụng trên thiết bị nhúng (General-Purpose Input/Output)
HTTP	Giao thức truyền tải siêu văn bản (HyperText Transfer Protocol)
IoT	Internet vạn vật (Internet of Things)
JSON	Định dạng trao đổi dữ liệu dạng văn bản (JavaScript Object Notation)
JWT	Chuẩn token dùng để xác thực và phân quyền (JSON Web Token)
MQTT	Giao thức truyền thông nhẹ theo mô hình publish/subscribe, thường dùng trong IoT
ORM	Kỹ thuật ánh xạ đối tượng với cơ sở dữ liệu quan hệ (Object-Relational Mapping)
REST	Kiểu kiến trúc giao tiếp cho hệ thống web (Representational State Transfer)
SQL	Ngôn ngữ truy vấn có cấu trúc dùng để thao tác với cơ sở dữ liệu quan hệ (Structured Query Language)
SSE	Cơ chế truyền dữ liệu một chiều từ server tới client (Server-Sent Events)
UI	Giao diện người dùng (User Interface)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Ai trong chúng ta cũng đã từng rơi vào tình huống không biết để đồ đạc ở đâu cho an toàn. Một sinh viên mang laptop đến trường nhưng không muốn xách theo cả buổi, một hành khách cần rảnh tay trong lúc chờ chuyến bay, hay một cư dân đặt hàng online nhưng không có nhà để nhận — những tình huống như vậy xảy ra hằng ngày, ở khắp nơi, và với rất nhiều người.

Nhu cầu gửi và nhận đồ đạc tại các không gian công cộng vì thế không hề nhỏ. Thế nhưng, cách giải quyết hiện tại ở nhiều nơi vẫn còn khá thủ công và bất tiện. Người gửi phải tìm đến quầy có nhân viên trực, xếp hàng chờ đợi, rồi điền giấy tờ ghi chép. Nếu đến ngoài giờ hành chính, hoặc vào lúc đông người, gần như không có lựa chọn nào khác ngoài việc... tự mang đồ theo.

Những nơi có tủ gửi đồ thì thường dùng sử dụng các chìa khóa truyền thống - nghe có vẻ ổn nhưng thực tế phát sinh khá nhiều rủi ro. Chìa khóa dễ mất, thực sự khó quản lý khi có nhiều người cùng quản lý tủ, người quản lý sẽ không thể biết được là tủ nào đang được sử dụng, tủ nào còn trống, và còn sử dụng đến bao lâu nữa, tủ còn có thể sử dụng được không hay khóa tủ đã hỏng rồi.

Nếu bài toán này được giải quyết tốt, lợi ích sẽ rất thiết thực đối với cả người dùng hay bên quản lý tủ. Người dùng có thể chủ động gửi và lấy đồ bất cứ lúc nào, chủ động thuê tủ không cần nhờ đến hay phụ thuộc vào một cá nhân nào khác. Đơn vị quản lý tủ thì có thể tiết kiệm được nhân lực, và nắm rõ tình hình thực tế của tủ để có thể thực hiện điều chỉnh hay sửa chữa, bảo hành nếu trong trường hợp cần thiết. Đây là bài toán đặt ra cho tất cả các khu vực, các không gian công cộng có nhu cầu lưu trữ tạm thời.

1.2 Mục tiêu và phạm vi đề tài

Nhu cầu gửi đồ tạm thời tại các không gian công cộng là có thật và diễn ra hằng ngày nhưng cách giải quyết hiện tại ở các phần lớn nơi vẫn còn rất thủ công. Tại nhiều trường đại học và thư viện, người dùng muốn gửi đồ thì phải liên hệ với nhân viên quản lý, ghi thông tin vào sổ tay và nhận chìa khóa vật lý. Tại các sân bay hay các sảnh chung cư, tình trạng cũng tương tự hoặc không có dịch vụ gửi đồ, chỉ có quầy trực với giờ hoạt động giới hạn.

Nhận thấy được xu hướng này, một số đơn vị đã bắt đầu thử nghiệm tủ khóa thông minh tại Việt Nam. Điển hình là mô hình SmartBox bên Viettel Post với hệ thống cho phép gửi, nhận hàng và thuê tủ thông qua ứng dụng di động, hoạt động 24/7 không cần nhân viên. Đây là tín hiệu rõ ràng cho thấy thị trường đang có nhu cầu

và người dùng sẵn sàng thay đổi thói quen.

Tuy nhiên, các giải pháp hiện tại vẫn chưa thực sự phù hợp với bài toán gửi đồ cá nhân tạm thời tại các không gian đặc thù như trường học, ký túc xá hay sảnh tòa nhà. Hệ thống bên Viettel Post được xây dựng để phục vụ cho nhu cầu bên đơn vị vận chuyển của họ - nhu cầu giao nhận hàng kết nối với các sàn thương mại điện tử, tích hợp sâu vào hạ tầng của chính họ. Chính vì vậy, việc một đơn vị như đại học hay một chung cư muốn tự triển khai độc lập sẽ không có lựa chọn phù hợp. Bên cạnh đó, người vận hành tại các đơn vị như vậy cũng cần phải cần có công cụ để có thể quản lý tập trung các tủ theo thời gian thực nhưng bên các nền tảng lớn chưa thấy họ cung cấp các công cụ quản lý chi tiết. Nhìn chung, dù xu hướng đang dần hình thành, các hệ thống tủ khóa thông minh phục vụ các nhu cầu gửi đồ cá nhân vẫn vắng bóng tại các trường đại học và ký túc xá ở Việt Nam.

Từ thực trạng trên, đề tài xây dựng một hệ thống tủ khóa thông minh tự phục vụ độc lập - không phụ thuộc vào bất kỳ hệ sinh thái thương mại nào để bất kỳ đơn vị nào cũng có thể tự triển khai và quản lý. Hệ thống tập trung giải quyết đúng bài toán : người dùng cần gửi đồ nhanh, an toàn, không cần nhân viên hỗ trợ; còn bên vận hành thì nắm rõ toàn bộ tình hình theo thời gian thực mà không nhất thiết cần có mặt tại chỗ.

1.3 Định hướng giải pháp

Đề án hướng tới xây dựng một hệ thống tủ khóa thông minh phục vụ nhu cầu gửi đồ nhanh chóng và quản lý được hệ thống tủ tập trung theo thời gian thực. Để xây dựng được hệ thống đó, cần xác định cách tổ chức các thành phần hệ thống với nhau. Một hệ thống tủ khóa thông minh bao gồm nhiều đối tượng tham gia cùng lúc : người dùng sử dụng App trên điện thoại, người dùng sử dụng trực tiếp tại màn hình ở trạm tủ, trang quản trị cho người vận hành và phần cứng tủ khóa nên việc chọn một kiến trúc phù hợp là bước quan trọng đầu tiên.

Sau khi xem xét qua các mô hình đã có từ trước, đề án lựa chọn kiến trúc theo client-server với máy chủ là phần đóng vai trò xử lý toàn bộ nghiệp vụ, các thành phần còn lại sẽ giao tiếp qua REST API. Đây là mô hình phổ biến, tài liệu đầy đủ và phù hợp với quy mô mục tiêu ban đầu của hệ thống. Khác với kiến trúc microservices vốn chỉ thực sự phát huy ưu điểm ở các hệ thống lớn với đội ngũ phát triển đông người, hoặc kiến trúc nguyên khối vốn khó bảo trì và mở rộng khi hệ thống có nhiều thiết bị tham gia.

Về phía phần cứng, giao tiếp giữa tủ khóa và máy chủ được thực hiện qua giao thức MQTT. Đây là giao thức được thiết kế riêng cho các thiết bị nhúng có tài nguyên hạn chế, phù hợp cho Raspberry PI được đặt tại các trạm tủ. Giao thức trên

cho phép máy chủ gửi lệnh mở khóa và nhận lại trạng thái ngăn tử một cách nhanh chóng và ổn định. Trong khi đó, trang quản trị sử dụng socket.io để nhận cập nhật ngay lập tức khi có thay đổi xảy ra, thay vì phải tải lại trang – điều đặc biệt quan trọng khi người vận hành cần theo dõi nhiều tử cùng lúc theo thời gian thực.

Về xác thực người dùng, hai phương thức được lựa chọn là mã QR và mã Pin. Thay vì sử dụng thêm thẻ từ hay vân tay đòi hỏi phải bổ sung thêm phần cứng thì hai phương thức này cần màn hình cảm ứng và phần camera.

Dựa trên các định hướng đó, giải pháp được xây dựng thành bốn thành phần phối hợp với nhau : máy chủ trung tâm xử lý toàn bộ nghiệp vụ, ứng dụng di động cho người dùng thuê và quản lý phiên thuê cá nhân, màn hình gắn trực tiếp tại tử để người dùng thao tác trực tiếp tại tử và trang quản trị cho người vận hành có thể theo dõi hệ thống từ xa. người dùng có thể thuê ngăn tử, thanh toán trực tuyến, mở khóa bằng mã QR hoặc PIN và nhận thông báo tự động trong quá trình sử dụng mà không cần sự can thiệp của nhân viên.

Kết thúc quá trình thực hiện, đã xây dựng được hệ thống có thể chạy thực tế với luồng thuê tử từ đầu đến cuối hoạt động ổn định, quá trình từ lúc người dùng chọn ngăn tử, thanh toán, nhận mã, mở khóa, cho đến khi kết thúc và hệ thống tự động giải phóng ngăn tử. Trang quản trị cho phép người vận hành theo dõi trạng thái từng ngăn theo thời gian thực và xem lịch sử hoạt động mà không cần có mặt tại địa điểm. Đặc biệt hơn, đây là một hệ thống có tính mở - có thể triển khai ở nhiều địa điểm khác nhau bằng cách cấu hình thêm tử mới qua trang quản trị không cần thay đổi cấu trúc hệ thống.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày quá trình khảo sát và phân tích yêu cầu của hệ thống. Từ thực trạng và bài toán đã xác định ở Chương 1, chương này đi sâu vào việc làm rõ các đối tượng người dùng và nhu cầu thực tế của họ, từ đó xây dựng các yêu cầu chức năng và phi chức năng cụ thể cho hệ thống. Chương 2 cũng mô tả các luồng hoạt động chính và đặt nền móng cho các quyết định thiết kế ở các chương tiếp theo.

Chương 3 giới thiệu các cơ sở lý thuyết và công nghệ được sử dụng trong đồ án. Chương này trình bày các khái niệm nền tảng liên quan đến kiến trúc hệ thống,

giao thức giao tiếp giữa các thành phần, cơ chế xác thực và bảo mật, cũng như các công nghệ cụ thể được lựa chọn cho từng thành phần của hệ thống. Đây là cơ sở lý luận quan trọng để người đọc hiểu được các quyết định kỹ thuật được đưa ra trong các chương sau.

Chương 4 trình bày toàn bộ quá trình thiết kế, triển khai và đánh giá hệ thống. Chương này mô tả kiến trúc tổng thể của hệ thống gồm bốn thành phần chính, thiết kế cơ sở dữ liệu, giao diện người dùng và các luồng xử lý nghiệp vụ cốt lõi. Bên cạnh đó, chương này cũng trình bày kết quả triển khai thực tế và đánh giá hệ thống dựa trên các tiêu chí đã đề ra ở Chương 2.

Chương 5 tập trung trình bày các giải pháp và đóng góp kỹ thuật nổi bật nhất của đồ án. Cụ thể, chương này đi sâu vào ba vấn đề kỹ thuật chính mà đồ án đã giải quyết: luồng đồng bộ trạng thái tử theo thời gian thực giữa phần cứng và trang quản trị; hệ thống ghép nối tử tự động với máy chủ ; và cơ chế tự động hết hạn thuê và giải phóng ngăn tử đảm bảo tính nhất quán dữ liệu. Mỗi giải pháp được phân tích từ bài toán đặt ra, cách tiếp cận, đến kết quả đạt được.

Chương 6 kết luận toàn bộ đồ án bằng cách nhìn lại những gì đã đạt được so với mục tiêu ban đầu, đồng thời thẳng thắn chỉ ra những hạn chế còn tồn tại. Chương này cũng đề xuất các hướng phát triển tiếp theo cho hệ thống, bao gồm khả năng mở rộng sang các môi trường triển khai khác và các tính năng có thể bổ sung trong tương lai.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

2.1 Khảo sát thực tế

Để xây dựng một hệ thống tủ khóa thông minh thực sự hữu ích, trước hết cần nhìn lại những sản phẩm đã có trên thị trường để nắm rõ nhu cầu của người dùng. Việc này giúp học hỏi được những điểm mạnh, tránh đi vào lối mòn, đồng thời nhận ra các điểm chưa tốt và những khoảng trống có thể khai thác.

2.1.1 Hệ thống SantaPocket

SantaPocket là một dịch vụ tủ giao nhận thông minh đã được ra mắt năm 2020 và mở rộng tại các chung cư ở TP. Hồ Chí Minh. Điểm đáng chú ý nhất là gần như mọi thao tác đều xoay quanh điện thoại: người dùng tải ứng dụng SantaPocket trên iOS hoặc Android, hoặc quét mã QR dán trực tiếp trên tủ để tải nhanh.

Ứng dụng cung cấp ba chức năng chính là gửi hàng, nhận hàng và thuê tủ; trong đó, thuê tủ lại chia thành tự thuê dành cho cá nhân và thuê từ xa dành cho người giao hàng. Luồng sử dụng rất ngắn gọn: người dùng mở ứng dụng, chọn "Thuê" rồi "Tự thuê", sau đó làm theo hướng dẫn để mở ngăn và cất đồ. Về chi phí, mức giá thuê được tính theo thời gian, chỉ từ 500 đồng cho mỗi 30 phút sử dụng một ngăn tủ, và ứng dụng có thêm mục "Lịch sử giao nhận" cho biết người dùng đang có bao nhiêu đơn, kèm số tủ, thời gian thuê và mức phí ước tính.

Đáng chú ý, SantaPocket cũng đã nhắm tới nhóm học sinh, sinh viên. Khi đặt tại các cửa hàng tiện lợi Ministop, hệ thống hướng đến nhiều nhóm người dùng hoạt động 24/7 với quy trình thao tác đơn giản trên ứng dụng. **Một số ưu điểm của hệ thống:**

- Quy trình sử dụng đơn giản, người dùng có thể thao tác trực tiếp trên ứng dụng di động hoặc quét mã QR tại tủ.
- Có khả năng theo dõi lịch sử giao nhận, thời gian thuê và chi phí ước tính ngay trên ứng dụng.
- Mức phí sử dụng linh hoạt theo thời gian, giúp người dùng dễ tiếp cận với chi phí thấp.

Bên cạnh đó vẫn còn tồn tại vài điểm hạn chế:

- Cách tính phí theo từng 30 phút phù hợp với việc gửi đồ vài giờ, nhưng sẽ trở nên bất tiện và tốn kém nếu cần giữ đồ nhiều ngày hoặc suốt một học kỳ, vì không có gói giá theo ngày hay theo tháng.
- "Tự thuê" chỉ là một nhánh phụ bên trong một ứng dụng vốn được thiết kế

chính cho việc giao nhận hàng thương mại điện tử, nên trải nghiệm chưa được tối ưu hoàn toàn cho mục đích cất giữ đồ cá nhân dài ngày.

2.1.2 Hệ thống ViettelPost SmartBox

Ở chiều ngược lại, ViettelPost SmartBox cho thấy một hệ thống được triển khai ở quy mô lớn bởi một doanh nghiệp logistics lớn. Từ cuối năm 2024 đến giữa năm 2025, Viettel Post đã lắp đặt hơn 600 tủ tại nhiều tỉnh thành, chủ yếu ở khu dân cư, tòa nhà văn phòng, trường học và bệnh viện – việc xuất hiện tại trường học cho thấy nhu cầu trong môi trường này là có thật.

Hệ thống hỗ trợ ba hình thức gửi, nhận và thuê tủ; riêng chức năng thuê cho phép khách thuê theo giờ, ngày, tuần hoặc tháng để cất giữ đồ ăn, bưu kiện hay đồ dùng cá nhân. Cách tính giá cũng rõ ràng theo kích thước: dao động từ khoảng 5.000 đến 50.000 đồng mỗi ngày tùy ngăn nhỏ, trung bình hay lớn, và bảng giá được hiển thị ngay khi chọn tủ trên ứng dụng. Về cách mở tủ, người dùng đưa mã QR vào vùng quét của camera phía dưới màn hình tủ, hoặc nhập mã bí mật, rồi kiểm tra thông tin và lấy hàng dưới camera của tủ; các thông báo như mã đơn, địa chỉ tủ, mã mở tủ và thời hạn lưu trữ được gửi qua SMS, Zalo hoặc ứng dụng Viettel Post.

Một số ưu điểm của hệ thống:

- Cách tính giá linh hoạt theo nhiều mốc thời gian, đáp ứng được cả nhu cầu ngắn hạn lẫn dài hạn, cùng quy mô triển khai lớn cho thấy mức độ ổn định và tin cậy đã được kiểm chứng trên thực tế.
- Phương thức mở tủ bằng mã QR hoặc mã bí mật cũng là một giải pháp đơn giản, dễ dùng cho người lớn tuổi lẫn người mới làm quen công nghệ.

Một số điểm hạn chế:

- Đây vẫn là một dịch vụ thương mại tập trung, vận hành và thu phí bởi Viettel Post, nên một đơn vị như nhà trường hay ký túc xá không thể tự kiểm soát giá thuê, dữ liệu người dùng hay tùy biến quy trình cho phù hợp với đặc thù sinh viên.
- Việc phụ thuộc vào một bên thứ ba cũng đồng nghĩa chi phí thuê tủ về lâu dài sẽ cao hơn nếu so với một hệ thống tự đầu tư và tự vận hành.

Tổng hợp lại, SantaPocket và ViettelPost SmartBox đều có nhiều điểm mạnh đáng tham khảo, đặc biệt ở cách tổ chức quy trình gửi, nhận, thuê tủ, cơ chế xác thực bằng mã QR hoặc mã bí mật, cũng như khả năng quản lý trạng thái tủ thông qua nền tảng số. Tuy nhiên, cả hai hệ thống đều được định hướng chủ yếu cho bài toán giao nhận hàng hóa và vận hành theo mô hình dịch vụ thương mại do doanh nghiệp cung cấp. Vì vậy, nhu cầu cất giữ đồ cá nhân tại các điểm công cộng như

trường học, ký túc xá, thư viện, chung cư hoặc khu sinh hoạt chung vẫn còn khoảng trống để xây dựng một giải pháp phù hợp hơn.

Đây là khoảng trống mà đề tài hướng tới giải quyết. Hệ thống được xây dựng theo hướng phục vụ nhu cầu gửi giữ đồ cá nhân tại các điểm công cộng, trong đó người thuê có thể tìm tủ còn trống, chọn gói thuê phù hợp, thanh toán và sử dụng mã PIN hoặc mã QR để mở tủ. Bên cạnh đó, hệ thống cung cấp trang quản trị cho đơn vị chủ quản nhằm theo dõi trạng thái tủ theo thời gian thực, quản lý gói giá, quản lý ngăn tủ và xử lý sự cố khi phát sinh. Cách tiếp cận này giúp hệ thống phù hợp hơn với những đơn vị muốn trực tiếp quản lý và vận hành tủ tại địa điểm của mình, thay vì phụ thuộc hoàn toàn vào một dịch vụ giao nhận bên ngoài.

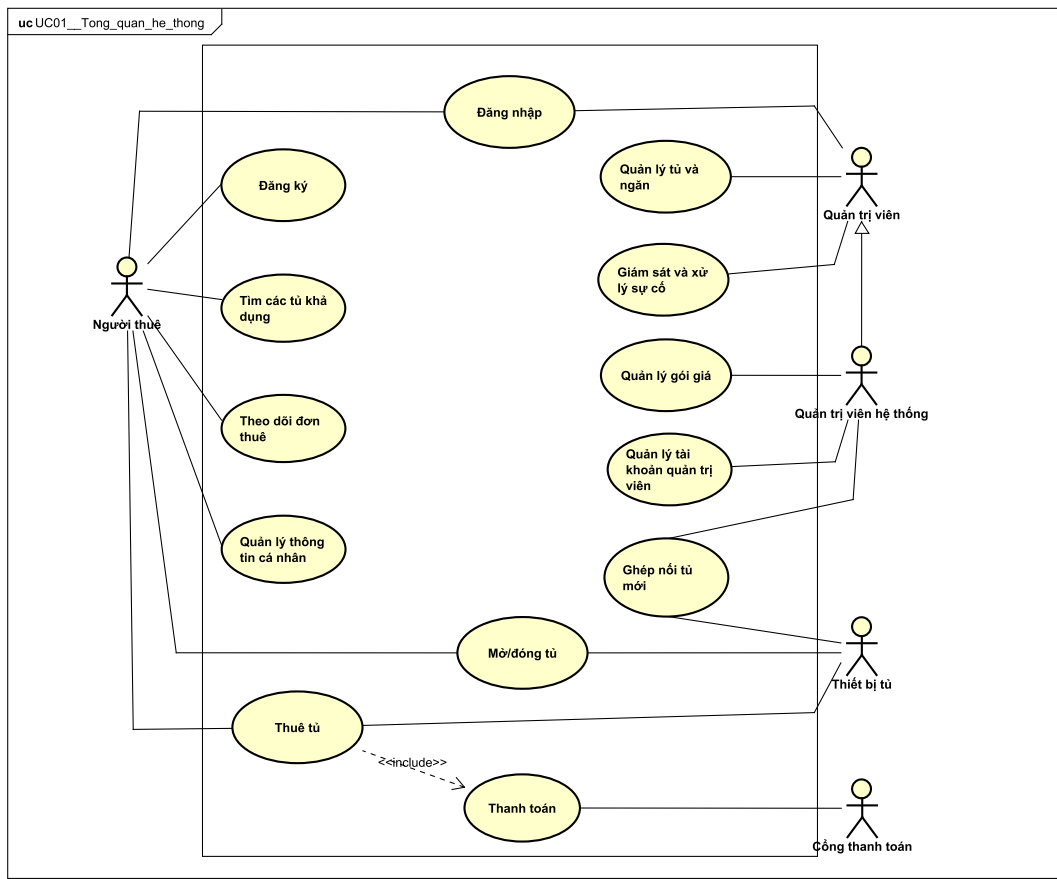
Trên cơ sở khảo sát, hệ thống sẽ được định hướng như sau:

- **Kế thừa từ SantaPocket:** đơn giản hóa quy trình tự thuê tủ, hỗ trợ người dùng thao tác thuận tiện thông qua ứng dụng di động và mã QR tại tủ.
- **Kế thừa từ ViettelPost SmartBox:** hỗ trợ nhiều hình thức thuê theo thời gian, sử dụng mã PIN hoặc mã QR để xác thực và mở tủ, đồng thời quản lý trạng thái tủ thông qua hệ thống phần mềm.
- **Khắc phục hạn chế chung:** hướng tới mô hình do đơn vị chủ quản trực tiếp quản lý và vận hành, giúp chủ động hơn trong việc thiết lập giá thuê, quản lý dữ liệu người dùng và điều chỉnh quy trình theo đặc thù từng địa điểm.
- **Bổ sung năng lực quản trị:** xây dựng trang quản trị cho phép theo dõi trạng thái tủ theo thời gian thực, quản lý tủ, ngăn tủ, gói giá và hỗ trợ xử lý sự cố từ xa.

2.2 Tổng quan chức năng

Phần này trình bày tổng quan các chức năng của hệ thống dưới góc nhìn tương tác giữa người dùng và phần mềm, thông qua các biểu đồ use case. Phần trình bày bắt đầu bằng biểu đồ use case tổng quát (mục 2.2.1), thể hiện toàn bộ tác nhân tham gia hệ thống cùng các chức năng chính ở mức khái quát. Với những chức năng có nhiều cách thực hiện khác nhau như thuê tủ hay mở/đóng tủ, các chức năng đó được phân rã thành các biểu đồ con riêng biệt (mục 2.2.2) để làm rõ từng nhánh. Cuối cùng, hai quy trình nghiệp vụ quan trọng — quy trình thuê và sử dụng tủ, cùng quy trình ghép nối tủ mới — được mô tả dưới dạng biểu đồ hoạt động, thể hiện cách nhiều chức năng kết hợp với nhau để hoàn thành một nghiệp vụ trọn vẹn (mục 2.2.3). Phần này chỉ dừng lại ở mức mô tả tổng quan; đặc tả chi tiết cho từng use case quan trọng được trình bày riêng ở phần 2.3.

2.2.1 Biểu đồ use case tổng quát



Hình 2.1: Biểu đồ use case tổng quát của hệ thống

Mô tả sơ đồ use case:

Các tác nhân có trong sơ đồ:

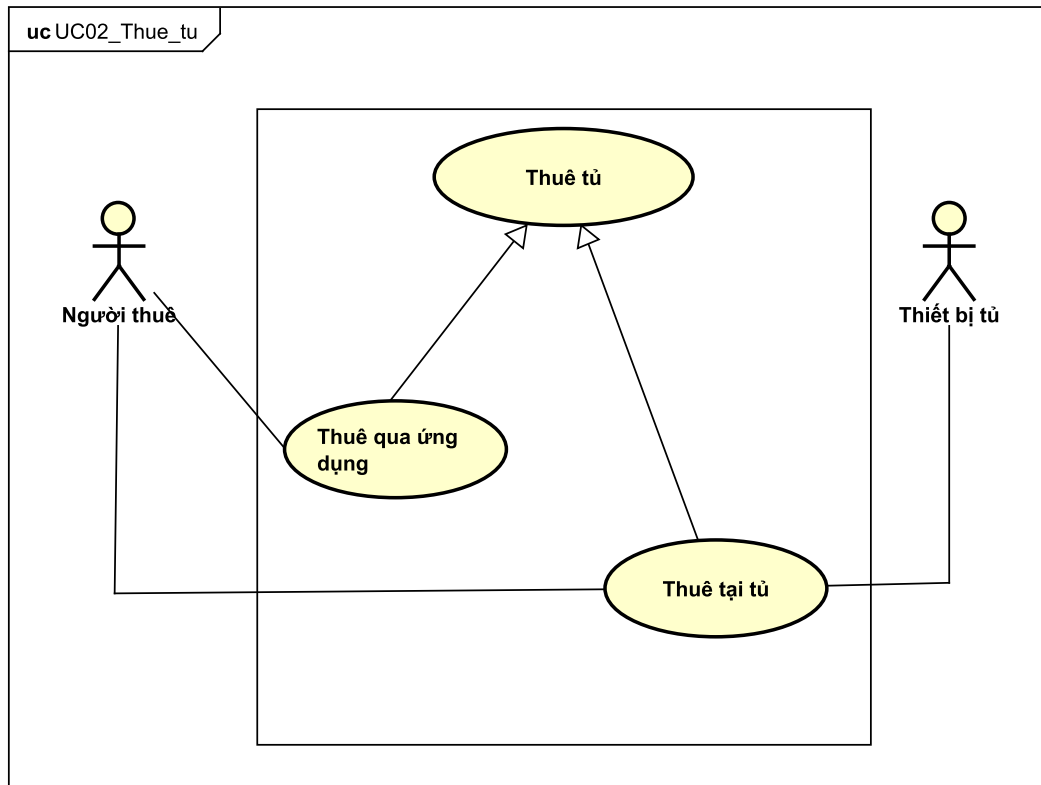
- **Người thuê:** là người sử dụng các dịch vụ của hệ thống, bao gồm gửi đồ, nhận đồ, thuê tủ và mở tủ bằng mã PIN hoặc mã QR.
- **Quản trị viên:** là người đại diện cho đơn vị chủ quản, chịu trách nhiệm vận hành hệ thống tại các địa điểm được phân công, theo dõi trạng thái tủ và xử lý các vấn đề phát sinh.
- **Quản trị viên hệ thống:** là vai trò đặc biệt hóa từ quản trị viên, có đầy đủ các chức năng của quản trị viên thông thường, đồng thời có thêm quyền quản lý tài khoản quản trị viên và cấu hình chung của hệ thống.
- **Thiết bị tử:** là tác nhân hệ thống đại diện cho thiết bị kiosk được lắp đặt tại tủ vật lý. Thành phần này cho phép người dùng thao tác trực tiếp trên màn hình cảm ứng, quét mã QR, nhập mã PIN, đồng thời giao tiếp với máy chủ để đồng bộ dữ liệu và điều khiển phần cứng mở khóa.
- **Cổng thanh toán:** là hệ thống bên ngoài chịu trách nhiệm xử lý các giao dịch

thanh toán trực tuyến và gửi kết quả thanh toán về hệ thống. ““

Các use case chính trong sơ đồ gồm:

- **Đăng ký/Đăng nhập:** người thuê tạo tài khoản hoặc truy cập vào ứng dụng để sử dụng dịch vụ.
- **Tìm tủ khả dụng:** người thuê tìm kiếm và xem các tủ, nhấn còn trống theo địa điểm mong muốn.
- **Thuê tủ:** người thuê tạo một đơn thuê mới, thông qua ứng dụng hoặc trực tiếp tại kiosk.
- **Thanh toán:** người thuê thanh toán phí thuê tủ thông qua cổng thanh toán trực tuyến.
- **Mở/đóng tủ:** người thuê truy cập vào ngăn tủ đã thuê bằng mã PIN hoặc mã QR.
- **Theo dõi đơn thuê:** người thuê xem lịch sử các lần thuê và nhận thông báo khi đơn thuê sắp hết hạn.
- **Trả tủ sớm:** người thuê chủ động kết thúc đơn thuê trước thời hạn để giải phóng ngăn tủ.
- **Quản lý thông tin cá nhân:** người thuê xem và cập nhật thông tin tài khoản của mình.
- **Quản lý tủ và ngăn:** quản trị viên thêm, sửa và theo dõi trạng thái của các tủ và ngăn tủ.
- **Quản lý gói giá:** quản trị viên thiết lập các mức giá thuê theo lần, theo ngày hoặc theo tháng.
- **Giám sát và xử lý sự cố:** quản trị viên theo dõi tình trạng hoạt động của các tủ và xử lý khi có sự cố.
- **Ghép nối tủ mới:** quản trị viên duyệt yêu cầu đăng ký một tủ vừa lắp đặt vào hệ thống.
- **Quản lý tài khoản quản trị viên:** quản trị viên hệ thống tạo và phân quyền cho các quản trị viên khác.

2.2.2 Biểu đồ use case phân rã Thuê tủ



Hình 2.2: Biểu đồ use case phân rã chức năng Thuê tủ

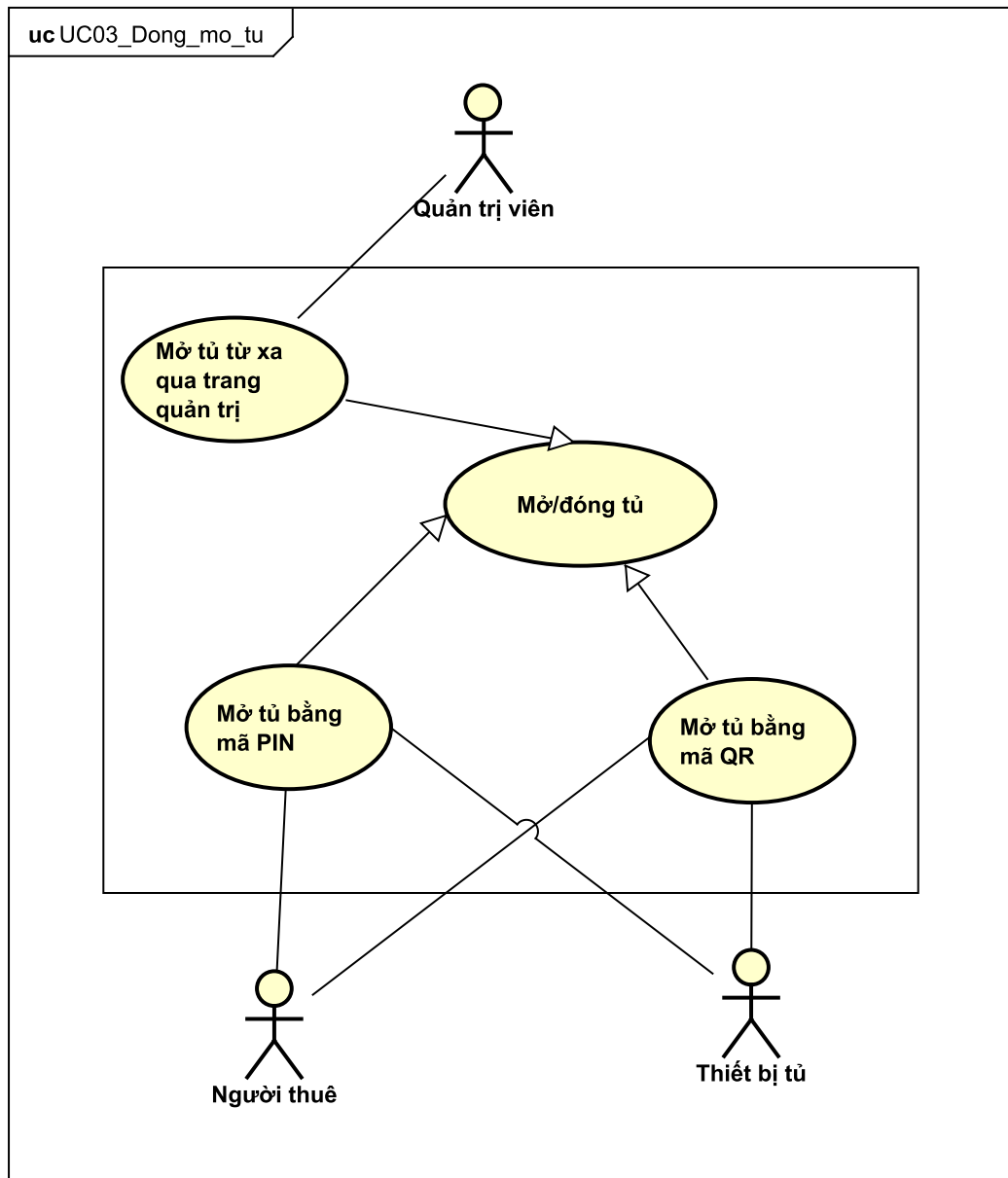
Mô tả sơ đồ use case phân rã:

Tác nhân: Người thuê, thiết bị tủ.

Mô tả các usecase chính trong sơ đồ:

- **Thuê qua ứng dụng:** người thuê đã đăng nhập vào ứng dụng di động, sau đó tìm tủ khả dụng, chọn ngăn tủ còn trống và chọn gói thuê phù hợp. Do người dùng đã có tài khoản và có thể thao tác đầy đủ trên ứng dụng.
- **Thuê tại tủ:** người thuê chưa cần đăng nhập ứng dụng mà thao tác trực tiếp trên thiết bị tủ. Người thuê nhập số điện thoại để hệ thống ghi nhận thông tin liên hệ chọn gói thuê được hiển thị tại tủ.

2.2.3 Biểu đồ use case phân rã Đóng mở tủ



Hình 2.3: Biểu đồ use case phân rã chức năng Đóng mở tủ

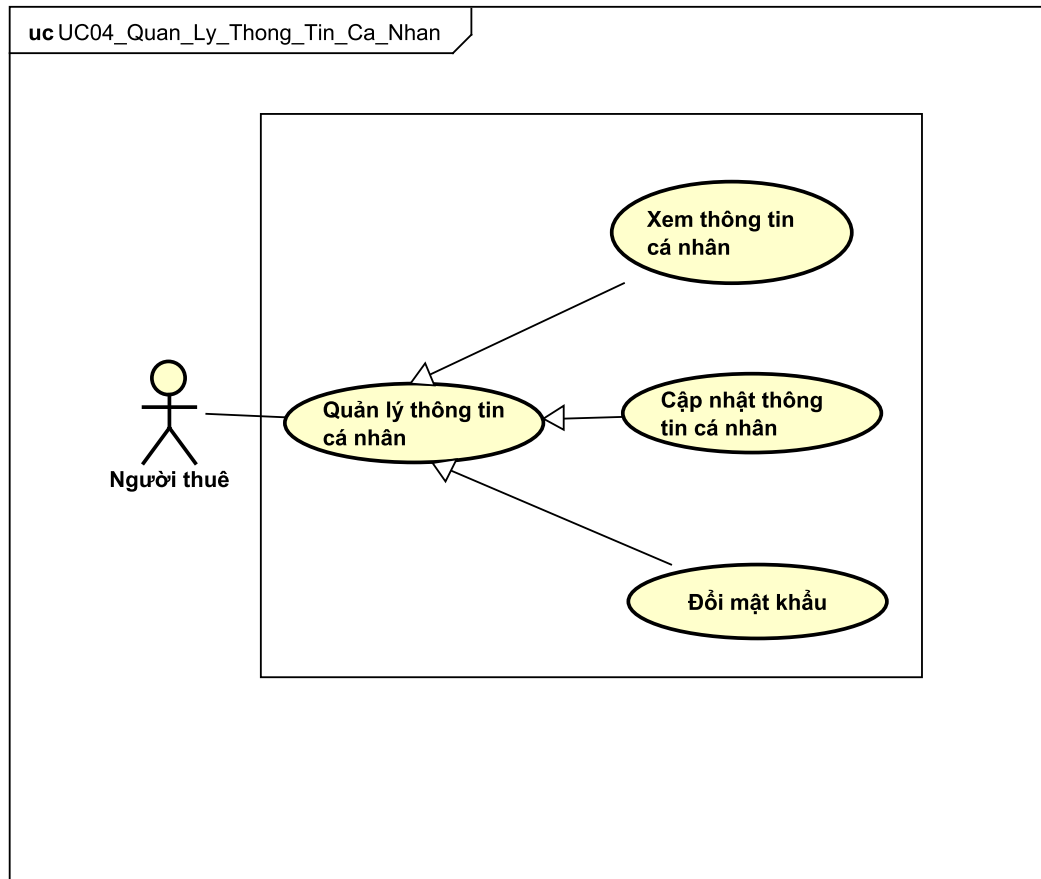
Mô tả biểu đồ use case phân rã

Tác nhân: Người thuê, Quản trị viên, Thiết bị tủ.

Mô tả các use case trong biểu đồ:

- **Mở tủ bằng mã PIN:** Người thuê nhập mã PIN tại tủ để mở ngăn tủ đã thuê.
- **Mở tủ bằng mã QR:** Người thuê quét mã QR tại Tủ để mở ngăn tủ đã thuê.
- **Mở tủ từ xa qua trang quản trị:** Quản trị viên chủ động mở một tủ bất kỳ từ trang quản trị, không cần PIN hay QR, dùng khi cần hỗ trợ người thuê.

2.2.4 Biểu đồ use case phân rã Quản lý thông tin cá nhân



Hình 2.4: Biểu đồ use case phân rã chức năng Quản lý thông tin cá nhân

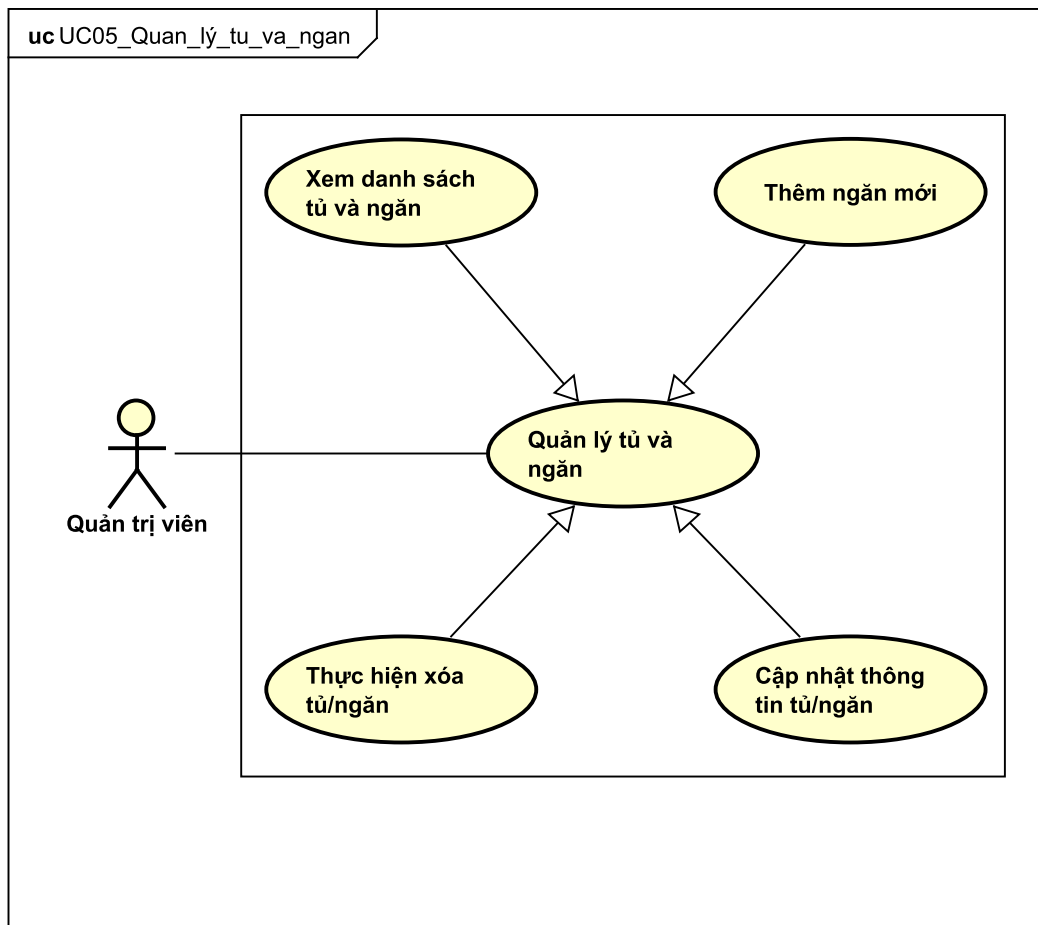
Mô tả biểu đồ use case phân rã

Tác nhân: Người thuê.

Mô tả các use case trong biểu đồ:

- **Xem thông tin cá nhân:** Người thuê xem các thông tin tài khoản của mình như họ tên, số điện thoại, email.
- **Cập nhật thông tin cá nhân:** Người thuê chỉnh sửa các thông tin cá nhân của tài khoản.
- **Đổi mật khẩu:** Người thuê thay đổi mật khẩu đăng nhập để đảm bảo tính bảo mật cho tài khoản.

2.2.5 Biểu đồ use case phân rã Quản lý tủ và ngăn



Hình 2.5: Biểu đồ use case phân rã chức năng Quản lý tủ và ngăn

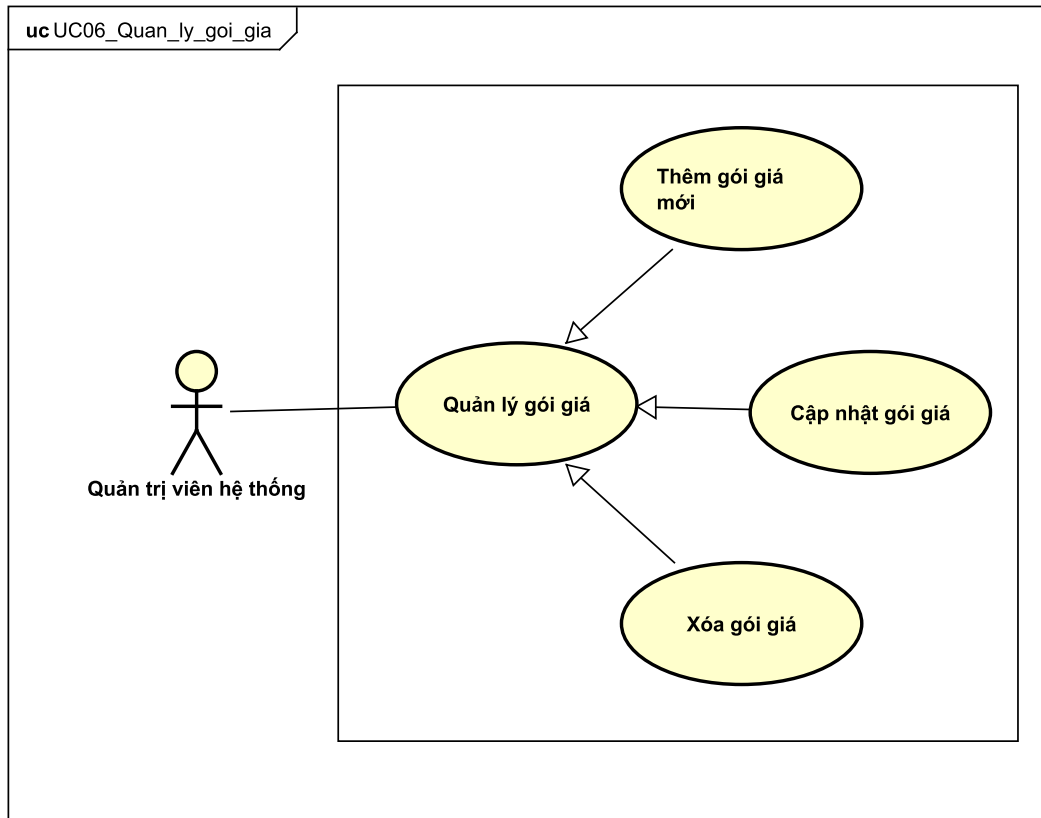
Mô tả biểu đồ use case phân rã

Tác nhân: Quản trị viên.

Mô tả các use case trong biểu đồ:

- **Xem danh sách tủ và ngăn:** Quản trị viên xem thông tin và trạng thái của các tủ, ngăn tủ tại địa điểm mình quản lý.
- **Thêm tủ/ngăn mới:** Quản trị viên thêm tủ hoặc ngăn tủ mới vào hệ thống.
- **Cập nhật thông tin tủ/ngăn:** Quản trị viên chỉnh sửa thông tin của tủ hoặc ngăn tủ đã có.
- **Xóa tủ/ngăn:** Quản trị viên xóa tủ hoặc ngăn tủ không còn sử dụng.

2.2.6 Biểu đồ use case phân rã Quản lý gói giá



Hình 2.6: Biểu đồ use case phân rã chức năng Quản lý gói giá

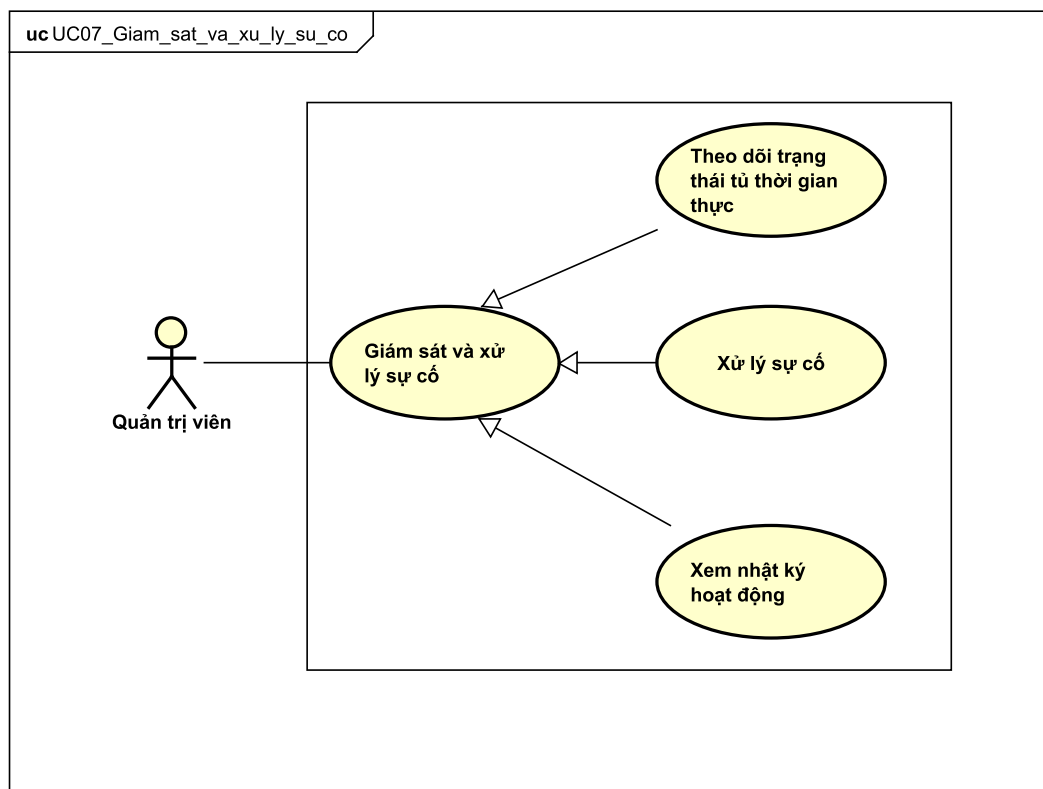
Mô tả biểu đồ use case phân rã

Tác nhân: Quản trị viên hệ thống.

Mô tả các use case trong biểu đồ:

- **Thêm gói giá mới:** Quản trị viên hệ thống tạo gói giá mới (theo lần, theo ngày, theo tháng).
- **Cập nhật gói giá:** Quản trị viên hệ thống chỉnh sửa thông tin của gói giá đã có.
- **Xóa gói giá:** Quản trị viên hệ thống xóa gói giá không còn áp dụng.

2.2.7 Biểu đồ use case phân rã Giám sát và xử lý sự cố



Hình 2.7: Biểu đồ use case phân rã chức năng Giám sát và xử lý sự cố

Mô tả biểu đồ use case phân rã

Tác nhân: Quản trị viên.

Mô tả các use case trong biểu đồ:

- **Theo dõi trạng thái tủ theo thời gian thực:** Quản trị viên xem tình trạng hoạt động (online/offline, đang sử dụng, trống) của các tủ.
- **Xem nhật ký hoạt động:** Quản trị viên xem lịch sử các lần mở/đóng, lỗi và sự kiện đã xảy ra tại từng tủ.
- **Xử lý sự cố:** Quản trị viên can thiệp khi có sự cố như kẹt khóa, mất kết nối, đánh dấu tủ ngừng hoạt động khi cần.

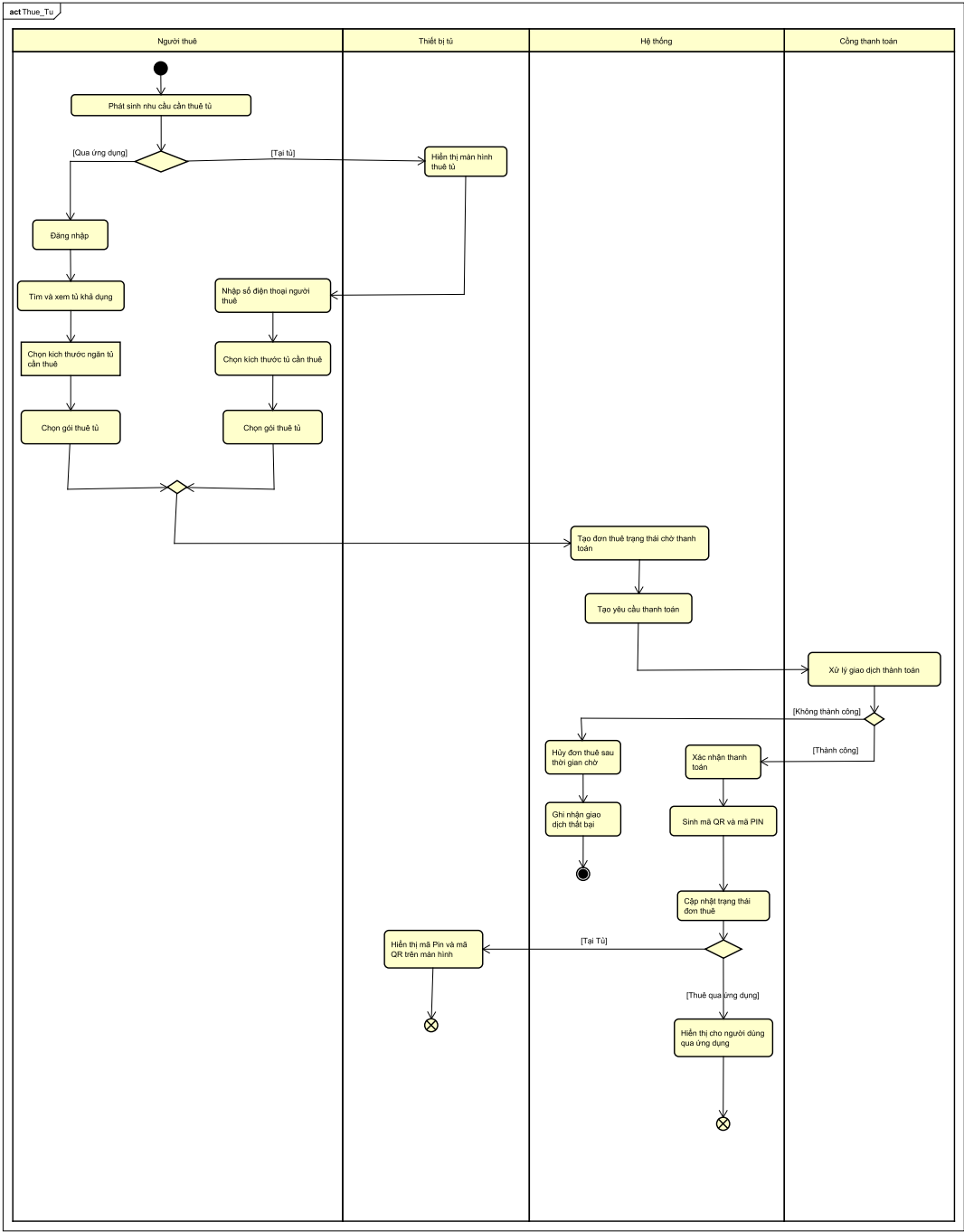
2.2.8 Quy trình nghiệp vụ

Hệ thống có một vài nghiệp vụ chính như: tạo đơn thuê tủ, sử dụng tủ và ghép nối với tủ mới. Dưới đây là mô tả ngắn gọn từng quy trình thông qua sơ đồ hoạt động.

a, Quy trình tạo đơn thuê tủ

Quy trình tạo đơn thuê tủ gồm các bước cơ bản sau:

- **Bước 1:** Chọn kênh thuê tủ, có thể thuê qua ứng dụng hoặc thuê trực tiếp tại tủ.
- **Bước 2:** Chọn kích thước ngăn, sau đó hệ thống tự động gán một ngăn trống phù hợp.
- **Bước 3:** Chọn gói thuê tủ.
- **Bước 4:** Tiến hành thanh toán.
- **Bước 5:** Nếu thanh toán thất bại, đơn thuê tự động bị hủy sau thời gian chờ; nếu thanh toán thành công, hệ thống sinh mã PIN và mã QR.
- **Bước 6:** Hệ thống gửi mã qua ứng dụng hoặc hiển thị trực tiếp trên màn hình thiết bị tủ tùy theo kênh đã chọn.

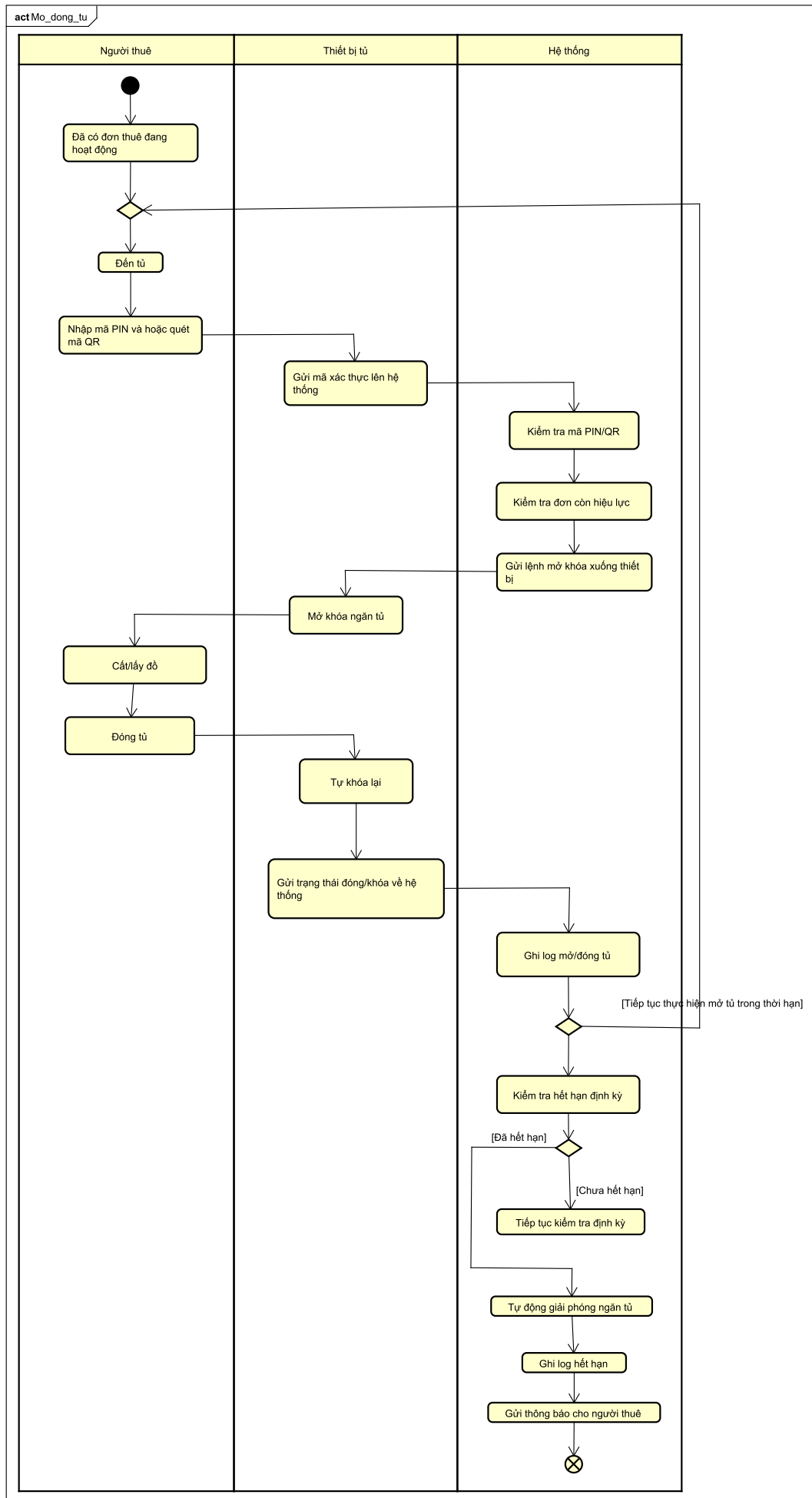


Hình 2.8: Biểu đồ hoạt động quy trình tạo đơn thuê tủ

b, Quy trình sử dụng tủ

Quy trình sử dụng tủ gồm các bước cơ bản sau:

- **Bước 1:** Đến tủ, xác thực bằng PIN hoặc QR.
- **Bước 2:** Mở tủ, cất hoặc lấy đồ, sau đó đóng tủ.
- **Bước 3:** Lặp lại trong suốt thời hạn thuê.
- **Bước 4:** Hệ thống tự động kiểm tra và giải phóng ngăn tủ khi hết hạn.

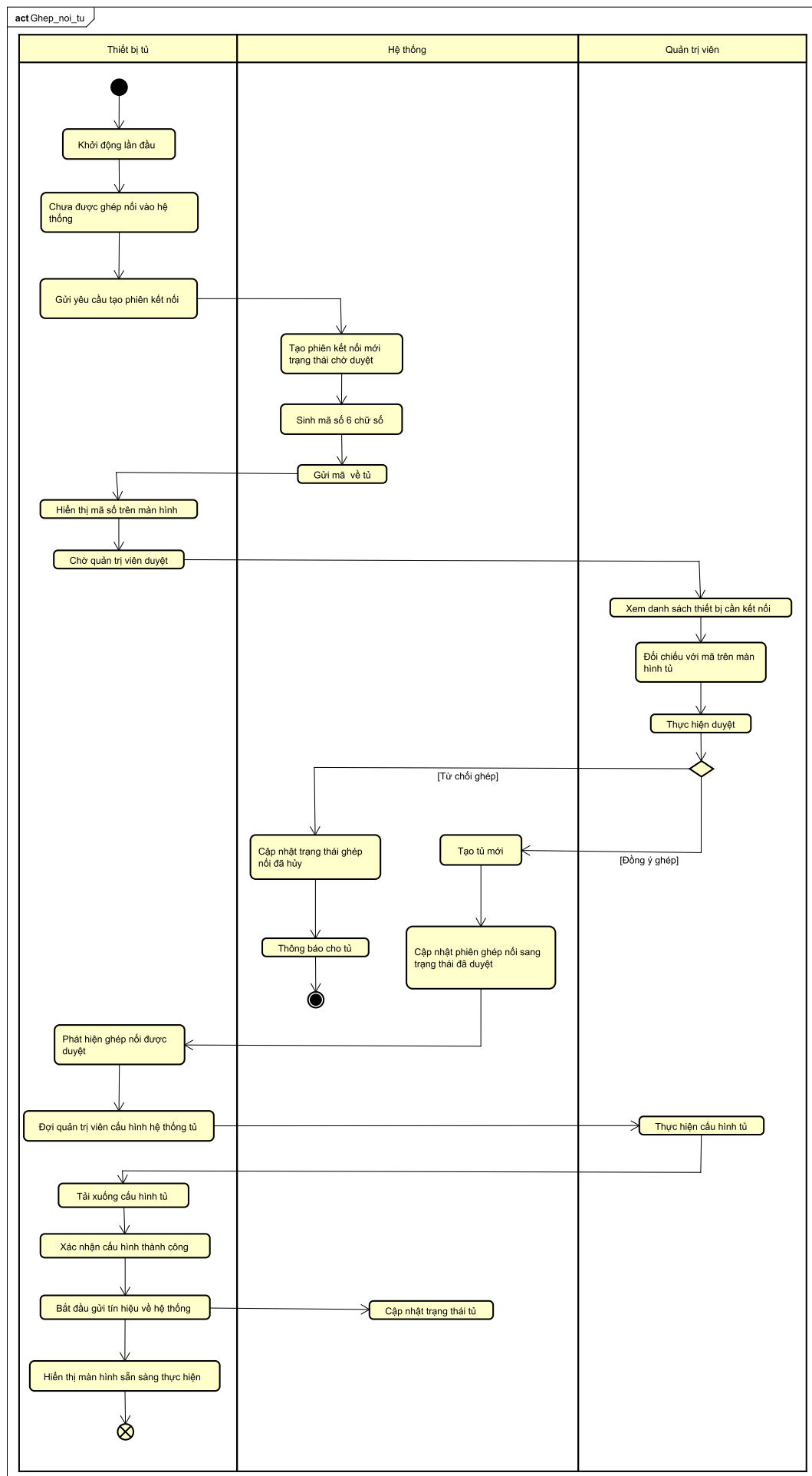


Hình 2.9: Biểu đồ hoạt động quy trình sử dụng tủ

c, Quy trình ghép nối tử mới

Quy trình ghép nối tử mới gồm các bước cơ bản sau:

- **Bước 1:** Kiosk gửi yêu cầu ghép nối và hiển thị mã xác nhận.
- **Bước 2:** Quản trị viên đối chiếu mã và duyệt yêu cầu.
- **Bước 3:** Hệ thống tạo tử mới, cập nhật trạng thái đang cấu hình.
- **Bước 4:** Kiosk tải cấu hình tử về và xác nhận.
- **Bước 5:** Hệ thống cập nhật tử sang trạng thái hoạt động.



Hình 2.10: Biểu đồ hoạt động quy trình ghép nối tử mới

2.3 Đặc tả chức năng

2.3.1 Đặc tả use case Thuê qua ứng dụng

Mã usecase		UC-01
Tên usecase		Thuê qua ứng dụng
Tác nhân		Người thuê
Tiền điều kiện		Người thuê đã đăng nhập thành công vào ứng dụng
STT	Thực hiện bởi	Hành động
1	Người thuê	Chọn địa điểm muốn thuê tủ
2	Hệ thống	Hiển thị danh sách tủ và ngăn còn trống tại địa điểm đã chọn
3	Người thuê	Chọn kích thước ngăn tủ mong muốn
4	Hệ thống	Tự động gán một ngăn tủ còn trống phù hợp với kích thước đã chọn
5	Hệ thống	Hiển thị danh sách các gói thuê tủ khả dụng
6	Người thuê	Chọn gói thuê tủ
7	Hệ thống	Tạo đơn thuê ở trạng thái chờ thanh toán
8	Hệ thống	Tạo yêu cầu thanh toán, chuyển sang use case Thanh toán
9	Hệ thống	Nhận kết quả thanh toán thành công
10	Hệ thống	Sinh mã PIN và mã QR cho ngăn tủ
11	Hệ thống	Cập nhật đơn thuê sang trạng thái hoạt động
12	Hệ thống	Gửi thông báo kèm mã PIN và mã QR cho người thuê
Luồng sự kiện thay thế (ngoại lệ)		
2a	Hệ thống	Hiển thị thông báo không còn tủ hoặc ngăn trống tại địa điểm đã chọn
4a	Hệ thống	Hiển thị thông báo hiện không còn ngăn trống phù hợp với kích thước đã chọn
9a	Hệ thống	Nếu thanh toán thất bại hoặc hết thời gian chờ, tự động hủy đơn thuê và hiển thị thông báo
Hậu điều kiện		Đơn thuê được tạo và chuyển sang trạng thái hoạt động; ngăn tủ được đánh dấu đã có người thuê; người thuê nhận được mã PIN và mã QR để sử dụng.

Bảng 2.1: Đặc tả use case Thuê qua ứng dụng

2.3.2 Đặc tả use case Mở/đóng tủ thông qua PIN hoặc QR

Mã usecase		UC-02
Tên usecase		Mở/đóng tủ thông qua PIN hoặc QR
Tác nhân		Người thuê, Thiết bị tủ
Tiền điều kiện		Người thuê có đơn thuê đang ở trạng thái hoạt động
STT	Thực hiện bởi	Hành động
1	Người thuê	Đến tủ, chọn cách xác thực bằng mã PIN hoặc mã QR
2	Người thuê	Nhập mã PIN tại màn hình kiosk, hoặc quét mã QR
3	Thiết bị tủ	Gửi mã đã nhập hoặc quét lên hệ thống để xác thực
4	Hệ thống	Kiểm tra mã hợp lệ và đơn thuê còn hiệu lực
5	Hệ thống	Gửi lệnh mở khóa xuống thiết bị tủ
6	Thiết bị tủ	Mở khóa ngăn tủ tương ứng
7	Người thuê	Cất hoặc lấy đồ, đóng tủ lại
8	Thiết bị tủ	Phát hiện cửa đã đóng, tự động khóa lại, ghi nhận trạng thái
Luồng sự kiện thay thế (ngoại lệ)		
4a	Hệ thống	Nếu mã PIN hoặc mã QR không hợp lệ, hiển thị thông báo lỗi, từ chối mở khóa
4b	Hệ thống	Nếu đơn thuê đã hết hạn hoặc bị hủy, từ chối mở khóa và hiển thị thông báo
Hậu điều kiện		Ngăn tủ được mở nếu xác thực thành công, hoặc giữ nguyên trạng thái khóa nếu thất bại; sự kiện mở/đóng được ghi vào nhật ký hoạt động.

Bảng 2.2: Đặc tả use case Mở/đóng tủ thông qua PIN hoặc QR

2.3.3 Đặc tả use case Ghép nối tủ mới

Mã usecase		UC-03
Tên usecase		Ghép nối tủ mới
Tác nhân		Quản trị viên, Thiết bị tủ
Tiền điều kiện		Tủ vật lý đã được lắp đặt, kết nối mạng, chưa được ghép nối vào hệ thống
STT	Thực hiện bởi	Hành động
1	Thiết bị tủ	Gửi yêu cầu tạo phiên ghép nối tới hệ thống
2	Hệ thống	Tạo phiên ghép nối, sinh mã QR và mã số gồm 6 chữ số

Tiếp theo trang sau

Bảng 2.3: Ghép nối tử mới (tiếp theo)

STT	Thực hiện bởi	Hành động
3	Thiết bị tử	Hiển thị mã QR và mã số trên màn hình
4	Quản trị viên	Xem danh sách yêu cầu ghép nối đang chờ trên trang quản trị
5	Quản trị viên	Đối chiếu mã hiển thị tại tử với danh sách, duyệt yêu cầu
6	Hệ thống	Tạo tử mới ở trạng thái đang cấu hình, cập nhật phiên ghép nối
7	Thiết bị tử	Phát hiện phiên đã được duyệt, tải cấu hình tử về
8	Thiết bị tử	Xác nhận tải cấu hình thành công
9	Hệ thống	Cập nhật trạng thái tử sang hoạt động
Luồng sự kiện thay thế (ngoại lệ)		
5a	Quản trị viên	Từ chối yêu cầu nếu mã không khớp hoặc không xác định được tử; hệ thống hủy phiên ghép nối
7a	Hệ thống	Nếu thiết bị tử không phản hồi trong thời gian quy định, phiên ghép nối tự động hết hạn
Hậu điều kiện		Tử mới được thêm vào hệ thống ở trạng thái hoạt động, sẵn sàng phục vụ; hoặc phiên ghép nối bị hủy nếu bị từ chối hoặc hết hạn.

Bảng 2.3: Đặc tả use case Ghép nối tử mới

2.3.4 Đặc tả use case Thanh toán

Mã usecase		UC-04
Tên usecase		Thanh toán
Tác nhân		Người thuê, Cổng thanh toán
Tiền điều kiện		Đã có đơn thuê ở trạng thái chờ thanh toán
STT	Thực hiện bởi	Hành động
1	Hệ thống	Tạo yêu cầu thanh toán với số tiền tương ứng gói đã chọn
2	Hệ thống	Gửi yêu cầu tới Cổng thanh toán, nhận về mã QR hoặc liên kết thanh toán
3	Người thuê	Quét mã QR hoặc nhấp liên kết để thanh toán qua ứng dụng ngân hàng hoặc ví điện tử
4	Cổng thanh toán	Xử lý giao dịch

Tiếp theo trang sau

Bảng 2.4: Thanh toán (tiếp theo)

STT	Thực hiện bởi	Hành động
5	Cổng thanh toán	Gửi kết quả giao dịch về hệ thống
6	Hệ thống	Cập nhật trạng thái đơn thuê sang đã thanh toán
7	Hệ thống	Tiếp tục luồng sinh mã PIN và mã QR cho đơn thuê
Luồng sự kiện thay thế (ngoại lệ)		
4a	Cổng thanh toán	Nếu giao dịch thất bại, gửi kết quả thất bại về hệ thống
6a	Hệ thống	Nếu hết thời gian chờ mà chưa nhận được kết quả thanh toán, tự động hủy đơn thuê
Hậu điều kiện		Đơn thuê chuyển sang trạng thái đã thanh toán nếu giao dịch thành công; hoặc bị hủy nếu thất bại hoặc hết thời gian chờ.

Bảng 2.4: Đặc tả use case Thanh toán

2.3.5 Đặc tả use case Quản lý tủ và ngăn

Mã usecase		UC-05
Tên usecase		Quản lý tủ và ngăn
Tác nhân		Quản trị viên
Tiền điều kiện		Quản trị viên đã đăng nhập thành công vào trang quản trị
STT	Thực hiện bởi	Hành động
1	Quản trị viên	Chọn mục Quản lý tủ trên trang quản trị
2	Hệ thống	Hiển thị danh sách tủ và ngăn tủ theo địa điểm được giao
3	Quản trị viên	Chọn thêm mới, chỉnh sửa hoặc xóa một tủ hoặc ngăn tủ
4	Hệ thống	Hiển thị biểu mẫu tương ứng hoặc hộp thoại xác nhận
5	Quản trị viên	Nhập hoặc chỉnh sửa thông tin, xác nhận thao tác
6	Hệ thống	Kiểm tra tính hợp lệ của dữ liệu nhập vào
7	Hệ thống	Lưu thay đổi vào cơ sở dữ liệu
8	Hệ thống	Hiển thị thông báo thao tác thành công, cập nhật lại danh sách
Luồng sự kiện thay thế (ngoại lệ)		

Tiếp theo trang sau

Bảng 2.5: Quản lý tủ và ngăn (tiếp theo)

STT	Thực hiện bởi	Hành động
6a	Hệ thống	Nếu dữ liệu không hợp lệ, hiển thị thông báo lỗi và yêu cầu nhập lại
7a	Hệ thống	Nếu xóa một ngăn tủ đang có đơn thuê hoạt động, từ chối thao tác và hiển thị cảnh báo
Hậu điều kiện		Thông tin tủ và ngăn tủ được cập nhật trong hệ thống; danh sách hiển thị phản ánh đúng thay đổi.

Bảng 2.5: Đặc tả use case Quản lý tủ và ngăn

2.3.6 Đặc tả use case Giám sát và xử lý sự cố

Mã usecase		UC-06
Tên usecase		Giám sát và xử lý sự cố
Tác nhân		Quản trị viên
Tiền điều kiện		Quản trị viên đã đăng nhập thành công vào trang quản trị
STT	Thực hiện bởi	Hành động
1	Quản trị viên	Chọn mục Giám sát trên trang quản trị
2	Hệ thống	Hiển thị danh sách tủ kèm trạng thái thời gian thực
3	Quản trị viên	Chọn một tủ để xem chi tiết
4	Hệ thống	Hiển thị nhật ký hoạt động gần nhất của tủ đó
5	Quản trị viên	Phát hiện sự cố, chọn xử lý
6	Hệ thống	Hiển thị các lựa chọn xử lý (mở khóa thủ công, đánh dấu tủ ngừng hoạt động)
7	Quản trị viên	Chọn hành động xử lý phù hợp
8	Hệ thống	Thực hiện hành động, cập nhật trạng thái tủ
9	Hệ thống	Ghi nhận sự cố và cách xử lý vào nhật ký hoạt động
Luồng sự kiện thay thế (ngoại lệ)		
8a	Hệ thống	Nếu thiết bị tủ không phản hồi lệnh xử lý, hiển thị thông báo lỗi và đề xuất liên hệ kỹ thuật
Hậu điều kiện		Trạng thái tủ được cập nhật theo hành động xử lý; sự cố và cách xử lý được lưu vào nhật ký hoạt động.

Bảng 2.6: Đặc tả use case Giám sát và xử lý sự cố

2.4 Yêu cầu phi chức năng

Bên cạnh các chức năng đã trình bày, hệ thống cần đáp ứng một số yêu cầu phi chức năng quan trọng sau :

- **Tính dễ dùng:** Giao diện ứng dụng di động và màn hình kiosk được thiết kế đơn giản, các bước thuê tủ được rút gọn tối đa để người dùng mới làm quen nhanh chóng, đặc biệt tại kiosk nơi không phải ai cũng quen thao tác với màn hình cảm ứng.
- **Hiệu năng:** Trạng thái tủ được đồng bộ theo thời gian thực tối trang quản trị, đảm bảo độ trễ ở mức thấp; thao tác mở tủ bằng PIN hoặc QR phản hồi gần như tức thì sau khi xác thực thành công.
- **Độ tin cậy:** Hệ thống tự động phát hiện khi tủ mất kết nối và cập nhật trạng thái offline; cơ chế tự động hết hạn đảm bảo ngăn tủ luôn được giải phóng đúng thời hạn dù không có sự can thiệp thủ công.
- **Tính an toàn bảo mật:** Mã PIN được mã hóa trước khi lưu trữ, mã QR được ký để chống giả mạo; hệ thống phân quyền rõ ràng giữa Quản trị viên và Quản trị viên hệ thống để giới hạn phạm vi thao tác phù hợp.
- **Yêu cầu về công nghệ sử dụng:** Hệ thống cần được xây dựng theo kiến trúc phân tán, tách biệt rõ ràng giữa phần backend, ứng dụng tại tủ, trang quản trị và ứng dụng di động, đảm bảo khả năng mở rộng và dễ bảo trì về sau. Các công nghệ cụ thể được trình bày chi tiết ở Chương 3.

Trong chương này, nội dung nghiên cứu đã trình bày kết quả khảo sát một số hệ thống tủ khóa thông minh hiện có, qua đó xác định khoảng trống và định hướng giải pháp cho đề tài. Tiếp theo, chương đã giới thiệu tổng quan các chức năng của hệ thống tủ gửi đồ thông minh thông qua các biểu đồ use case, phân rã những chức năng phức tạp và mô tả các quy trình nghiệp vụ quan trọng. Cuối cùng, chương đã đặc tả chi tiết một số use case tiêu biểu cùng các yêu cầu phi chức năng mà hệ thống cần đáp ứng. Trên cơ sở đó, chương tiếp theo sẽ trình bày cơ sở lý thuyết và các công nghệ được sử dụng trong quá trình xây dựng hệ thống.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này trình bày các cơ sở lý thuyết và công nghệ được sử dụng trong quá trình xây dựng hệ thống tử gửi đồ thông minh. Nội dung chương được chia thành ba nhóm chính: các thiết bị phần cứng cấu thành thiết bị tử, các giao thức giao tiếp giữa các thành phần trong hệ thống, và các công nghệ phần mềm được sử dụng để phát triển máy chủ backend, ứng dụng tại tử, trang quản trị và ứng dụng di động.

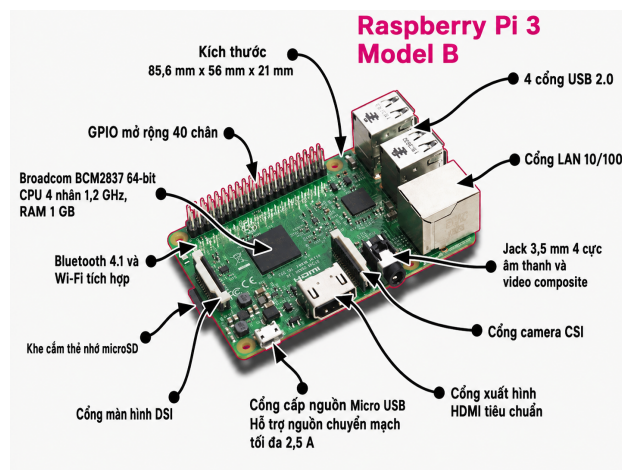
3.1 Thiết bị phần cứng

Hệ thống tử gửi đồ tự động không chỉ là một ứng dụng phần mềm thuần túy mà còn bao gồm một thiết bị vật lý đặt tại điểm cho thuê, chịu trách nhiệm điều khiển trực tiếp các cơ cấu khóa và giao tiếp với người dùng tại chỗ. Vì vậy, việc lựa chọn đúng thiết bị phần cứng có ý nghĩa quan trọng, ảnh hưởng trực tiếp đến độ ổn định, khả năng mở rộng và chi phí triển khai thực tế của hệ thống.

3.1.1 Raspberry Pi

Raspberry Pi là một máy tính đơn bo mạch nhỏ gọn, thường được sử dụng trong các hệ thống nhúng và ứng dụng IoT [1]. Thiết bị này có thể chạy hệ điều hành, kết nối mạng, xử lý dữ liệu và giao tiếp với thiết bị ngoại vi thông qua các cổng kết nối hoặc hệ thống chân GPIO.

Trong đồ án, phần cứng sử dụng máy tính Raspberry Pi 3 Model B. Phiên bản này được trang bị bộ xử lý Broadcom BCM2837 lõi tứ ARM Cortex-A53 64-bit tốc độ 1.2GHz, RAM 1GB, cổng HDMI, các cổng USB, Ethernet 10/100, Wi-Fi 802.11n, Bluetooth 4.1/BLE và hàng chân GPIO 40 chân. Ngoài ra, Raspberry Pi 3 Model B còn hỗ trợ các chuẩn giao tiếp như I2C, SPI, UART và cổng CSI để kết nối camera, phù hợp với những ứng dụng cần kết hợp giao tiếp mạng, hiển thị giao diện và điều khiển thiết bị ngoại vi.



Hình 3.1: Raspberry Pi 3 Model B (Nguồn: raspberrypi.com)

Trong hệ thống tủ gửi đồ thông minh, Raspberry Pi 3 Model B được sử dụng làm bộ điều khiển trung tâm của kiosk. Bo mạch này đảm nhiệm việc chạy ứng dụng tại tủ, hiển thị giao diện trên màn hình cảm ứng, giao tiếp với máy chủ backend, xử lý quét mã QR và gửi lệnh điều khiển tới các khóa tủ thông qua mạch mở rộng I/O và relay. Nhờ đó, hệ thống có đủ khả năng xử lý đồng thời giao diện người dùng và các tác vụ điều khiển phần cứng, trong khi vẫn giữ được kích thước nhỏ gọn phù hợp với thiết bị đặt tại điểm cho thuê.

3.1.2 Màn hình cảm ứng

Màn hình cảm ứng đóng vai trò là giao diện tương tác trực tiếp giữa thiết bị tủ và người dùng tại điểm thuê. Thông qua màn hình này, người dùng có thể xem hướng dẫn, nhập mã PIN và theo dõi trạng thái của từng thao tác. Vì vậy, màn hình cần có kích thước đủ lớn để dễ quan sát, phản hồi chạm nhanh và tương thích trực tiếp với Raspberry Pi mà không cần bổ sung phần cứng trung gian.

Màn hình DSI 5 inch được kết nối với Raspberry Pi qua cổng DSI (Display Serial Interface) tích hợp sẵn trên bo mạch, nhờ đó không cần sử dụng cổng HDMI hoặc bộ chuyển đổi ngoài. Độ phân giải 720×1280 pixel theo chiều dọc phù hợp với giao diện kiosk dạng đứng, cho phép hiển thị các nút bấm kích thước lớn và nội dung hướng dẫn rõ ràng. Tấm nền cảm ứng điện dung hỗ trợ đa điểm chạm, giúp người dùng thao tác tự nhiên tương tự như trên điện thoại thông minh.



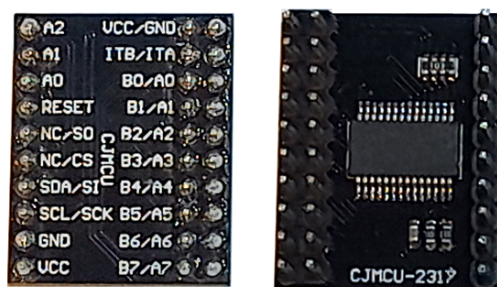
Hình 3.2: Màn hình cảm ứng DSI 5 inch (Nguồn: waveshare)

Màn hình này được lựa chọn vì kết nối DSI trực tiếp giúp giảm số lượng dây dẫn và thiết bị phụ trợ, đồng thời tiêu thụ điện thấp hơn so với màn hình HDMI thông thường. Đây là những đặc điểm phù hợp với thiết bị kiosk cần vận hành liên tục tại điểm công cộng.

3.1.3 Module mở rộng MCP23017

Trong các hệ thống nhúng, số chân điều khiển của vi xử lý trung tâm thường có giới hạn, trong khi số lượng thiết bị ngoại vi cần kết nối có thể lớn hơn số chân sẵn có. Một giải pháp phổ biến là sử dụng mạch mở rộng vào/ra làm lớp trung gian, cho phép vi xử lý giao tiếp với nhiều đường tín hiệu thông qua một bus chung thay vì phải nối trực tiếp từng dây điều khiển riêng lẻ.

MCP23017 là chip mở rộng vào/ra 16-bit của hãng Microchip. Thông qua hai dây giao tiếp theo chuẩn I2C, chip này cung cấp tối đa mười sáu đường tín hiệu độc lập. MCP23017 cũng hỗ trợ cấu hình địa chỉ phần cứng, nhờ đó có thể kết nối nhiều chip trên cùng một bus I2C mà không gây xung đột địa chỉ.

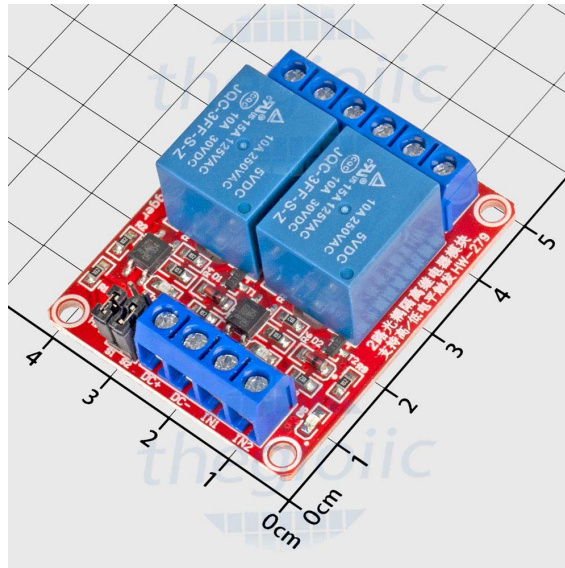


Hình 3.3: Module mở rộng GPIO MCP23017 (Nguồn: nshopvn.com)

Trong hệ thống tủ gửi đồ thông minh, MCP23017 được sử dụng để mở rộng số đường điều khiển từ Raspberry Pi tới các relay khóa tủ. Thay vì nối trực tiếp từng relay vào chân GPIO của Raspberry Pi, các tín hiệu điều khiển được truyền qua MCP23017 chỉ với hai dây SDA và SCL. Cách thiết kế này giúp đơn giản hóa việc đi dây, giảm số chân GPIO cần sử dụng và thuận lợi hơn khi mở rộng thêm ngăn tủ trong tương lai.

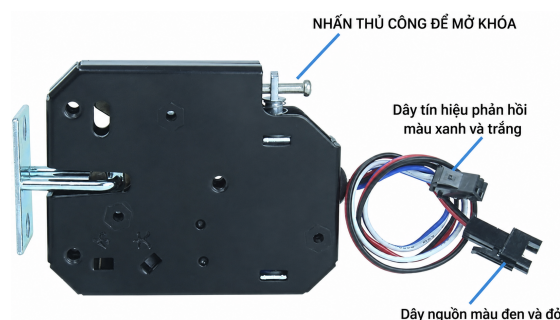
3.1.4 Relay và khóa điện từ

Relay là một loại công tắc được đóng hoặc mở bằng tín hiệu điện thay vì thao tác cơ học trực tiếp. Bên trong relay có một cuộn dây và một tiếp điểm cơ khí; khi có dòng điện nhỏ chạy qua cuộn dây, từ trường sinh ra sẽ hút tiếp điểm, từ đó đóng hoặc ngắt một mạch điện khác có công suất lớn hơn tín hiệu điều khiển ban đầu. Nhờ vậy, relay thường được dùng làm cầu nối giữa mạch điều khiển tín hiệu nhỏ và thiết bị tiêu thụ điện lớn hơn, đồng thời tạo lớp cách ly an toàn giữa hai phía.



Hình 3.4: Module relay 5V (Nguồn: thegioiic.com)

Khóa điện từ XG-07C là loại khóa sử dụng cơ chế nam châm điện để đóng mở chốt cơ học. Khi được cấp nguồn 5V, cuộn dây bên trong tạo ra từ trường kéo chốt vào, qua đó mở khóa ngăn tủ. Khi ngắt nguồn, lò xo bên trong đẩy chốt trở lại vị trí đóng. Ngoài ra, XG-07C còn tích hợp một microswitch nhỏ cho phép đọc trạng thái thực tế của chốt. Nhờ tín hiệu phản hồi này, hệ thống có thể biết ngăn tủ đang thực sự mở hay đóng, thay vì chỉ dựa vào lệnh điều khiển đã gửi đi.



Hình 3.5: Khóa điện từ XG-07C 5V (Nguồn: lktronics.com)

Relay và khóa XG-07C được chọn vì phối hợp tốt với nhau trong cùng mức điện áp 5V, để kết nối với tín hiệu xuất ra từ MCP23017. Đặc biệt, tín hiệu phản hồi từ microswitch tích hợp trong XG-07C cho phép hệ thống xác nhận trạng thái thực tế của từng ngăn tủ, tăng độ tin cậy khi vận hành.

3.1.5 Camera

Camera CSI là loại camera kết nối trực tiếp với Raspberry Pi thông qua cổng Camera Serial Interface. So với camera USB, kết nối CSI có ưu điểm là gọn dây, độ trễ thấp và phù hợp hơn với các thiết bị nhúng có không gian lắp đặt hạn chế.

Loại camera này thường được sử dụng trong các ứng dụng cần thu nhận hình ảnh trực tiếp như giám sát, nhận dạng hình ảnh hoặc quét mã trên thiết bị tự phục vụ.

Trong hệ thống tủ gửi đồ thông minh, camera CSI được gắn tại thiết bị tủ để phục vụ chức năng quét mã QR của người dùng. Khi người dùng đưa mã QR vào vùng quét, camera thu nhận hình ảnh và chuyển khung hình về Raspberry Pi để phần mềm kiosk xử lý. Nhờ đó, người dùng có thể thao tác nhận đồ hoặc mở tủ trực tiếp tại kiosk mà không bắt buộc phải nhập mã thủ công trong mọi trường hợp.

Camera CSI cũng phù hợp với thiết kế phần cứng của thiết bị tủ do có kích thước nhỏ, dễ bố trí phía sau mặt trước kiosk và không chiếm dụng cổng USB. Điều này giúp Raspberry Pi vẫn có thể dành các cổng USB cho thiết bị ngoại vi khác nếu cần, đồng thời giữ hệ thống dây kết nối bên trong tủ gọn gàng hơn.



Hình 3.6: Camera CSI cho Raspberry Pi (Nguồn: raspberrypi.com)

3.2 Giao thức giao tiếp

Hệ thống sử dụng hai giao thức giao tiếp chính phục vụ hai mục đích khác nhau: HTTP để các ứng dụng giao tiếp với máy chủ theo mô hình yêu cầu–phản hồi, và MQTT để máy chủ giao tiếp với thiết bị tủ theo mô hình bất đồng bộ phù hợp với môi trường IoT. Ngoài ra, chuẩn I2C được sử dụng ở tầng phần cứng để Raspberry Pi giao tiếp với các chip mở rộng GPIO bên trong thiết bị tủ.

3.2.1 HTTP

HTTP (HyperText Transfer Protocol) là giao thức nền tảng của Web, hoạt động theo mô hình yêu cầu–phản hồi (request–response): phía client gửi một yêu cầu tới server, server xử lý và trả về kết quả tương ứng. Mỗi yêu cầu HTTP mang theo thông tin về hành động cần thực hiện (GET, POST, PUT, DELETE), địa chỉ tài

nguyên cần truy cập, và dữ liệu kèm theo nếu có.

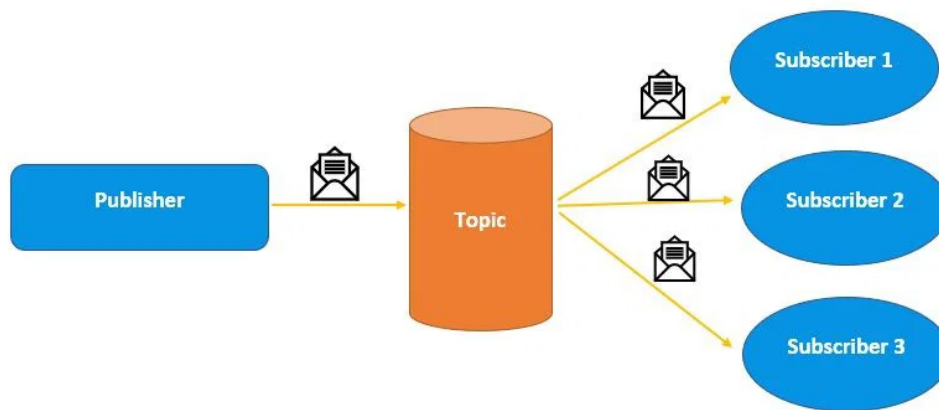
HTTP hoạt động theo cơ chế không trạng thái (stateless), nghĩa là mỗi yêu cầu được xử lý độc lập, server không lưu lại ngữ cảnh của các yêu cầu trước. Điều này giúp server dễ mở rộng vì không cần duy trì thông tin phiên cho từng client, đồng thời các yêu cầu có thể được phân phối tới nhiều server khác nhau mà không cần đồng bộ trạng thái.

Trong hệ thống, HTTP là nền tảng giao tiếp giữa ba loại client (ứng dụng di động, trang quản trị web, và ứng dụng tại tủ) với máy chủ backend. Toàn bộ các thao tác nghiệp vụ như đăng nhập, đặt tủ, xác thực mã PIN hay lấy thông tin lượt thuê đều được thực hiện qua các yêu cầu HTTP.

3.2.2 Giao thức MQTT

MQTT là giao thức truyền thông nhẹ, được chuẩn hóa bởi OASIS và thường được sử dụng trong các hệ thống IoT. Theo đặc tả MQTT 5.0, MQTT hoạt động theo mô hình client-server publish/subscribe, có thiết kế mở, đơn giản và phù hợp với các môi trường hạn chế về tài nguyên như hệ thống máy-máy và Internet of Things [2]. Trong mô hình này, các thiết bị không gửi dữ liệu trực tiếp cho nhau mà trao đổi thông điệp thông qua một thành phần trung gian gọi là broker. Thiết bị gửi dữ liệu được gọi là publisher, thiết bị nhận dữ liệu được gọi là subscriber, còn dữ liệu được phân loại theo các topic.

Ưu điểm của MQTT là dung lượng thông điệp nhỏ, cơ chế truyền nhận đơn giản và khả năng tách rời giữa bên gửi và bên nhận. Nhờ cơ chế publish/subscribe, một thiết bị có thể gửi dữ liệu lên một topic mà không cần biết cụ thể thiết bị nào sẽ nhận dữ liệu đó. Ngược lại, nhiều thiết bị khác nhau có thể đăng ký nhận dữ liệu từ cùng một topic. Cách tổ chức này giúp hệ thống dễ mở rộng, giảm sự phụ thuộc trực tiếp giữa các thành phần và phù hợp với các hệ thống có nhiều thiết bị phân tán.



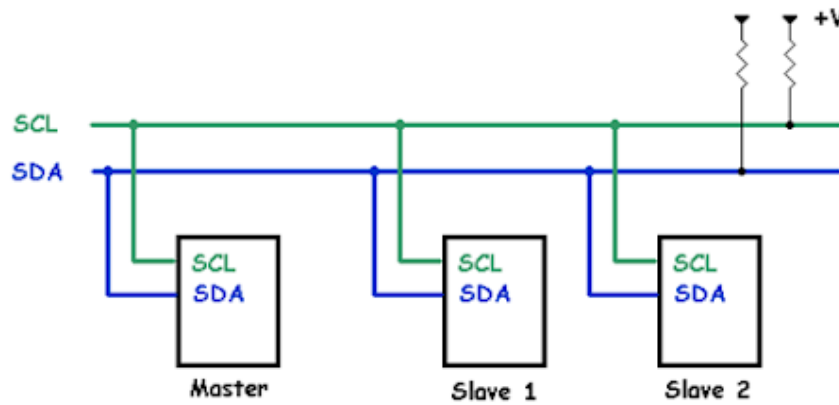
Hình 3.7: Mô hình publish/subscribe của MQTT (Nguồn: 200Lab)

Trong hệ thống tủ gửi đồ, MQTT được sử dụng cho các luồng giao tiếp mang tính thiết bị giữa máy chủ backend và thiết bị tủ, chẳng hạn gửi lệnh mở khóa, nhận trạng thái hoạt động và heartbeat định kỳ. Trong khi đó, các thao tác nghiệp vụ như tạo lượt thuê, lấy danh sách gói giá hoặc kiểm tra thanh toán vẫn được thực hiện qua HTTP/REST. Cách kết hợp này giúp hệ thống tận dụng ưu điểm của HTTP trong các thao tác yêu cầu–phản hồi thông thường, đồng thời sử dụng MQTT cho các tác vụ IoT cần độ trễ thấp và khả năng truyền sự kiện bất đồng bộ

3.2.3 Chuẩn giao tiếp I2C

I2C (Inter-Integrated Circuit) là một chuẩn giao tiếp nối tiếp hai dây được NXP công bố và duy trì, thường dùng để kết nối vi xử lý với các thiết bị ngoại vi trong cùng một bo mạch hoặc giữa các bo mạch gần nhau.

Trên một bus I2C chỉ cần hai dây: SDA (Serial Data) truyền dữ liệu và SCL (Serial Clock) đồng bộ nhịp. Một thiết bị đóng vai trò master chủ động gửi yêu cầu, các thiết bị còn lại đóng vai trò slave và chỉ phản hồi khi được gọi đúng địa chỉ. Nhờ cơ chế định địa chỉ này, nhiều thiết bị khác nhau có thể chia sẻ chung một bus mà không xung đột tín hiệu.



Hình 3.8: Sơ đồ bus I2C với một master và nhiều slave

Trong hệ thống tủ gửi đồ, Raspberry Pi đóng vai trò master, còn các chip MCP23017 đóng vai trò slave trên cùng một bus I2C. Chỉ với hai dây SDA và SCL, Raspberry Pi có thể điều khiển toàn bộ các relay khóa tủ thông qua các chip MCP23017, mỗi chip mang một địa chỉ riêng từ 0x20 đến 0x27.

3.3 Công nghệ phần mềm

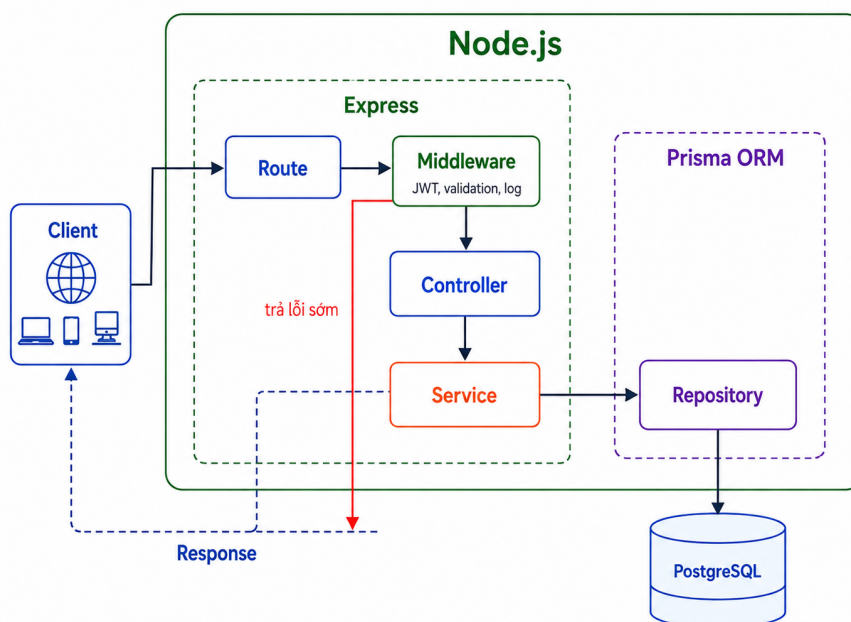
3.3.1 Node.js, Express và Prisma cho máy chủ backend

Node.js là môi trường thực thi JavaScript phía máy chủ, hoạt động theo cơ chế đơn luồng kết hợp với vòng lặp sự kiện [3], thay vì tạo một luồng xử lý riêng cho từng kết nối. Nhờ cơ chế này, một tiến trình Node.js có thể tiếp nhận và xử lý đồng thời nhiều yêu cầu mạng mà không tiêu tốn quá nhiều tài nguyên hệ thống. Trong đồ án, máy chủ backend được xây dựng bằng TypeScript, một phần mở rộng của JavaScript có hỗ trợ kiểu dữ liệu tĩnh. Nhờ đó, các lỗi như truyền sai kiểu dữ liệu giữa các hàm hoặc truy cập nhầm trường dữ liệu không tồn tại có thể được phát hiện sớm trong quá trình phát triển, thay vì chỉ xuất hiện khi hệ thống chạy thử.

Trên nền tảng Node.js, Express được sử dụng để xây dựng các API cho hệ thống [4]. Express là một framework gọn nhẹ, cho phép tổ chức API theo từng nhóm chức năng và cung cấp cơ chế định tuyến để ánh xạ từng đường dẫn URL với phần xử lý tương ứng. Bên cạnh đó, các thành phần trung gian (middleware) được dùng để xử lý những tác vụ chung như kiểm tra phiên đăng nhập, ghi log yêu cầu và xử lý lỗi tập trung.

Ở tầng truy xuất dữ liệu, Prisma được sử dụng thay cho việc viết trực tiếp các câu lệnh SQL [5]. Cấu trúc dữ liệu được mô tả trong một tệp định nghĩa riêng, từ đó Prisma sinh ra các hàm thao tác cơ sở dữ liệu tương ứng và hỗ trợ kiểm tra kiểu dữ liệu. Cách tiếp cận này giúp giảm nguy cơ viết sai câu lệnh truy vấn hoặc nhầm

tên cột, đồng thời bảo đảm các thay đổi trong cấu trúc dữ liệu được áp dụng nhất quán giữa mã nguồn và cơ sở dữ liệu thật. PostgreSQL được chọn làm hệ quản trị cơ sở dữ liệu vì hỗ trợ tốt các ràng buộc dữ liệu chặt chẽ và có khả năng xử lý ổn định khi khối lượng giao dịch tăng dần.



Hình 3.9: Tổng quan công nghệ phía backend

Bên cạnh Node.js và Express, các framework như Spring Boot hoặc Django cũng có thể được sử dụng để xây dựng máy chủ backend. Tuy nhiên, trong phạm vi đề án, Node.js và Express phù hợp hơn do có cấu trúc gọn nhẹ, thời gian khởi tạo dự án nhanh và dễ tích hợp với các thư viện JavaScript/TypeScript ở các thành phần khác của hệ thống. Việc sử dụng chung hệ sinh thái JavaScript/TypeScript cho backend, trang quản trị và một phần ứng dụng di động cũng giúp quá trình phát triển thống nhất hơn, giảm chi phí chuyển đổi giữa nhiều ngôn ngữ lập trình.

3.3.2 Python và PySide6 cho ứng dụng tại tủ

Python là ngôn ngữ lập trình bậc cao, có cú pháp rõ ràng và hệ sinh thái thư viện phong phú, thường được sử dụng trong các hệ thống nhúng, tự động hóa và xử lý dữ liệu. Trên Raspberry Pi, Python được hỗ trợ tốt và có thể dễ dàng kết hợp với các thư viện phục vụ điều khiển phần cứng, giao tiếp mạng, xử lý ảnh và quản lý cấu hình. Vì vậy, Python phù hợp để xây dựng ứng dụng chạy trực tiếp trên thiết bị tủ.

PySide6 là bộ binding chính thức của Qt cho Python, cho phép xây dựng giao diện đồ họa bằng Python [6]. Trong hệ thống tủ gửi đồ thông minh, PySide6 được

sử dụng để phát triển ứng dụng kiosk chạy trên Raspberry Pi. Ứng dụng này hiển thị các màn hình thao tác như chọn chức năng, nhập mã PIN, quét mã QR, chọn gói thuê, thanh toán và thông báo kết quả cho người dùng.

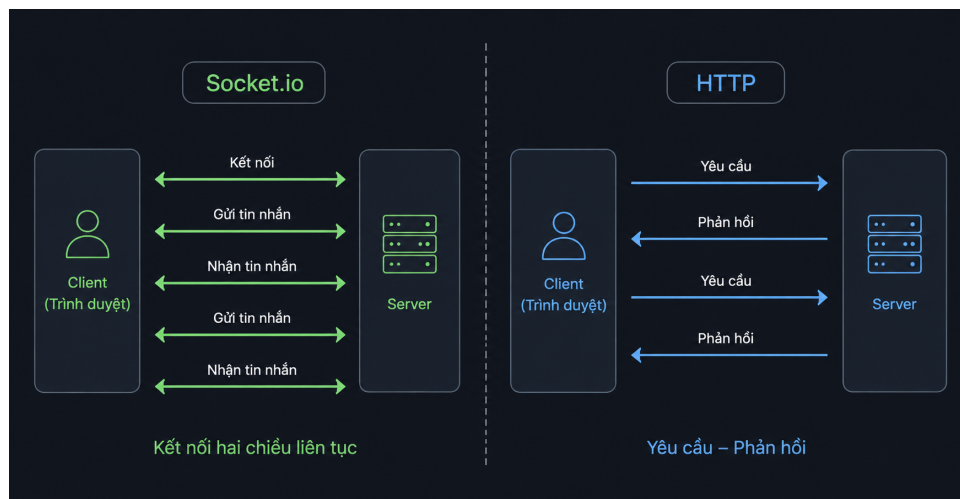
Với đặc thù thiết bị kiosk cần hoạt động ổn định ở chế độ toàn màn hình trên màn hình cảm ứng, PySide6 phù hợp hơn so với giao diện web nhúng trong phạm vi đề án. Lý do là PySide6 cho phép kiểm soát trực tiếp vòng đời màn hình, sự kiện chạm và các tác vụ phần cứng cục bộ. Ngoài ra, ứng dụng PySide6 có thể tích hợp thuận tiện với các thư viện Python khác để quét mã QR, giao tiếp MQTT, gọi REST API và điều khiển phần cứng thông qua GPIO hoặc I2C.

Nhờ đó, ứng dụng tại tủ vừa đảm nhiệm vai trò giao diện tương tác với người dùng, vừa đóng vai trò cầu nối giữa người dùng, máy chủ backend và phần cứng điều khiển khóa tủ.

3.3.3 React và Socket.io cho trang quản trị

Trang quản trị là nơi người quản lý theo dõi tình trạng toàn bộ hệ thống tủ và thực hiện các thao tác quản trị khi cần thiết. Giao diện cần phản ánh trạng thái hệ thống một cách kịp thời mà không yêu cầu người dùng phải tải lại trang thủ công. Để xây dựng giao diện này, đề án sử dụng React, một thư viện JavaScript cho phép chia giao diện thành các component độc lập và có thể tái sử dụng. Khi dữ liệu thay đổi, React cập nhật lại phần giao diện tương ứng dựa trên trạng thái mới, thay vì xây dựng lại toàn bộ trang.

Để trang quản trị nhận biết được sự thay đổi dữ liệu theo thời gian thực, đề án sử dụng thêm Socket.io, một thư viện hỗ trợ kết nối hai chiều giữa trình duyệt và máy chủ dựa trên WebSocket. Khi ứng dụng tại tủ gửi trạng thái mới lên máy chủ qua MQTT, máy chủ xử lý và đẩy sự kiện tương ứng tới trang quản trị qua Socket.io, giúp người quản lý quan sát gần như ngay lập tức các thay đổi xảy ra tại thiết bị vật lý.



Hình 3.10: So sánh cơ chế giao tiếp giữa HTTP và WebSocket

React được lựa chọn vì cộng đồng lớn, tài liệu phong phú và cách tổ chức giao diện theo component phù hợp. Ngoài ra, cách tư duy khi làm việc với React có nhiều điểm tương đồng với React Native dùng cho ứng dụng di động, giúp việc phát triển song song hai giao diện trở nên thống nhất hơn. Đối với Socket.io, lý do chọn thay vì WebSocket thuần là Socket.io đã tích hợp sẵn cơ chế kết nối lại khi mất mạng và quản lý phiên kết nối, giảm khối lượng xử lý ở tầng ứng dụng.

3.3.4 React Native cho ứng dụng di động

Ứng dụng di động là kênh để người dùng đặt tủ từ xa, theo dõi lượt thuê và xem mã QR trước khi tới lấy đồ. Đề án sử dụng React Native là framework cho phép xây dựng ứng dụng di động bằng JavaScript và React [7], nhưng thay vì hiển thị bằng các thành phần web thì React Native chuyển các thành phần giao diện đó thành thành phần gốc (native) của hệ điều hành, nên ứng dụng có giao diện và cảm giác sử dụng tương tự ứng dụng viết hoàn toàn bằng ngôn ngữ gốc của nền tảng.

Ngoài React Native, Flutter và phát triển native bằng Kotlin cũng là những lựa chọn phổ biến để xây dựng ứng dụng di động. Flutter có ưu điểm về khả năng tùy biến giao diện và hiệu năng hiển thị nhờ sử dụng engine riêng, trong khi Kotlin cho phép khai thác sâu các đặc tính của nền tảng Android. Tuy nhiên, trong phạm vi đề án, React Native phù hợp hơn do sử dụng JavaScript/TypeScript và mô hình component tương tự React trên trang quản trị. Cách tiếp cận này giúp thống nhất công nghệ giữa các thành phần giao diện, rút ngắn thời gian phát triển và giảm chi phí chuyển đổi giữa nhiều ngôn ngữ lập trình. Hiện tại, ứng dụng mới được triển khai cho Android, nhưng React Native vẫn để ngỏ khả năng mở rộng sang iOS trong tương lai mà không cần viết lại toàn bộ giao diện từ đầu.

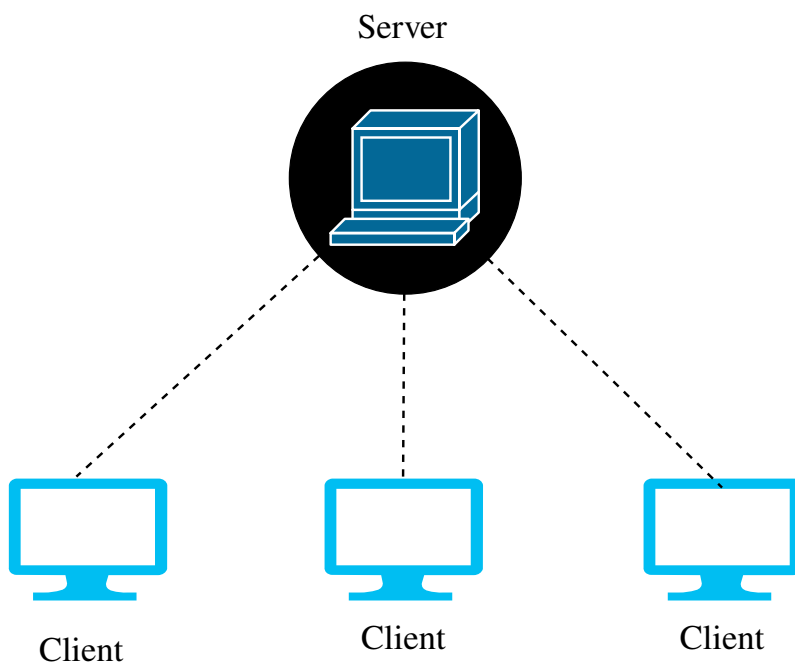
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Ở chương 3, các cơ sở lý thuyết về thiết bị phần cứng, giao thức giao tiếp và công nghệ phần mềm đã được trình bày. Trên cơ sở đó, chương này trình bày quá trình áp dụng các nền tảng đã lựa chọn vào việc thiết kế và xây dựng hệ thống tử gửi đồ thông minh. Nội dung chương bao gồm kiến trúc tổng thể của hệ thống, kiến trúc phần cứng, kiến trúc phần mềm, thiết kế giao diện, thiết kế lớp, thiết kế cơ sở dữ liệu, kết quả xây dựng các thành phần chính và quá trình kiểm thử hệ thống.

4.1 Kiến trúc hệ thống

4.1.1 Kiến trúc tổng quan

Đề án lựa chọn kiến trúc client-server làm nền tảng kiến trúc tổng thể cho hệ thống tử gửi đồ thông minh, vì đây là kiến trúc phù hợp với các hệ thống có nhiều loại client khác nhau cùng truy cập vào một nguồn dữ liệu chung, đồng thời giúp tập trung hoá việc quản lý dữ liệu để đảm bảo tính nhất quán. Mô hình client-server tổng quát được mô tả như Hình 4.1.



Hình 4.1: Mô hình kiến trúc Client-Server

Trong kiến trúc client-server, hệ thống được tổ chức thành hai nhóm vai trò chính là server và client.

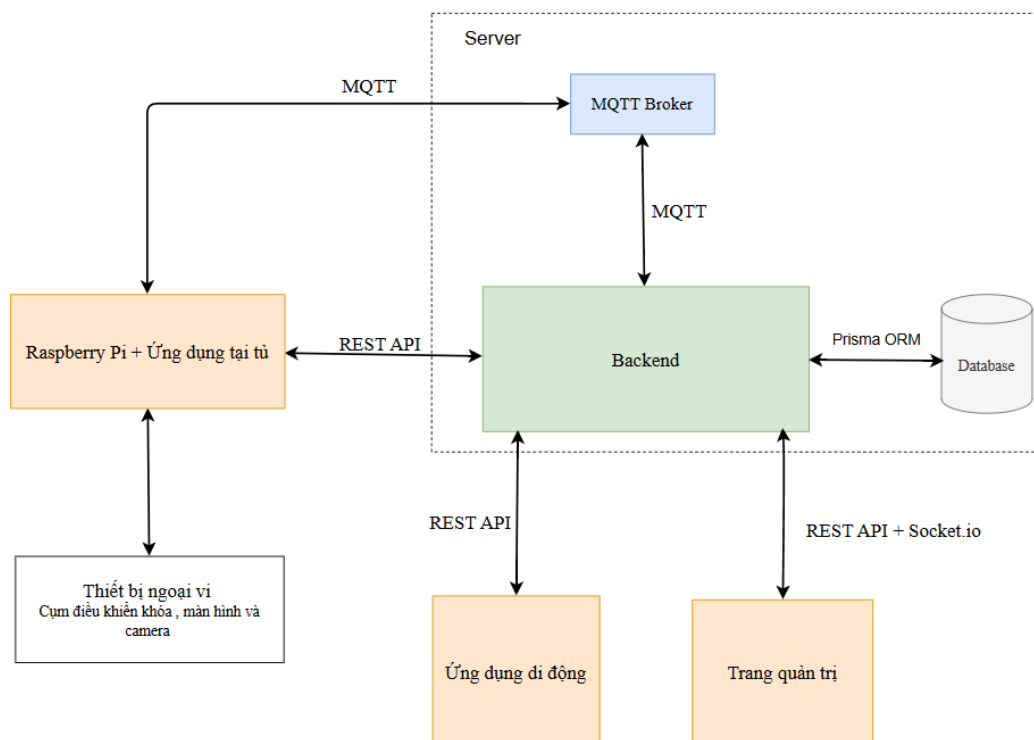
- **Server:** là thành phần trung tâm, chịu trách nhiệm tiếp nhận yêu cầu từ các client, xử lý logic nghiệp vụ và quản lý dữ liệu dùng chung của toàn hệ thống.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Trong hệ thống tử thông minh, máy chủ backend đóng vai trò điều phối chính, giúp ứng dụng di động, trang quản trị và thiết bị tử cùng làm việc trên một nguồn dữ liệu thống nhất. Cách tổ chức này hạn chế tình trạng dữ liệu bị phân tán, đồng thời đảm bảo các thay đổi về người dùng, lượt thuê, thanh toán hoặc trạng thái tử được đồng bộ giữa các thành phần.

- **Client:** là các thành phần gửi yêu cầu đến server để lấy dữ liệu, cập nhật dữ liệu hoặc thực hiện một thao tác nghiệp vụ cụ thể. Mỗi client có giao diện và cách tương tác riêng phù hợp với nền tảng sử dụng, chẳng hạn ứng dụng di động dành cho người thuê, trang web dành cho quản trị viên và ứng dụng tại tủ chạy trên thiết bị tử. Mặc dù đảm nhiệm các vai trò khác nhau, các client này đều giao tiếp với backend để đảm bảo dữ liệu hiển thị và kết quả xử lý được nhất quán.

Áp dụng mô hình trên vào hệ thống tử gửi đồ thông minh, các thành phần được tổ chức cụ thể như Hình 4.2.



Hình 4.2: Kiến trúc tổng quan hệ thống

Phía server là cụm máy chủ trung tâm, gồm máy chủ backend xử lý toàn bộ logic nghiệp vụ, cơ sở dữ liệu PostgreSQL lưu trữ dữ liệu dùng chung, và MQTT

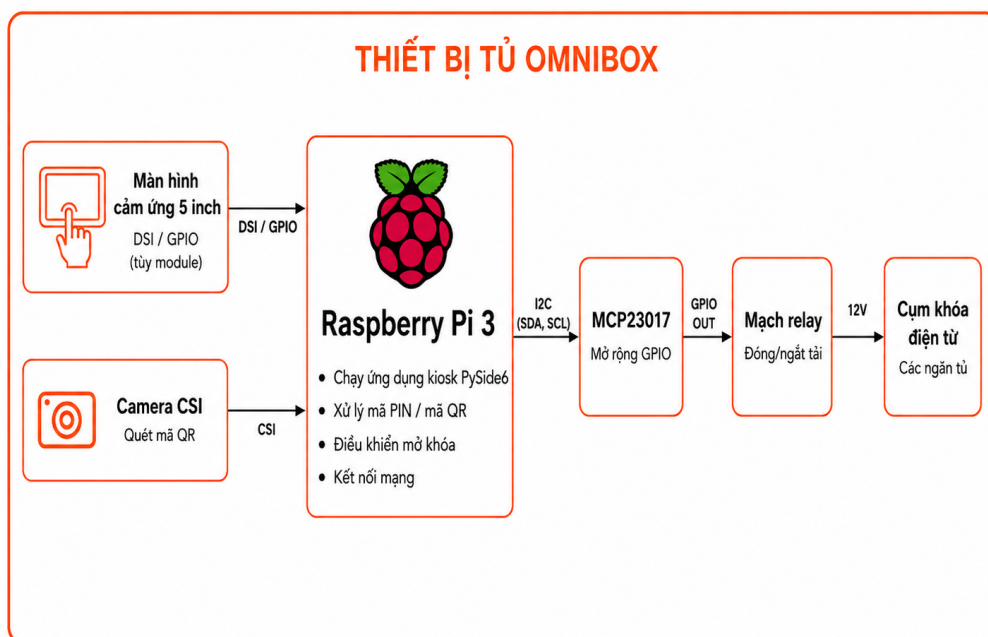
Broker làm trạm trung chuyển tin nhắn với thiết bị tử. Trong đó, backend là thành phần lõi trực tiếp tiếp nhận và xử lý yêu cầu, còn PostgreSQL và MQTT Broker là các thành phần hạ tầng phục vụ cho backend.

Phía client gồm ba thành phần. Ứng dụng di động và trang quản trị web giao tiếp với backend chủ yếu qua REST API; riêng trang quản trị sử dụng thêm Socket.io để nhận cập nhật trạng thái tức thời. Thiết bị tử vừa gọi REST API để thực hiện các thao tác nghiệp vụ, vừa trao đổi tin nhắn qua MQTT Broker để nhận lệnh mở khóa và gửi trạng thái theo thời gian thực.

Bên dưới thiết bị tử là cụm phần cứng vật lý gồm khóa điện tử, camera và màn hình. Cụm phần cứng này không trực tiếp tham gia vào mô hình client-server mà được ứng dụng tại tử điều khiển trực tiếp thông qua chuẩn I2C và các chân GPIO. Chi tiết về phần cứng được trình bày ở mục tiếp theo.

4.1.2 Kiến trúc phần cứng

a, Sơ đồ khối phần cứng



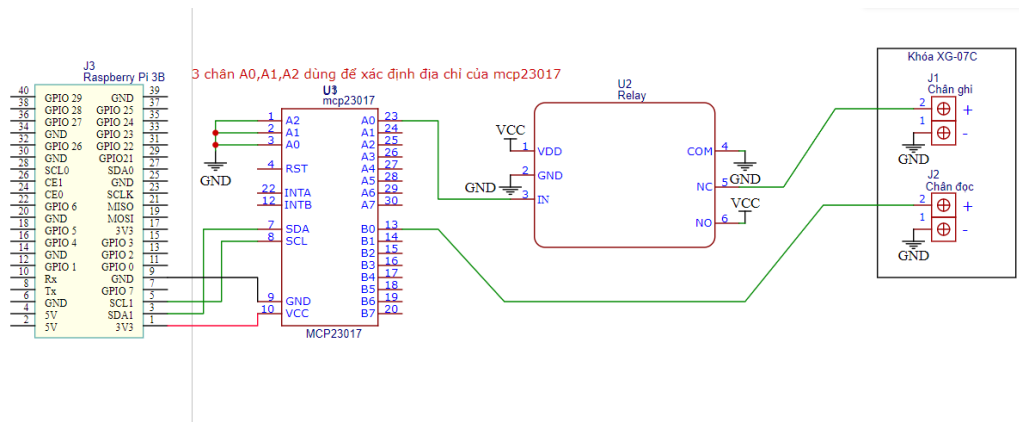
Hình 4.3: Sơ đồ khối hệ thống OmniBox

Hình 4.3 mô tả tổng quan kiến trúc phần cứng của thiết bị tử. Raspberry Pi 3 đóng vai trò bộ xử lý trung tâm, kết nối với màn hình cảm ứng 5 inch qua cổng DSI và camera CSI để phục vụ thao tác hiển thị, nhập liệu và quét mã QR. Đối với khối điều khiển khóa, Raspberry Pi giao tiếp với chip mở rộng GPIO MCP23017 qua

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

chuẩn I2C; MCP23017 sau đó xuất tín hiệu điều khiển đến module relay để đóng ngắt nguồn cấp cho các khóa điện từ tại từng ngăn tủ.

b, Sơ đồ nguyên lý



Hình 4.4: Sơ đồ nguyên lý mạch điều khiển khóa từ

Hình 4.4 trình bày sơ đồ nguyên lý của mạch điều khiển khóa từ. Raspberry Pi 3 giao tiếp với MCP23017 thông qua chuẩn I2C; trong thiết kế này, chip MCP23017 được cấu hình địa chỉ 0x20 bằng cách kéo ba chân A0, A1 và A2 xuống GND.

MCP23017 cho phép cấu hình địa chỉ I2C thông qua ba chân A0, A1 và A2. Mỗi chân có thể đặt ở mức cao (VCC) hoặc mức thấp (GND), tạo ra tối đa tám tổ hợp địa chỉ khác nhau từ 0x20 đến 0x27. Việc cả ba chân đều nối GND khiến chip sử dụng địa chỉ mặc định thấp nhất là 0x20. Cơ chế định địa chỉ này cho phép kết nối nhiều chip MCP23017 trên cùng một bus I2C mà không bị xung đột. Khi hệ thống cần mở rộng thêm nhiều ngăn tủ trong tương lai, mỗi cụm khóa có thể sử dụng một chip MCP23017 riêng, trong khi Raspberry Pi vẫn chỉ cần hai dây SDA và SCL để quản lý toàn bộ.

Tín hiệu mở khóa được xuất từ chân GPA0 của MCP23017 tới chân IN của module relay. Relay hoạt động theo nguyên lý nam châm điện: khi chân IN nhận tín hiệu kích từ MCP23017, dòng điện nhỏ chạy qua cuộn dây bên trong relay tạo ra từ trường, làm tiếp điểm chuyển từ vị trí NC sang NO. Khi đó, tiếp điểm COM–NO đóng lại, cho phép nguồn 5V cấp cho khóa điện từ XG-07C; chốt khóa được kéo vào và ngăn tủ được mở. Khi MCP23017 ngắt tín hiệu điều khiển, từ trường trong relay mất đi, tiếp điểm trở về trạng thái ban đầu, khóa mất điện và chốt được đẩy ra để khóa lại.

Khóa điện từ XG-07C có bốn dây, chia thành hai cặp với chức năng riêng biệt. Cặp dây thứ nhất là đường cấp nguồn 5V cho cuộn nam châm bên trong khóa, được nối vào cầu đầu J1; đây là đường điều khiển được relay đóng ngắt trực tiếp. Cặp

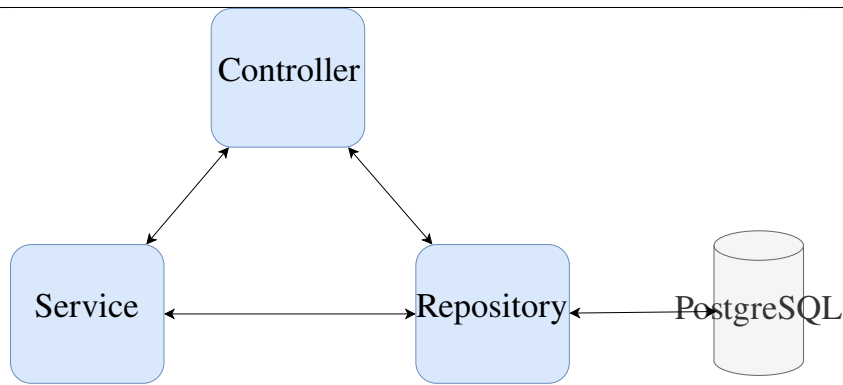
dây thứ hai là đường tín hiệu phản hồi từ microswitch tích hợp trong khóa, được nối vào cầu đầu J2 và đưa về chân GPB0 của MCP23017. Microswitch hoạt động theo cơ chế cơ học: khi chốt khóa ở trạng thái đóng, chốt tì vào công tắc làm mạch kín và GPB0 đọc mức cao (TRUE); khi relay kích làm chốt kéo vào, công tắc được nhả ra, mạch hở và GPB0 đọc mức thấp (FALSE). Nhờ tín hiệu phản hồi này, hệ thống có thể xác nhận trạng thái thực tế của từng ngăn tủ thay vì chỉ dựa vào lệnh điều khiển đã gửi đi.

4.1.3 Kiến trúc phần mềm

Về mặt phần mềm, hệ thống bao gồm bốn thành phần chính, mỗi thành phần đảm nhiệm một vai trò riêng và được xây dựng trên nền tảng công nghệ phù hợp:

- **Máy chủ backend:** được xây dựng bằng Node.js, TypeScript, Express và Prisma, đóng vai trò trung tâm xử lý logic nghiệp vụ và lưu trữ dữ liệu. Thành phần này bao gồm các module: xác thực, quản lý người dùng, quản lý quản trị viên, quản lý tủ và vị trí, quản lý lượt thuê, thanh toán, ghép nối thiết bị, và thông báo.
- **Trang quản trị Web:** được xây dựng bằng React và Vite, dành cho tác nhân Quản trị viên. Thành phần này bao gồm các module: quản lý tủ, quản lý ngăn tủ, quản lý vị trí, theo dõi trạng thái thời gian thực, duyệt ghép nối thiết bị, quản lý người dùng, quản lý lượt thuê, và xem nhật ký hoạt động.
- **Ứng dụng tại tủ:** được xây dựng bằng PySide6 chạy trên Raspberry Pi, dành cho tác nhân Khách hàng thao tác trực tiếp tại điểm thuê tủ. Thành phần này bao gồm các module: hiển thị màn hình chờ, quét mã QR, nhập mã PIN, xác nhận và mở khóa, điều khiển khóa và cảm biến, giao tiếp qua MQTT và REST API.
- **Ứng dụng di động:** được xây dựng bằng React Native (Expo), dành cho tác nhân Khách hàng. Thành phần này bao gồm các module: đăng ký/dăng nhập, xem vị trí các điểm cho thuê tủ, chọn tủ và gói thuê, tạo lượt thuê, hiển thị mã QR, xem lịch sử thuê, nhận thông báo, và quản lý hồ sơ cá nhân.

Trong số bốn thành phần trên, máy chủ backend là phần phức tạp nhất về mặt tổ chức mã nguồn. Để tách biệt rõ trách nhiệm giữa các tầng xử lý, backend được thiết kế theo mô hình kiến trúc Controller–Service–Repository (CSR), được mô tả trong Hình 4.5.



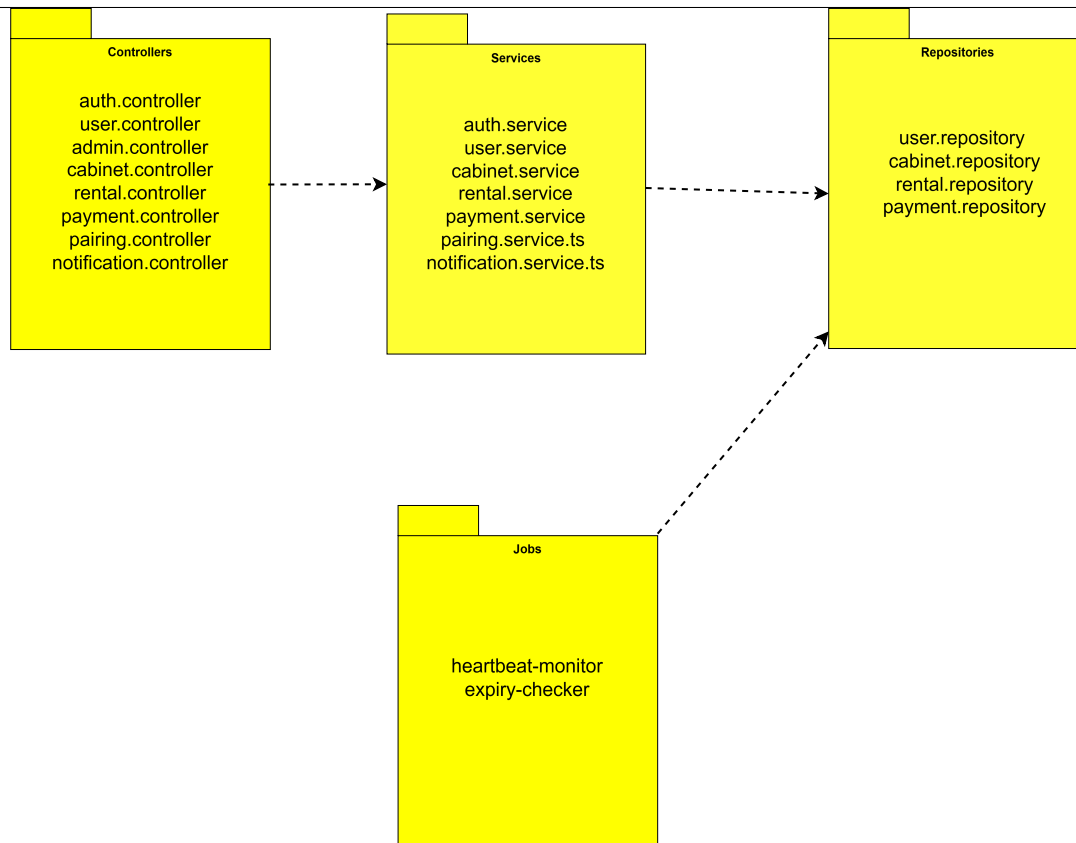
Hình 4.5: Kiến trúc CSR trong máy chủ backend

Trong mô hình này, các thành phần chính gồm:

- **Controller:** tiếp nhận yêu cầu từ client, kiểm tra dữ liệu đầu vào, gọi đến tầng Service tương ứng và trả về phản hồi cho client.
- **Service:** xử lý logic nghiệp vụ chính của hệ thống, chẳng hạn tính tiền thuê hoặc kiểm tra trạng thái tủ trước khi cho phép mở khóa, sau đó gọi đến tầng Repository để lấy hoặc cập nhật dữ liệu.
- **Repository:** chịu trách nhiệm truy xuất và lưu trữ dữ liệu trực tiếp với cơ sở dữ liệu PostgreSQL thông qua Prisma ORM, qua đó tách biệt logic truy vấn dữ liệu khỏi logic nghiệp vụ.

4.1.4 Kiến trúc chi tiết backend

Cấu trúc tổng quan các gói trong backend được mô tả như Hình 4.6.



Hình 4.6: Sơ đồ chi tiết các gói trong backend

Gói *controllers* chịu trách nhiệm tiếp nhận các yêu cầu REST API theo từng miền nghiệp vụ và gọi xuống tầng *services* tương ứng để xử lý. Các file chính trong gói này gồm:

- *auth.controller.ts, user.controller.ts, admin.controller.ts*
- *cabinet.controller.ts, rental.controller.ts, payment.controller.ts*
- *pairing.controller.ts, notification.controller.ts*

Gói *services* xử lý logic nghiệp vụ chính của hệ thống theo từng domain. Các file chính trong gói này gồm:

- *auth.service.ts, user.service.ts, cabinet.service.ts*
- *rental.service.ts, payment.service.ts, pairing.service.ts*
- *notification.service.ts*

Trong đó, *rental.service.ts* cần gọi thêm sang *cabinet.service.ts* để kiểm tra tủ còn trống và sang *payment.service.ts* để xác nhận thanh toán trước khi hoàn tất một lượt thuê. Một số service không có repository riêng sẽ gọi sang repository của domain liên quan, chẳng hạn *auth.service.ts* và *notification.service.ts* gọi tới *user.repository.ts*, còn *pairing.service.ts* gọi tới *cabinet.repository.ts*. Riêng *ad-*

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

min.controller.ts không có service riêng mà gọi trực tiếp tới nhiều service khác để tổng hợp dữ liệu hiển thị cho quản trị viên.

Gói *repositories* chịu trách nhiệm truy xuất dữ liệu thông qua Prisma ORM. Các file chính gồm *user.repository.ts*, *cabinet.repository.ts*, *rental.repository.ts* và *payment.repository.ts*. Mỗi repository chỉ tập trung vào thao tác dữ liệu của bảng hoặc nhóm bảng tương ứng, giúp tách biệt logic truy vấn khỏi logic nghiệp vụ.

Ngoài ba gói chính theo kiến trúc CSR, backend còn có gói *jobs* chứa các tác vụ chạy theo lịch. File *heartbeat-monitor.job.ts* định kỳ kiểm tra và cập nhật trạng thái kết nối của các kiosk, gọi trực tiếp tới *cabinet.repository.ts*. File *expiry-checker.job.ts* định kỳ kiểm tra và xử lý các lượt thuê đã hết hạn, gọi trực tiếp tới *rental.repository.ts*. Hai file này bỏ qua tầng *services* vì logic xử lý tương đối đơn giản, không cần áp dụng nhiều quy tắc nghiệp vụ phức tạp.

4.2 Thiết kế chi tiết

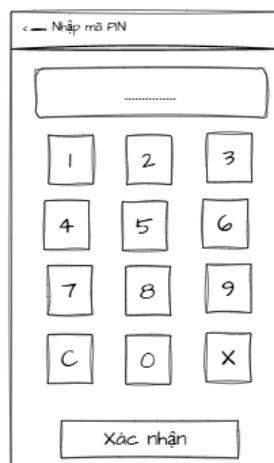
4.2.1 Thiết kế giao diện

Trước khi xây dựng giao diện hoàn chỉnh, đồ án thực hiện thiết kế wireframe cho các màn hình chính nhằm xác định bố cục, luồng thao tác và vị trí các thành phần quan trọng. Các wireframe tập trung vào ba nhóm giao diện: ứng dụng tại tủ, ứng dụng di động và trang quản trị.

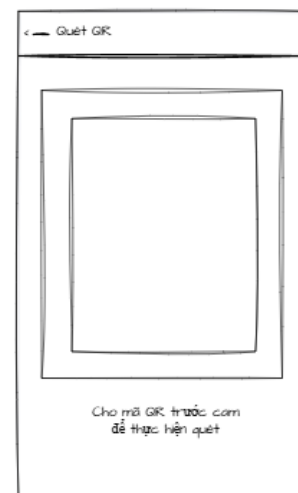
Đối với ứng dụng tại tủ, sau đây là các wireframe minh họa cho những màn hình thao tác chính:



Hình 4.7: Wireframe màn hình chính trên kiosk



Hình 4.8: Wireframe nhập mã PIN trên kiosk



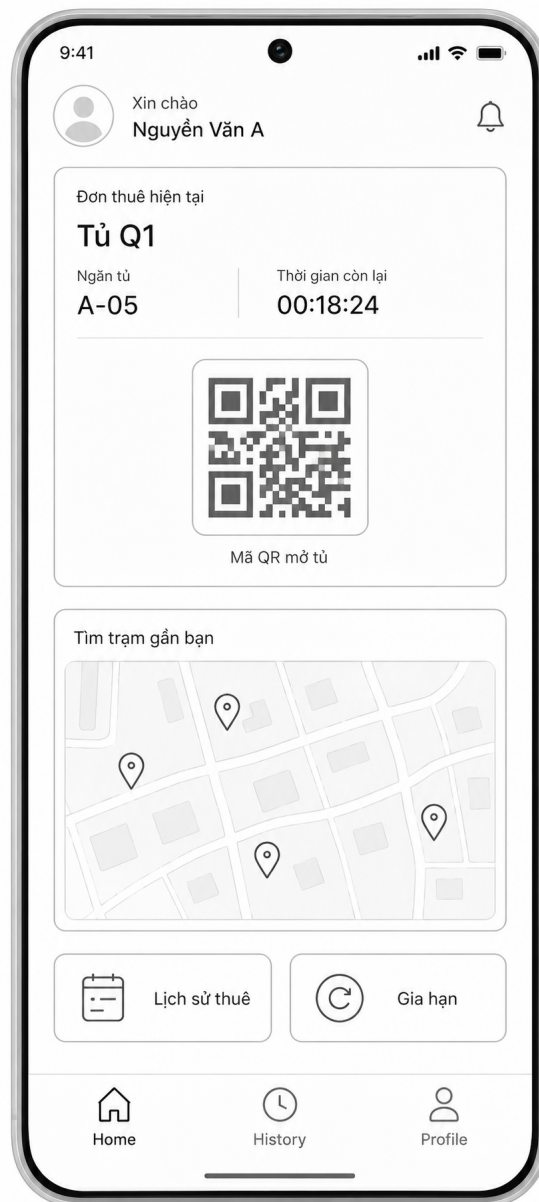
Hình 4.9: Wireframe quét mã QR trên kiosk

Màn hình chính của kiosk hiển thị các lựa chọn thao tác lớn để người dùng

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

nhANH chóng chọn luồng thuê tủ, nhập mã PIN hoặc quét mã QR. Màn hình nhập PIN tập trung vào bàn phím số và vùng hiển thị mã đã nhập, giúp người dùng thao tác rõ ràng trên màn hình cảm ứng. Màn hình quét QR ưu tiên vùng camera lớn cùng hướng dẫn đặt mã vào khung quét, hỗ trợ mở tủ nhanh khi người dùng đã có mã trên ứng dụng di động.

Đối với ứng dụng di động, sau đây là wireframe minh hoạ cho màn hình trang chủ:



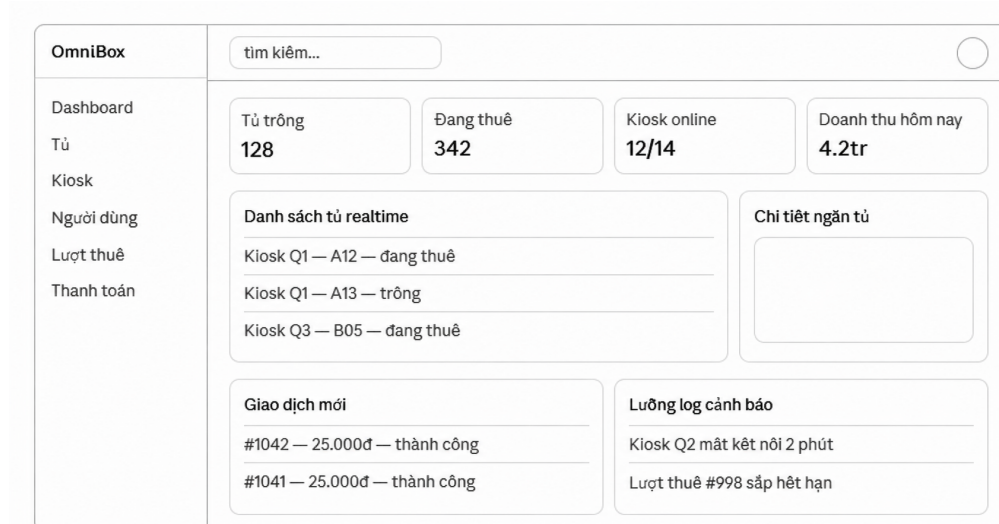
Hình 4.10: Wireframe màn hình trang chủ trên ứng dụng di động

Trang chủ hiển thị thông tin đơn thuê hiện tại của khách hàng, bao gồm tủ và ngăn tủ đang thuê, thời gian còn lại, kèm mã QR để mở tủ trực tiếp từ ứng dụng mà không cần quay lại kiosk thao tác thủ công. Phía dưới là bản đồ hiển thị các trạm gần vị trí người dùng, cùng hai lối tắt truy cập nhanh tới lịch sử thuê và chức năng

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

gia hạn. Thanh điều hướng dưới cùng cho phép chuyển giữa trang chủ, lịch sử thuê và hồ sơ cá nhân.

Đối với trang quản trị Web, sau đây là wireframe minh hoạ cho màn hình Dashboard:



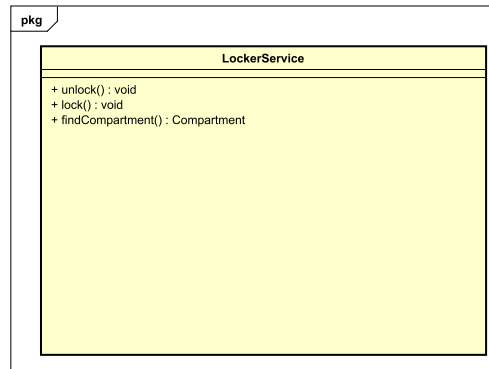
Hình 4.11: Wireframe màn hình Dashboard quản trị viên

Giao diện được chia làm ba khu vực chính: thanh điều hướng bên trái liệt kê các chức năng quản lý (tủ, kiosk, người dùng, lướt thuê, thanh toán); khu vực phía trên hiển thị các chỉ số tổng quan (số tủ trống, số lượt đang thuê, số kiosk đang hoạt động, doanh thu trong ngày); phía dưới hiển thị danh sách tủ theo thời gian thực kèm chi tiết từng ngăn tủ, cùng bảng giao dịch mới nhất và luồng cảnh báo hệ thống.

4.2.2 Thiết kế lớp

Phần này trình bày thiết kế một số lớp dịch vụ quan trọng trong hệ thống, bao gồm các lớp xử lý nghiệp vụ thuê tủ, thanh toán và điều khiển mở khóa.

Lớp LockerService: Lớp chịu trách nhiệm điều khiển trực tiếp phần cứng khoá tủ thông qua giao thức MQTT, đóng vai trò cầu nối giữa logic phần mềm và thiết bị vật lý trên Raspberry Pi.

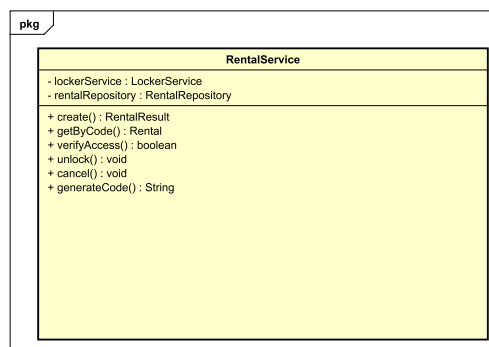


Hình 4.12: Thiết kế lớp LockerService

Mô tả các thuộc tính và phương thức của lớp:

- `mqttService`: tham chiếu tới lớp `MqttService` để gửi lệnh điều khiển khoá tới kiosk qua giao thức MQTT
- `unlock()`: gửi lệnh mở khoá tới ngăn tủ tương ứng, kèm thông tin phiên thuê để ghi log
- `lock()`: gửi lệnh đóng khoá tới ngăn tủ sau khi người dùng hoàn thành thao tác
- `findCompartment()`: phương thức nội bộ tìm kiếm thông tin ngăn tủ theo định danh trước khi thực hiện lệnh điều khiển

Lớp RentalService: Lớp xử lý toàn bộ nghiệp vụ thuê tủ trong hệ thống, từ khởi tạo phiên thuê, xác thực quyền truy cập, điều khiển mở khoá cho đến kết thúc hoặc huỷ phiên thuê.



Hình 4.13: Thiết kế lớp RentalService

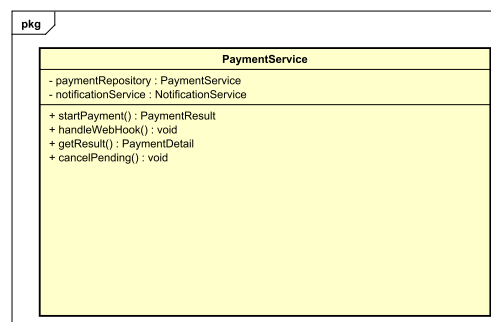
Mô tả các thuộc tính và phương thức của lớp:

- `lockerService`: tham chiếu tới lớp `LockerService` để thực hiện lệnh mở khoá vật lý khi người dùng xác thực thành công

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

- `rentalRepository`: tham chiếu tới lớp `RentalRepository` để truy xuất và lưu trữ dữ liệu phiên thuê
- `create()`: khởi tạo phiên thuê mới, tìm ngăn tủ còn trống phù hợp kích thước yêu cầu và tạo mã xác thực
- `getByCode()`: tra cứu thông tin phiên thuê theo mã PIN 6 chữ số
- `verifyAccess()`: xác thực quyền mở tủ thông qua mã QR, kiểm tra trạng thái và số lượt mở còn lại
- `unlock()`: tăng số lượt mở đã dùng và gọi `LockerService` để mở khoá vật lý, tự động kết thúc phiên nếu hết lượt
- `complete()`: kết thúc phiên thuê, cập nhật trạng thái ngăn tủ về trạng thái trống
- `cancel()`: huỷ phiên thuê, giải phóng ngăn tủ cho người dùng khác
- `generateCode()`: phương thức nội bộ sinh mã PIN 6 chữ số ngẫu nhiên, đảm bảo không trùng với các mã đang hoạt động

Lớp `PaymentService`: Lớp xử lý toàn bộ nghiệp vụ thanh toán, bao gồm khởi tạo giao dịch, tiếp nhận kết quả từ cổng thanh toán và huỷ giao dịch quá hạn.



Hình 4.14: Thiết kế lớp `PaymentService`

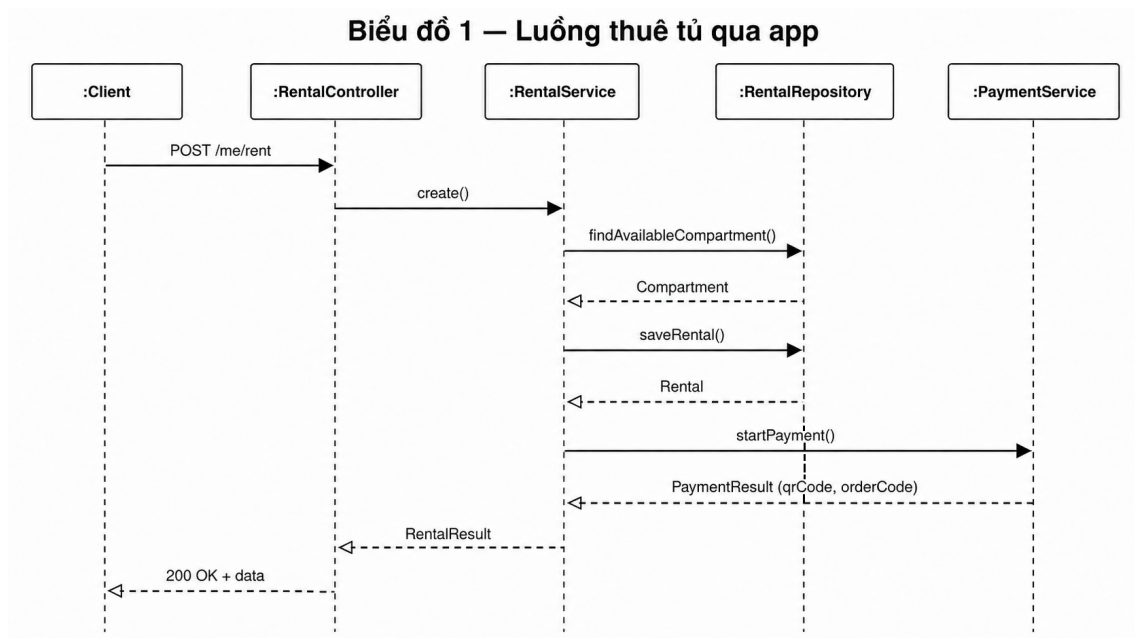
Mô tả các thuộc tính và phương thức của lớp:

- `paymentRepository`: tham chiếu tới lớp `PaymentRepository` để lưu trữ và truy xuất thông tin giao dịch
- `rentalRepository`: tham chiếu tới lớp `RentalRepository` để cập nhật trạng thái phiên thuê sau khi thanh toán thành công
- `notificationService`: tham chiếu tới lớp `NotificationService` để gửi thông báo cho người dùng sau khi giao dịch hoàn tất
- `startPayment()`: khởi tạo giao dịch thanh toán cho một phiên thuê, tạo

mã QR và đường dẫn thanh toán

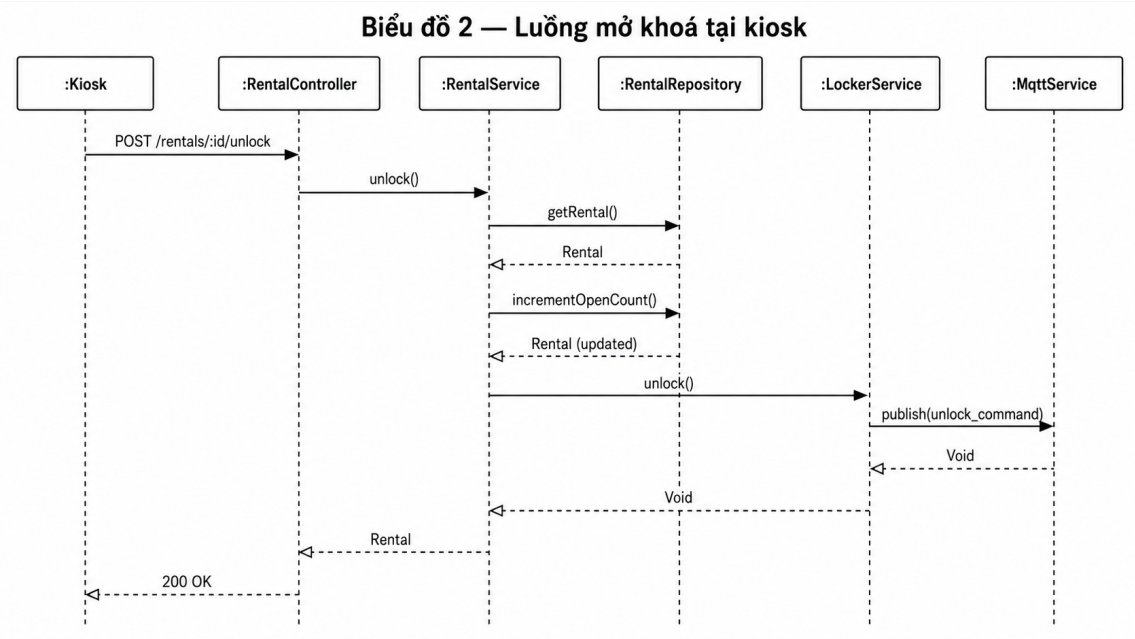
- `handleWebhook()` : tiếp nhận và xử lý kết quả thanh toán từ cổng PayOS, cập nhật trạng thái giao dịch và kích hoạt phiên thuê
- `getResult()` : truy vấn kết quả giao dịch theo mã đơn hàng, chỉ trả về khi thanh toán đã hoàn tất
- `cancelPending()` : huỷ giao dịch đang chờ thanh toán quá thời hạn, giải phóng ngăn tủ đã giữ chỗ

Sau đây là biểu đồ trình tự minh hoạ thiết kế lớp với hai luồng nghiệp vụ chính:



Hình 4.15: Biểu đồ trình tự luồng thuê tủ qua ứng dụng di động

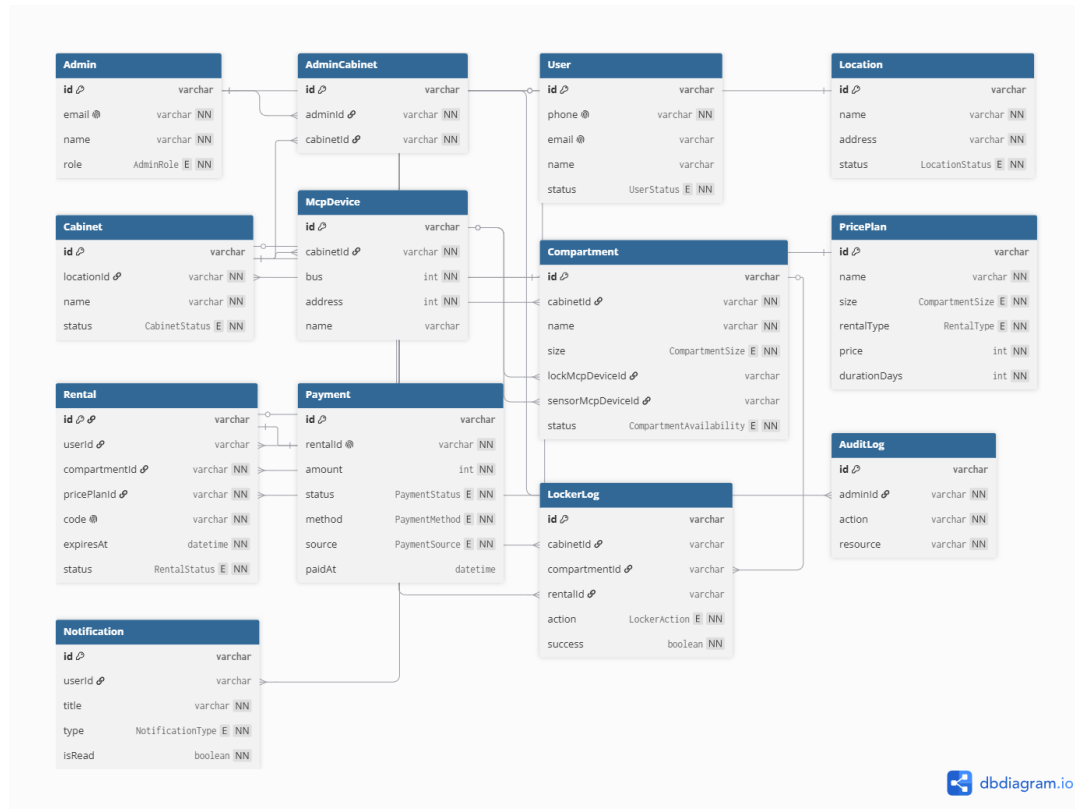
Khi người dùng thực hiện thuê tủ trên ứng dụng di động, yêu cầu được gửi tới RentalController rồi chuyển tiếp xuống RentalService. RentalService tìm ngăn tủ còn trống phù hợp kích thước yêu cầu thông qua RentalRepository, tạo bản ghi phiên thuê và sinh mã xác thực, sau đó gọi PaymentService để khởi tạo giao dịch thanh toán và trả về mã QR cho người dùng quét.



Hình 4.16: Biểu đồ trình tự luồng mở khoá tại kiosk

Khi người dùng xác thực tại kiosk bằng PIN hoặc QR, yêu cầu mở khoá được gửi tới RentalController rồi chuyển xuống RentalService. RentalService kiểm tra trạng thái phiên thuê và số lượt mở còn lại qua RentalRepository, sau đó gọi LockerService để phát lệnh mở khoá. LockerService chuyển lệnh thành bản tin MQTT gửi tới kiosk qua MqttService, Raspberry Pi nhận lệnh và kích hoạt khoá điện từ mở ngăn tủ tương ứng.

4.2.3 Thiết kế cơ sở dữ liệu



Hình 4.17: Biểu đồ thực thể liên kết của cơ sở dữ liệu

Bảng admins: Lưu trữ thông tin về những người quản trị hệ thống, bao gồm quản trị viên toàn quyền và quản trị viên phụ trách tủ. Bảng này được truy xuất bởi các API xử lý đăng nhập và phân quyền của ứng dụng web quản trị.

Bảng 4.1: Cấu tạo bảng admins

Tên trường	Kiểu dữ liệu	Mô tả
id	varchar	Khoá chính, id của quản trị viên
email	varchar	Địa chỉ email đăng nhập, là duy nhất
password_hash	varchar	Mật khẩu đã được băm
name	varchar	Tên của quản trị viên
role	enum	Vai trò: SUPER_ADMIN (toàn quyền), CABINET_ADMIN (phụ trách tủ)
created_at	datetime	Thời gian tạo tài khoản trên hệ thống
updated_at	datetime	Thời gian cập nhật thông tin lần cuối

Bảng users: Lưu trữ thông tin về người dùng cuối, tức những khách hàng trực tiếp thuê tủ. Bảng được truy xuất bởi các API quản lý tài khoản người dùng từ ứng

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

dụng di động và máy kiosk.

Bảng 4.2: Cấu tạo bảng users

Tên trường	Kiểu dữ liệu	Mô tả
id	varchar	Khoá chính, id của người dùng
phone	varchar	Số điện thoại, là thông tin định danh chính
email	varchar	Địa chỉ email của người dùng
password_hash	varchar	Mật khẩu đã được mã hoá
name	varchar	Tên của người dùng
fcm_token	varchar	Mã thiết bị dùng để gửi thông báo đẩy
status	enum	Trạng thái: ACTIVE (hoạt động), SUSPENDED (tạm khoá)
created_at	datetime	Thời gian tạo tài khoản trên hệ thống
updated_at	datetime	Thời gian cập nhật thông tin lần cuối

Bảng locations: Lưu trữ thông tin về các địa điểm đặt tủ, chẳng hạn một toà nhà hay khu ký túc xá. Bảng được truy xuất bởi các API hiển thị danh sách điểm thuê trên ứng dụng di động và quản lý địa điểm trên web quản trị.

Bảng 4.3: Cấu tạo bảng locations

Tên trường	Kiểu dữ liệu	Mô tả
id	varchar	Khoá chính, id của địa điểm
name	varchar	Tên của địa điểm
address	varchar	Địa chỉ cụ thể của địa điểm
latitude	float	Vĩ độ, phục vụ hiển thị trên bản đồ
longitude	float	Kinh độ, phục vụ hiển thị trên bản đồ
status	enum	Trạng thái: ACTIVE (hoạt động), IN-ACTIVE (ngừng hoạt động)
created_at	datetime	Thời gian tạo địa điểm trên hệ thống
updated_at	datetime	Thời gian cập nhật thông tin lần cuối

Bảng cabinets: Lưu trữ thông tin về từng tủ vật lý được lắp đặt tại một địa điểm. Mỗi bản ghi tương ứng với một tủ ngoài thực tế. Bảng được truy xuất bởi các API quản lý tủ và đồng bộ trạng thái thiết bị.

Bảng 4.4: Cấu tạo bảng cabinets

Tên trường	Kiểu dữ liệu	Mô tả
id	varchar	Khoá chính, id của tủ
location_id	varchar	Khoá ngoài, id của địa điểm chứa tủ
name	varchar	Tên của tủ
status	enum	Trạng thái hoạt động của tủ (đang hoạt động, mất kết nối, đang cấu hình...)
last_heartbeat_at	datetime	Thời điểm tủ gửi tín hiệu còn sống gần nhất
created_at	datetime	Thời gian thêm tủ vào hệ thống
updated_at	datetime	Thời gian cập nhật thông tin lần cuối

Bảng compartments: Lưu trữ thông tin về từng ngăn tủ riêng lẻ bên trong một tủ. Đây là đơn vị nhỏ nhất mà người dùng thực sự thuê. Bảng được truy xuất bởi các API tra cứu ngăn trống và xử lý đơn thuê.

Bảng 4.5: Cấu tạo bảng compartments

Tên trường	Kiểu dữ liệu	Mô tả
id	varchar	Khoá chính, id của ngăn tủ
cabinet_id	varchar	Khoá ngoài, id của tủ chứa ngăn này
name	varchar	Tên hoặc số hiệu của ngăn
size	enum	Kích thước ngăn: SMALL (nhỏ), LARGE (lớn)
status	enum	Trạng thái: trống, đang thuê, đang bảo trì, đã giữ chỗ
created_at	datetime	Thời gian tạo ngăn trên hệ thống
updated_at	datetime	Thời gian cập nhật thông tin lần cuối

Bảng price_plans: Lưu trữ thông tin về các gói cước mà hệ thống cung cấp. Bảng được truy xuất bởi các API hiển thị gói thuê trên ứng dụng di động, máy kiosk và quản lý gói cước trên web quản trị.

Bảng 4.6: Cấu tạo bảng price_plans

Tên trường	Kiểu dữ liệu	Mô tả
id	varchar	Khoá chính, id của gói cước
name	varchar	Tên của gói cước

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Tên trường	Kiểu dữ liệu	Mô tả
size	enum	Kích thước ngăn áp dụng: SMALL, LARGE
rental_type	enum	Hình thức thuê: ONCE (theo lượt), DAILY (theo ngày), MONTHLY (theo tháng)
price	int	Mức giá của gói cước
duration_days	int	Thời hạn của gói tính theo ngày
is_active	boolean	Gói cước còn được áp dụng hay không
created_at	datetime	Thời gian tạo gói cước trên hệ thống
updated_at	datetime	Thời gian cập nhật thông tin lần cuối

Bảng rentals: Lưu trữ thông tin về mỗi lượt thuê tủ của người dùng. Đây là bảng trung tâm, kết nối người dùng, ngăn tủ và gói cước. Bảng được truy xuất bởi hầu hết các API liên quan đến quá trình thuê và mở tủ.

Bảng 4.7: Cấu tạo bảng rentals

Tên trường	Kiểu dữ liệu	Mô tả
id	varchar	Khoá chính, id của lượt thuê
user_id	varchar	Khoá ngoài, id của người dùng thuê tủ
compartment_id	varchar	Khoá ngoài, id của ngăn được thuê
price_plan_id	varchar	Khoá ngoài, id của gói cước đã chọn
code	varchar	Mã PIN dùng để mở tủ
qr_token	varchar	Mã định danh cho mã QR mở tủ
expires_at	datetime	Thời điểm lượt thuê hết hạn
status	enum	Trạng thái: đang thuê, đã hoàn tất, đã huỷ, đã hết hạn
created_at	datetime	Thời gian tạo lượt thuê trên hệ thống
updated_at	datetime	Thời gian cập nhật thông tin lần cuối

Bảng payments: Lưu trữ thông tin thanh toán gắn với mỗi lượt thuê. Mỗi bản ghi tương ứng với một bản ghi ở bảng rentals. Bảng được truy xuất bởi các API xử lý thanh toán và đối soát giao dịch với cổng PayOS.

Bảng 4.8: Cấu tạo bảng payments

Tên trường	Kiểu dữ liệu	Mô tả
id	varchar	Khoá chính, id của giao dịch thanh toán
rental_id	varchar	Khoá ngoài, id của lượt thuê tương ứng
amount	int	Số tiền cần thanh toán
status	enum	Trạng thái: PENDING, PAID, RE-FUNDED, FAILED
method	enum	Phương thức thanh toán (PayOS,...)
source	enum	Nguồn phát sinh thanh toán: KIOSK hoặc APP
paid_at	datetime	Thời điểm thanh toán thành công
created_at	datetime	Thời gian tạo giao dịch trên hệ thống
updated_at	datetime	Thời gian cập nhật thông tin lần cuối

Bảng locker_logs: Lưu trữ nhật ký các sự kiện đóng mở tủ, chẳng hạn mỗi lần một ngăn được mở thành công hay thất bại. Bảng phục vụ việc tra cứu, đối soát và xử lý sự cố. Dữ liệu được ghi vào bởi các API và luồng xử lý sự kiện từ tủ.

Bảng 4.9: Cấu tạo bảng locker_logs

Tên trường	Kiểu dữ liệu	Mô tả
id	varchar	Khoá chính, id của bản ghi nhật ký
cabinet_id	varchar	Khoá ngoài, id của tủ liên quan
compartment_id	varchar	Khoá ngoài, id của ngăn liên quan
rental_id	varchar	Khoá ngoài, id của lượt thuê liên quan
action	enum	Loại sự kiện: mở, đóng, hết hạn, từ chối...
success	boolean	Sự kiện thực hiện thành công hay thất bại
created_at	datetime	Thời gian ghi nhận sự kiện

Bảng audit_logs: Lưu vết các thao tác quan trọng do quản trị viên thực hiện trên hệ thống, như tạo mới hay chỉnh sửa dữ liệu. Bảng giúp truy xuất trách nhiệm và bảo đảm tính minh bạch khi vận hành. Dữ liệu được ghi vào bởi các API quản trị.

Bảng 4.10: Cấu tạo bảng audit_logs

Tên trường	Kiểu dữ liệu	Mô tả
id	varchar	Khoá chính, id của bản ghi
admin_id	varchar	Khoá ngoài, id của quản trị viên thực hiện thao tác
action	enum	Hành động được thực hiện (đăng nhập, tạo, sửa, xoá...)
resource	varchar	Đối tượng dữ liệu bị tác động
created_at	datetime	Thời gian ghi nhận thao tác

Bảng notifications: Lưu trữ các thông báo gửi đến người dùng, ví dụ thông báo lượt thuê sắp hết hạn hay thanh toán thành công. Bảng được truy xuất bởi các API hiển thị danh sách thông báo trên ứng dụng di động.

Bảng 4.11: Cấu tạo bảng notifications

Tên trường	Kiểu dữ liệu	Mô tả
id	varchar	Khoá chính, id của thông báo
user_id	varchar	Khoá ngoài, id của người dùng nhận thông báo
title	varchar	Tiêu đề của thông báo
body	varchar	Nội dung của thông báo
type	enum	Loại thông báo (hết hạn, thanh toán, hệ thống...)
is_read	boolean	Thông báo đã được đọc hay chưa
sent_at	datetime	Thời điểm gửi thông báo

4.3 ‘Xây dựng ứng dụng’

4.3.1 Thư viện và công cụ sử dụng

Phần backend của hệ thống được xây dựng bằng Node.js và TypeScript, kết hợp với các thư viện hỗ trợ xây dựng REST API, truy xuất cơ sở dữ liệu, giao tiếp thời gian thực, thanh toán và bảo mật. Các thư viện, framework và công cụ chính được sử dụng được trình bày trong Bảng 4.12.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Nhóm công cụ	Thư viện/Công cụ	Vai trò trong backend
Nền tảng cốt lõi	Node.js, TypeScript, ts-node-dev	Node.js là môi trường chạy chính của backend; TypeScript được sử dụng làm ngôn ngữ lập trình nhằm tăng tính an toàn kiểu dữ liệu; ts-node-dev hỗ trợ tự động khởi động lại server khi mã nguồn thay đổi trong quá trình phát triển.
Framework API	Express	Framework dùng để xây dựng các RESTful API, tiếp nhận và xử lý yêu cầu từ ứng dụng di động, trang quản trị web và kiosk.
Cơ sở dữ liệu và ORM	Prisma ORM, @prisma/client, PostgreSQL	Prisma ORM và @prisma/client được sử dụng để ánh xạ dữ liệu và thực hiện các truy vấn tới PostgreSQL một cách an toàn, trực quan; đồng thời hỗ trợ công cụ Prisma Studio để quan sát và quản lý dữ liệu trong quá trình phát triển.
Truyền thông IoT	mqtt	Thư viện triển khai giao tiếp MQTT giữa backend và ứng dụng kiosk chạy trên Raspberry Pi, phục vụ truyền nhận các lệnh phần cứng như mở khoá tủ, cập nhật trạng thái và gửi tín hiệu heartbeat.
Thời gian thực	socket.io	Cung cấp kết nối hai chiều theo thời gian thực giữa backend với trang quản trị web, giúp cập nhật tức thời trạng thái tủ, lượt thuê và thanh toán.
Thanh toán và mã QR	@payos/node, qrcode	@payos/node được dùng để tích hợp cổng thanh toán PayOS, tạo mã thanh toán QR động và nhận thông báo giao dịch qua web-hook; qrcode được dùng để sinh mã QR mở tủ cho khách hàng quét tại kiosk.
Tác vụ nền	node-cron	Lập lịch các tác vụ tự động như kiểm tra lượt thuê quá hạn, xử lý lượt thuê cần huỷ và phát hiện kiosk mất kết nối.
Bảo mật và xác thực	bcrypt, jsonwebtoken, zod	bcrypt dùng để băm một chiều mật khẩu; jsonwebtoken dùng để tạo và xác thực JWT nhằm bảo vệ các API; zod dùng để kiểm tra cấu trúc và kiểu dữ liệu đầu vào của request trước khi xử lý nghiệp vụ.

Bảng 4.12: Danh sách thư viện và công cụ sử dụng ở phía backend

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Đối với ứng dụng di động Android, hệ thống sử dụng React Native kết hợp với Expo để phát triển giao diện đa nền tảng, đồng thời tích hợp các thư viện hỗ trợ điều hướng, bản đồ, mã QR, lưu trữ an toàn và quản lý trạng thái. Các công cụ chính được sử dụng trong ứng dụng di động được trình bày trong Bảng 4.13.

Nhóm công cụ	Thư viện/Công cụ	Vai trò trong ứng dụng Android
Nền tảng cốt lõi và điều hướng	React Native, Expo v54, Expo Router, React Navigation	React Native và Expo v54 được sử dụng làm nền tảng xây dựng ứng dụng di động đa nền tảng. Expo Router hỗ trợ định tuyến dựa trên cấu trúc thư mục, giúp tổ chức các màn hình rõ ràng. React Navigation được dùng để quản lý thanh điều hướng dưới cùng và luồng chuyển màn hình dạng stack.
Giao diện và trải nghiệm người dùng	NativeWind v5, Tailwind CSS v4, React Native Reanimated v4, Gesture Handler, @gorhom/bottom-sheet, expo-haptics	NativeWind và Tailwind CSS giúp viết style giao diện bằng các class Tailwind trực tiếp trong React Native. Reanimated và Gesture Handler hỗ trợ cử chỉ vuốt chạm và animation mượt mà. Gorhom Bottom Sheet được dùng để hiển thị bảng trượt thông tin trạm tử hoặc bộ lọc. Expo Haptics tạo phản hồi rung nhẹ khi người dùng thao tác với các nút chức năng quan trọng.
Bản đồ và tiện ích phần cứng	react-native-maps, expo-location, react-native-qr-code-svg	react-native-maps tích hợp bản đồ để hiển thị và tìm kiếm các trạm tử xung quanh người dùng. expo-location xin quyền và lấy tọa độ GPS theo thời gian thực để tính khoảng cách đến trạm tử gần nhất. react-native-qr-code-svg tạo mã QR động chứa token thuê để người dùng quét tại camera của kiosk.
Quản lý trạng thái và lưu trữ	Zustand, expo-secure-store	Zustand được dùng để quản lý trạng thái toàn cục như trạng thái đăng nhập, thông tin tử và đơn thuê hiện tại. expo-secure-store lưu trữ an toàn các dữ liệu nhạy cảm, đặc biệt là JWT token xác thực, bằng cơ chế mã hoá trên thiết bị Android.

Bảng 4.13: Danh sách thư viện và công cụ sử dụng trong ứng dụng Android

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

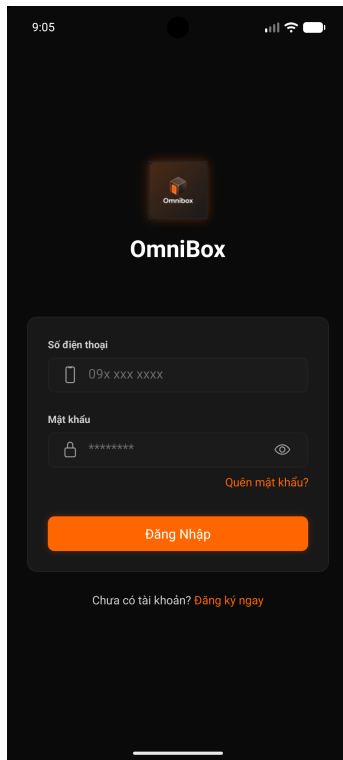
Đối với ứng dụng tại tủ, hệ thống sử dụng Python và PySide6 để xây dựng giao diện cảm ứng trên Raspberry Pi, đồng thời tích hợp các thư viện giao tiếp backend, camera, mã QR và phần cứng. Các công cụ chính được trình bày trong Bảng 4.14.

Nhóm công cụ	Thư viện/Công cụ	Vai trò trong ứng dụng tại tủ
Giao diện người dùng và đồ họa	PySide6, Qt Designer, QUiLoader, pyside6-rcc	PySide6 dùng để xây dựng giao diện kiosk cảm ứng. Qt Designer thiết kế layout dạng .ui và QUiLoader nạp động trong Python. pyside6-rcc đóng gói ảnh, logo và SVG thành resources_rc.py để tối ưu đọc tài nguyên trên Raspberry Pi.
Kết nối API và thiết bị IoT	paho-mqtt, Requests	paho-mqtt giúp kiosk giao tiếp thời gian thực với MQTT Broker để nhận lệnh mở khoá và gửi trạng thái tủ. Requests dùng để gọi REST API như xác thực mã PIN, lấy gói thuê và tạo đơn hàng.
Quản lý camera và xử lý hình ảnh	opencv-python-headless, Picamera2	opencv-python-headless điều khiển camera, lấy khung hình và xử lý video quét mã QR mà không cần giao diện đồ họa. Picamera2 tối ưu việc chụp ảnh và đọc QR trên camera Raspberry Pi.
Giao tiếp phần cứng và tiện ích	smbus2, qrcode, Pillow, PyYAML	smbus2 giao tiếp I2C với MCP23017 để điều khiển khoá điện và đọc cảm biến từ. qrcode kết hợp Pillow tạo ảnh QR thanh toán hiển thị trên kiosk. PyYAML đọc cấu hình từ config.yaml như API, MQTT và chân điều khiển phần cứng.

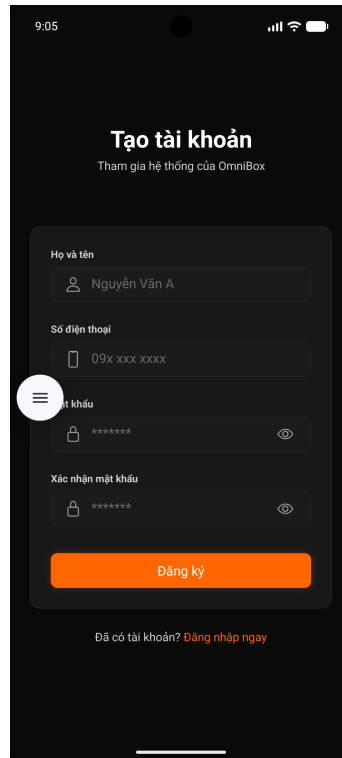
Bảng 4.14: Danh sách thư viện và công cụ sử dụng trong ứng dụng tại tủ

4.3.2 Minh họa các chức năng chính

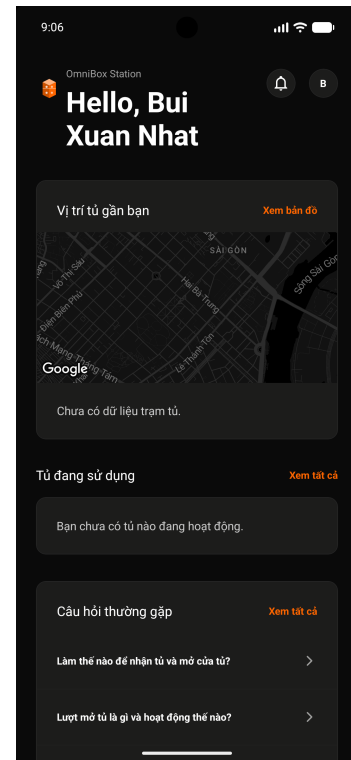
a, Ứng dụng di động



Hình 4.18: Đăng nhập

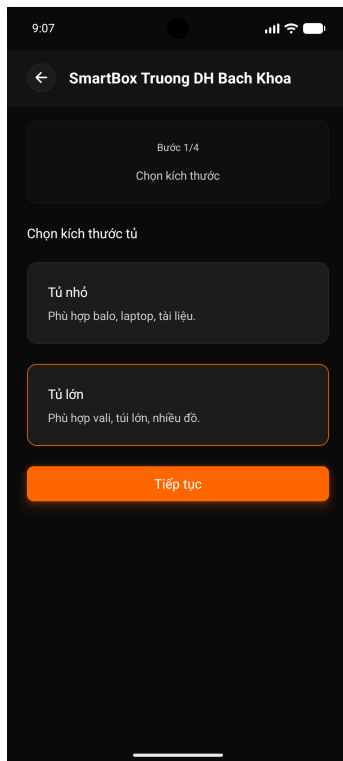


Hình 4.19: Đăng ký tài khoản

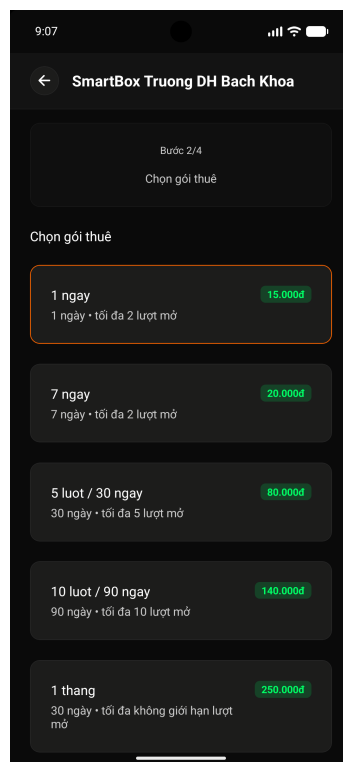


Hình 4.20: Trang chủ

Màn hình đăng nhập yêu cầu người dùng nhập số điện thoại và mật khẩu, đồng thời cung cấp tùy chọn quên mật khẩu và chuyển sang màn hình đăng ký nếu chưa có tài khoản. Màn hình đăng ký thu thập các thông tin cần thiết gồm họ tên, số điện thoại, mật khẩu và xác nhận mật khẩu. Trong hệ thống, số điện thoại được dùng làm định danh chính, phù hợp với cách người dùng thuê tủ tại kiosk mà không cần ghi nhớ tên đăng nhập phức tạp. Sau khi đăng nhập thành công, trang chủ hiển thị ba khu vực chính: bản đồ Google Maps đánh dấu vị trí các trạm tủ xung quanh người dùng, danh sách gợi ý các trạm gần nhất kèm khoảng cách và số ngăn còn trống, và danh sách các tủ người dùng đang thuê để theo dõi thời gian còn lại mà không cần mở từng màn hình chi tiết.



Hình 4.21: Chọn kích thước túi



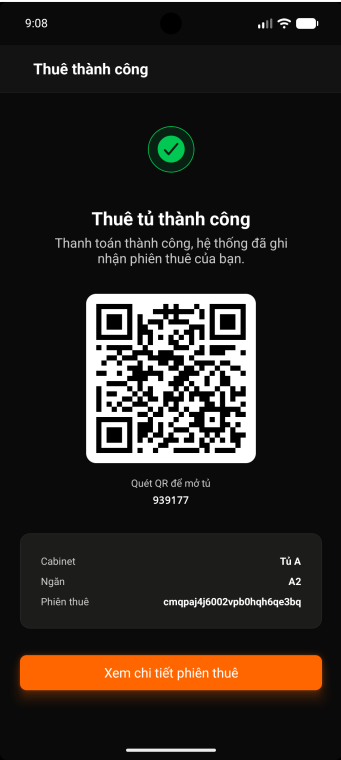
Hình 4.22: Chọn gói thuê



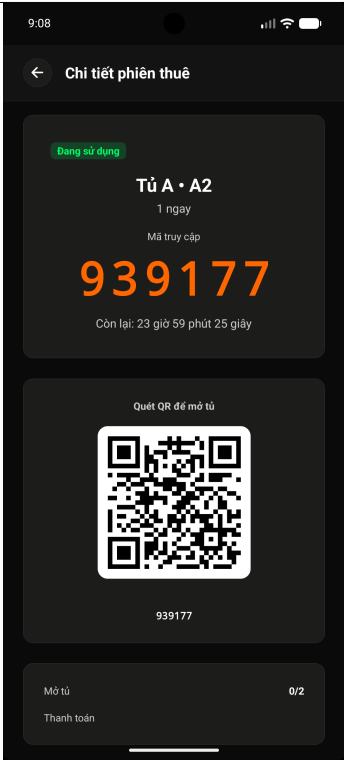
Hình 4.23: Thanh toán

Quy trình thuê tủ được thiết kế theo dạng từng bước (4 bước), hiển thị tiến trình rõ ràng ở đầu màn hình để người dùng biết mình đang ở đâu trong luồng thao tác. Bước đầu tiên cho phép chọn kích thước tủ phù hợp với nhu cầu. Bước tiếp theo cung cấp các gói thuê đa dạng từ 1 ngày đến 1 tháng, mỗi gói ghi rõ thời hạn, số lượt mở tủ tối đa và mức giá, giúp người dùng dễ dàng so sánh và lựa chọn theo nhu cầu thực tế. Bước thanh toán hiển thị mã VietQR tương thích với hầu hết các ứng dụng ngân hàng tại Việt Nam, kèm tổng số tiền cần thanh toán và đồng hồ đếm ngược thời gian giữ chỗ - nếu không thanh toán trong thời gian quy định, hệ thống tự động huỷ và giải phóng ngăn tủ cho người dùng khác.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



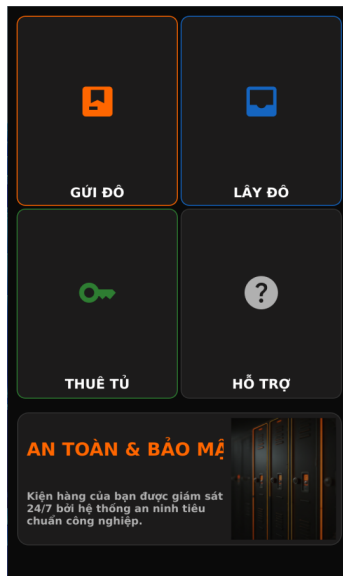
Hình 4.24: Thuê tủ thành công



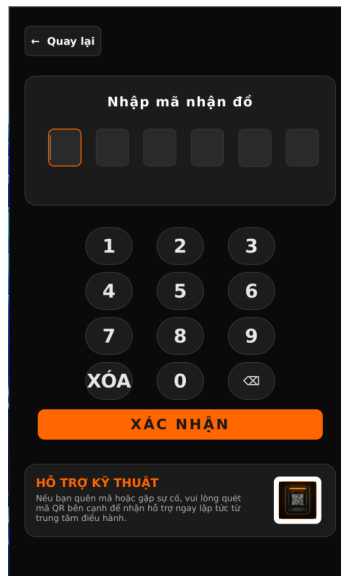
Hình 4.25: Chi tiết phiên thuê

Sau khi thanh toán được xác nhận, hệ thống lập tức tạo phiên thuê và hiển thị màn hình thành công với mã QR mở tủ kèm thông tin tủ và ngăn được phân công tự động. Người dùng có thể chọn xem chi tiết phiên thuê ngay hoặc quay lại sau - mã QR luôn có thể truy cập lại từ màn hình chi tiết phiên thuê bất kỳ lúc nào trong thời hạn thuê. Màn hình chi tiết phiên thuê hiển thị đầy đủ mã PIN 6 chữ số để nhập tại kiosk, mã QR để quét trực tiếp, thời gian còn lại được cập nhật liên tục, và số lượt mở tủ đã sử dụng so với tổng số lượt cho phép theo gói đã chọn - đây là màn hình trung tâm mà người dùng sẽ quay lại mỗi khi cần mở tủ trong suốt thời gian thuê.

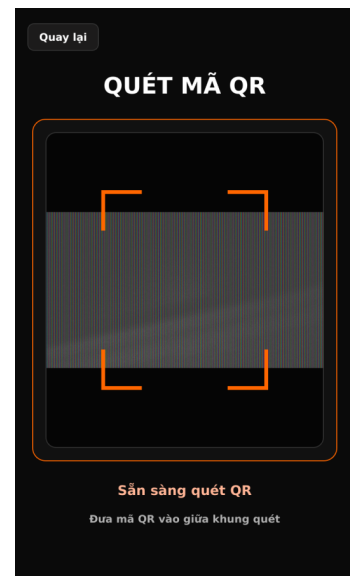
b, Ứng dụng tại tủ – kiosk



Hình 4.26: Màn hình chính kiosk

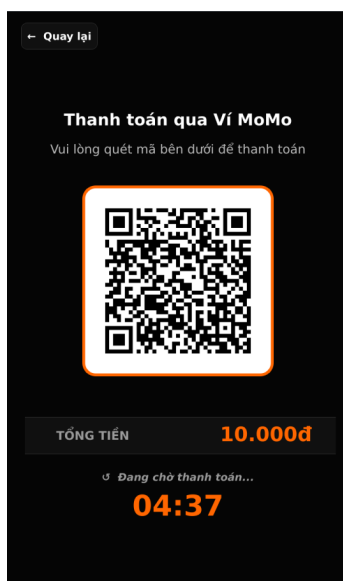


Hình 4.27: Nhập mã PIN tại kiosk

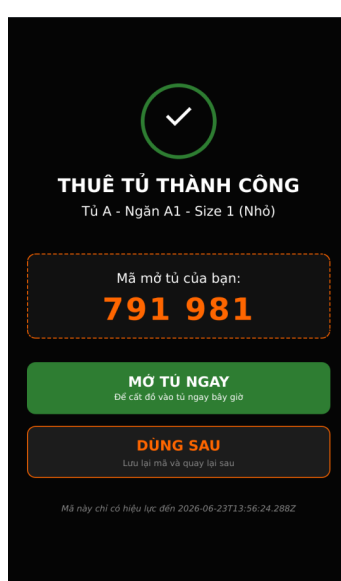


Hình 4.28: Quét mã QR tại kiosk

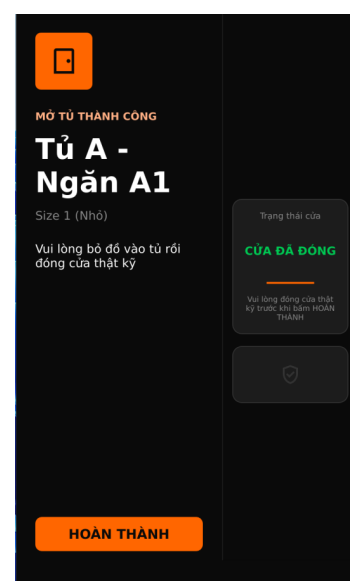
Ứng dụng tại tủ được thiết kế theo dạng kiosk cảm ứng, ưu tiên các nút thao tác lớn và nội dung hướng dẫn rõ ràng để người dùng có thể tự thực hiện mà không cần nhân viên hỗ trợ. Ba màn hình trên minh họa các chức năng chính trong luồng mở tủ: màn hình trang chủ để chọn chức năng, màn hình nhập mã PIN và màn hình quét mã QR. Hai phương thức PIN và QR cùng được hỗ trợ nhằm tạo sự linh hoạt: mã PIN phù hợp khi người dùng không tiện mở điện thoại, còn mã QR giúp thao tác nhanh hơn khi đã có sẵn mã trên ứng dụng di động.



Hình 4.29: Thanh toán tại kiosk



Hình 4.30: Thuê tủ thành công tại kiosk



Hình 4.31: Mở khóa thành công tại kiosk

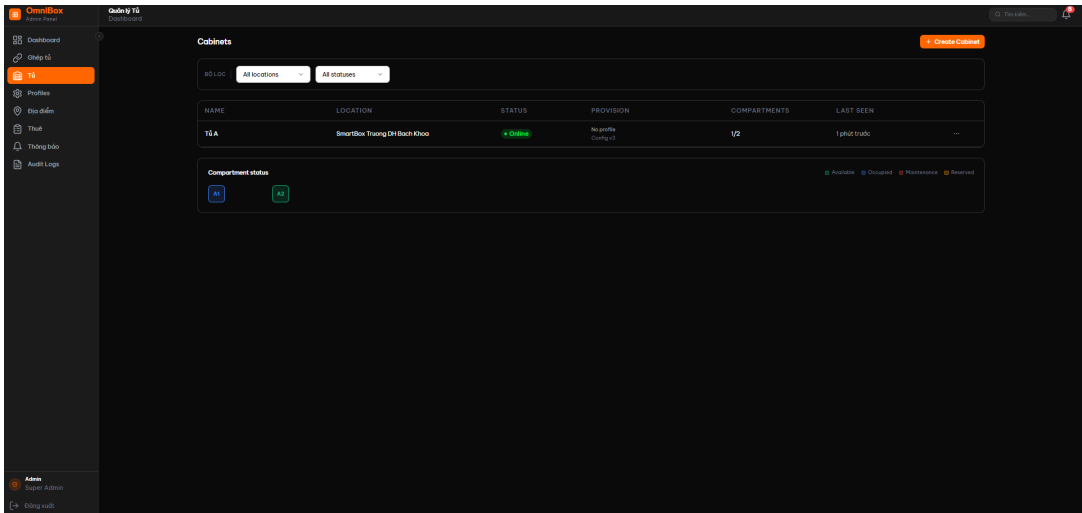
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Đối với luồng thuê trực tiếp tại tủ, kiosk hiển thị màn hình thanh toán bằng mã QR để người dùng quét và hoàn tất giao dịch. Sau khi thanh toán được xác nhận, hệ thống tạo phiên thuê, phân công ngăn tủ phù hợp và hiển thị kết quả thuê thành công. Khi người dùng xác thực mở tủ thành công, kiosk hiển thị thông báo kết quả rõ ràng để người dùng biết thao tác đã được xử lý và có thể nhận hoặc gửi đồ vào đúng ngăn tủ.

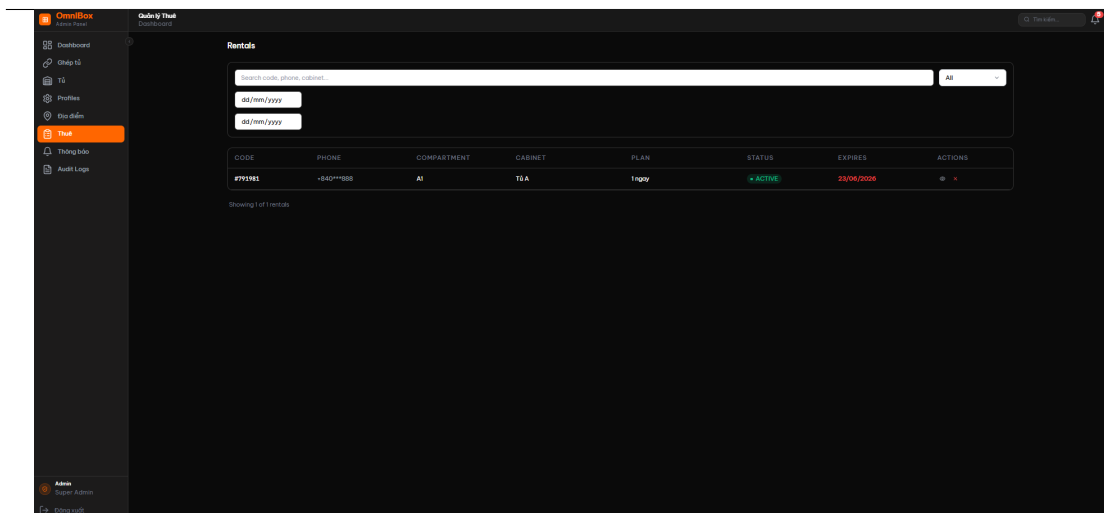
c, Trang quản trị admin



Hình 4.32: Dashboard quản trị



Hình 4.33: Quản lý tủ và ngăn tủ



Hình 4.34: Quản lý lượt thuê

Trang quản trị admin cung cấp giao diện theo dõi và vận hành hệ thống cho quản trị viên. Màn hình dashboard tổng hợp các chỉ số quan trọng như tình trạng tủ, số lượt thuê và trạng thái hoạt động của các kiosk. Màn hình quản lý tủ cho phép xem danh sách tủ, ngăn tủ và trạng thái từng ngăn để phục vụ bảo trì hoặc cấu hình. Màn hình quản lý lượt thuê hỗ trợ tra cứu các phiên thuê, theo dõi trạng thái thanh toán và xử lý các tình huống phát sinh trong quá trình vận hành.

4.3.3 Mô hình tủ gửi đồ



Hình 4.35: Mô hình phần cứng của thiết bị tủ ở trạng thái ban đầu



Hình 4.36: Mô hình phần cứng của thiết bị tủ ở trạng thái mở ngăn

Hình 4.35 minh họa mô hình phần cứng của thiết bị tủ OmniBox ở trạng thái ban đầu. Ở trạng thái này, các ngăn tủ đang đóng và màn hình cảm ứng hiển thị giao diện chính của kiosk, cho phép người dùng lựa chọn các thao tác như gửi đồ, nhận đồ, thuê tủ hoặc yêu cầu hỗ trợ.

Hình 4.36 minh họa trạng thái của mô hình sau khi hệ thống thực hiện lệnh mở tủ thành công. Một ngăn tủ được mở ra để người dùng đặt hoặc lấy đồ, đồng thời màn hình kiosk hiển thị thông báo mở tủ thành công và hướng dẫn người dùng hoàn tất thao tác với ngăn tủ.

4.4 Kiểm thử

Sau khi hoàn thiện các chức năng chính, quá trình kiểm thử được thực hiện nhằm xác nhận hệ thống hoạt động đúng theo yêu cầu đã đặt ra. Do hệ thống gồm nhiều thành phần phối hợp với nhau, bao gồm ứng dụng di động, ứng dụng tại tủ, máy chủ backend, cơ sở dữ liệu và phần cứng điều khiển khóa, việc kiểm thử được tổ chức theo các kịch bản sử dụng thực tế. Mỗi kịch bản mô phỏng một tình huống mà người thuê hoặc quản trị viên có thể gặp trong quá trình sử dụng hệ thống, sau đó kết quả thực tế được đối chiếu với kết quả mong muốn.

Cách kiểm thử theo kịch bản giúp đánh giá không chỉ hành vi của từng thành phần riêng lẻ, mà còn kiểm tra sự phối hợp giữa các thành phần trong toàn bộ hệ thống. Bên cạnh các luồng thành công như thuê tủ, thanh toán và mở tủ, quá trình kiểm thử cũng bao gồm các trường hợp hệ thống cần từ chối thao tác, chẳng hạn nhập sai mã PIN, quét mã QR hết hiệu lực hoặc thuê tủ khi không còn ngăn trống

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

phù hợp.

STT	Chức năng	Kịch bản kiểm thử	Kết quả mong muốn
1	Thuê tủ trên ứng dụng	1. Chọn một điểm đặt tủ 2. Chọn loại tủ nhỏ 3. Chọn gói thuê theo lượt 4. Xác nhận thuê	1. Hệ thống tự động chọn ô tủ trống đầu tiên phù hợp 2. Sinh mã PIN và mã QR cho lượt thuê 3. Hiện thị thông báo thuê thành công 4. Lượt thuê xuất hiện trong danh sách của người dùng với thông tin chính xác
2	Thuê tủ trên ứng dụng	1. Chọn một điểm đặt tủ đã hết ô trống loại nhỏ 2. Chọn loại tủ nhỏ 3. Xác nhận thuê	1. Hệ thống hiện thị thông báo “Hiện không còn tủ trống phù hợp” 2. Không tạo lượt thuê mới
3	Mở tủ bằng mã PIN tại kiosk	1. Chọn chức năng mở tủ 2. Nhập đúng mã PIN gồm 6 chữ số của lượt thuê đang hoạt động	1. Hệ thống xác thực mã thành công 2. Gửi lệnh mở khóa tới đúng ô tủ 3. Cửa tủ mở ra 4. Hoạt động được ghi lại trong nhật ký
4	Mở tủ bằng mã PIN tại kiosk	1. Chọn chức năng mở tủ 2. Nhập một mã PIN sai	1. Hệ thống báo mã không hợp lệ 2. Không gửi lệnh mở khóa 3. Cửa tủ vẫn đóng
5	Mở tủ bằng mã QR tại kiosk	1. Chọn chức năng quét mã QR 2. Đưa mã QR của lượt thuê đang hoạt động vào camera	1. Hệ thống đọc và xác thực mã QR thành công 2. Gửi lệnh mở khóa tới đúng ô tủ 3. Cửa tủ mở ra
<i>Xem tiếp ở trang sau</i>			

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

STT	Chức năng	Kịch bản kiểm thử	Kết quả mong muốn
6	Mở tủ bằng mã QR tại kiosk	1. Quét mã QR của một lượt thuê đã kết thúc	1. Hệ thống báo mã không còn hiệu lực 2. Không gửi lệnh mở khóa
7	Tự động kết thúc lượt thuê hết hạn	1. Tạo một lượt thuê với thời hạn ngắn 2. Chờ qua thời điểm hết hạn	1. Hệ thống tự động chuyển lượt thuê sang trạng thái hết hạn 2. Ô tủ được giải phóng để cho người khác thuê 3. Người dùng nhận được thông báo
8	Theo dõi trạng thái tủ trên trang quản trị	1. Mở trang quản trị 2. Tại kiosk, mở rồi đóng một ô tủ	1. Trạng thái cửa và khóa của ô tủ trên trang quản trị thay đổi gần như tức thì 2. Người quản trị không cần tải lại trang
9	Phát hiện tủ mất kết nối	1. Ngắt kết nối của một tủ 2. Chờ quá khoảng thời gian quy định không nhận được tín hiệu	1. Hệ thống tự động đánh dấu tủ ở trạng thái ngoại tuyến 2. Trạng thái này hiển thị trên trang quản trị
10	Ghép nối kiosk mới với hệ thống	1. Khởi động kiosk chưa được ghép nối 2. Kiosk hiển thị mã QR và mã 6 chữ số 3. Quản trị viên duyệt yêu cầu trên trang quản trị	1. Yêu cầu ghép nối hiện ra trong danh sách chờ duyệt 2. Sau khi duyệt, tủ được tạo và chuyển sang trạng thái đang cấu hình 3. Kiosk nhận được cấu hình và sẵn sàng hoạt động

Bảng 4.15: Các kịch bản kiểm thử chức năng của hệ thống

Kết quả kiểm thử

Sau khi thực hiện lần lượt các kịch bản trên, hầu hết các chức năng cốt lõi đều cho kết quả đúng như mong muốn. Bảng dưới đây tổng hợp kết quả kiểm thử trên các thành phần khác nhau của hệ thống.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Kịch bản	Ứng dụng	Kiosk	Trang quản trị
Kịch bản 1	pass	—	—
Kịch bản 2	pass	—	—
Kịch bản 3	—	pass	—
Kịch bản 4	—	pass	—
Kịch bản 5	—	pass	—
Kịch bản 6	—	pass	—
Kịch bản 7	pass	—	pass
Kịch bản 8	—	pass	pass
Kịch bản 9	—	—	pass
Kịch bản 10	—	pass	pass

Bảng 4.16: Kết quả kiểm thử trên các thành phần của hệ thống

Bên cạnh kiểm thử theo các kịch bản sử dụng, đồ án cũng xây dựng một số bài kiểm thử tự động cho chức năng sinh và xác thực mã QR ở phía máy chủ. Đây là khâu quan trọng vì mã QR được sử dụng để xác định một lượt thuê hợp lệ trước khi cho phép mở tủ. Các bài kiểm thử tập trung kiểm tra ba trường hợp chính: mã QR được sinh đúng quy trình sẽ được chấp nhận; mã QR bị chỉnh sửa nội dung sẽ bị từ chối; và mã QR đã hết hiệu lực cũng không được hệ thống chấp nhận. Qua đó, cơ chế xác thực bằng mã QR được kiểm tra về tính đúng đắn và góp phần nâng cao độ tin cậy của hệ thống.

Nhìn chung, kết quả kiểm thử cho thấy hệ thống đáp ứng được các yêu cầu chức năng đã đặt ra ở Chương 2, đặc biệt là ở những luồng quan trọng nhất: thuê tủ, xác thực mở tủ và đồng bộ trạng thái theo thời gian thực giữa tủ thật và trang quản trị.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Trong những chương trước, đồ án đã lần lượt phân tích nhu cầu của người dùng, lựa chọn công nghệ và mô tả cách hệ thống được thiết kế. Chương này tập trung vào bốn vấn đề đã được đầu tư nhiều công sức nhất trong quá trình xây dựng hệ thống, cũng là những đóng góp kỹ thuật cốt lõi của đồ án.

Bốn đóng góp đó gồm: (1) cách để người dùng mở tủ vừa nhanh vừa an toàn bằng mã PIN và mã QR; (2) cách giúp màn hình quản trị luôn phản ánh đúng tình trạng thật của từng ô tủ ngay tại thời điểm đang xảy ra; (3) cách lắp đặt một kiosk mới ngoài thực địa mà gần như không phải thao tác kỹ thuật phức tạp; và (4) cách hệ thống tự thu hồi tủ khi hết hạn thuê mà không cần ai can thiệp bằng tay.

5.1 Mở tủ an toàn bằng mã PIN và mã QR

5.1.1 Giới thiệu bài toán

Một người thuê ngăn có thể quay lại lấy đồ trong rất nhiều hoàn cảnh khác nhau: lúc thì có sẵn điện thoại và mạng ổn định, lúc lại chỉ có vài giây vội vã trước khi vào lớp, thậm chí có khi điện thoại hết pin. Vì vậy, hệ thống cần ít nhất hai cách để mở tủ, sao cho người dùng luôn có một lựa chọn phù hợp với tình huống của mình.

Tuy nhiên, sự tiện lợi không được phép đánh đổi bằng an toàn. Yêu cầu quan trọng nhất ở đây là: **không ai có thể mở được tủ của người khác**. Điều này dẫn tới hai mối lo cụ thể. Thứ nhất, nếu chẳng may cơ sở dữ liệu bị lộ, thì những mã mở tủ đã được cấp cũng không được phép rơi vào tay người khác ở dạng có thể dùng lại. Thứ hai, không một ai được phép tự tạo ra một mã hợp lệ để mở một tủ bất kỳ.

5.1.2 Giải pháp

Hai hình thức xác thực được xây dựng độc lập, mỗi hình thức phục vụ một tình huống và đều được kiểm tra ở phía máy chủ chứ không để thiết bị tự quyết định.

Mở tủ bằng mã PIN sáu chữ số. Đây là cách đơn giản và phổ thông nhất: người dùng chỉ cần nhớ sáu chữ số rồi gõ lên màn hình kiosk, không cần điện thoại hay mạng. Điểm được chú trọng nằm ở khâu bảo mật. Khi tạo một lượt thuê, hệ thống sinh ra mã PIN ngẫu nhiên nhưng *không lưu lại nguyên văn dãy số đó*. Thay vào đó, mã được chuyển thành một chuỗi “khó đảo ngược” rồi mới lưu vào cơ sở dữ liệu. Có thể hình dung quá trình này giống như lấy dấu vân tay của mã PIN: từ mã PIN ta tạo ra được dấu vân tay, nhưng từ dấu vân tay thì không thể lần ngược ra mã PIN ban đầu. Khi người dùng gõ PIN tại kiosk, hệ thống chỉ việc tạo lại dấu vân tay từ mã vừa nhập rồi đối chiếu; nếu trùng khớp thì mới cho mở. Nhờ vậy, ngay cả khi dữ liệu bị lộ, người ngoài cũng không thể đọc được mã PIN gốc của bất kỳ ai.

Mở tủ bằng mã QR. Với những người có sẵn điện thoại, quét một mã QR sẽ nhanh hơn việc gõ tay. Vấn đề kỹ thuật cần cân nhắc là: nên đặt gì *bên trong* mã QR. Nếu nhồi quá nhiều thông tin, hình QR sẽ trở nên dày đặc, khó quét, đặc biệt trong điều kiện ánh sáng kém. Giải pháp được chọn là đưa vào QR một chuỗi gọn nhẹ kèm theo một “con dấu xác thực”. Con dấu này được tính ra từ thông tin của lượt thuê cộng với một khoá bí mật mà chỉ máy chủ nắm giữ. Khi kiosk quét mã, máy chủ sẽ tính lại con dấu và so sánh: nếu khớp, nghĩa là mã đúng là do hệ thống cấp ra và chưa bị ai chỉnh sửa. Cách làm này vừa giữ cho mã QR đủ nhỏ để quét nhanh, vừa đảm bảo không thể làm giả, bởi người ngoài không biết khoá bí mật nên không thể tạo ra con dấu hợp lệ.

Điểm chung của cả hai hình thức là quyết định “có mở hay không” luôn nằm ở máy chủ. Kiosk chỉ đóng vai trò thu nhận mã rồi gửi về để kiểm tra, chứ bản thân thiết bị không lưu sẵn lời giải. Nhờ đó, dù thiết bị đặt ở nơi công cộng, logic xác thực vẫn được bảo vệ an toàn ở phía trong.

5.1.3 Kết quả

Người dùng có hai cách mở tủ độc lập, mỗi cách phù hợp với một hoàn cảnh: mã PIN dùng được kể cả khi không có điện thoại hay mạng chập chờn, còn mã QR cho trải nghiệm nhanh gọn khi điều kiện thuận lợi. Về mặt an toàn, mã PIN không thể bị đọc ngược ra từ cơ sở dữ liệu, còn mã QR không thể bị làm giả từ bên ngoài. Hai cơ chế này cùng nhau bảo đảm yêu cầu cốt lõi đã đặt ra: mỗi người chỉ mở được đúng tủ của mình.

5.2 Đồng bộ trạng thái tủ theo thời gian thực

5.2.1 Giới thiệu bài toán

Đề tài xây dựng hệ thống tủ gửi đồ có một đặc thù khác với nhiều phần mềm thông thường: trạng thái mà hệ thống quản lý không nằm gọn trong cơ sở dữ liệu, mà gắn liền với những cánh cửa và ổ khoá có thật ngoài đời. Một ô tủ tại bất kỳ thời điểm nào cũng có thể đang khoá hoặc đang mở, cửa đang đóng hay đang hé, và cả tủ thì có thể đang hoạt động bình thường hoặc đã mất kết nối.

Người quản trị viên ngồi trước màn hình cần nhìn thấy đúng những gì đang diễn ra ngay lúc đó, để còn kịp thời xử lý. Nếu màn hình hiển thị chậm hơn thực tế, hoặc tệ hơn là hiển thị sai, thì việc giám sát và điều khiển từ xa sẽ mất đi ý nghĩa. Bài toán đặt ra là làm sao để mọi thay đổi ở phần cứng được phản ánh lên giao diện gần như tức thời, mà không phải liên tục hỏi đi hỏi lại “có gì mới không” một cách lãng phí.

5.2.2 Giải pháp

Đường truyền thông tin được chia thành hai chặng, mỗi chặng dùng một cách giao tiếp phù hợp với hai đầu rất khác nhau: một bên là thiết bị phần cứng, một bên là trình duyệt web.

Chặng từ tủ về máy chủ. Mỗi tủ vật lý sẽ chủ động “báo tin” mỗi khi có sự kiện, ví dụ một ổ khoá vừa mở hay một cánh cửa vừa đóng, đồng thời đều đặn gửi một tín hiệu “còn sống” để cho biết nó vẫn đang hoạt động. Cách nhắn tin này hoạt động theo kiểu đăng tin lên các “kênh” và bên quan tâm thì đăng ký nhận tin từ kênh đó, rất nhẹ và tiết kiệm, phù hợp với những thiết bị nhỏ và đường truyền hạn chế. Máy chủ đăng ký lắng nghe các kênh này nên sẽ nhận được tin ngay khi tủ phát đi.

Chặng từ máy chủ lên màn hình quản trị. Khi máy chủ nhận được một sự kiện từ tủ, thông tin đó cần được đẩy lên giao diện càng nhanh càng tốt. Thay vì để trình duyệt cứ vài giây lại hỏi máy chủ một lần, một kết nối “luôn mở” được duy trì giữa hai bên. Nhờ kết nối này, máy chủ có thể chủ động gửi cập nhật lên màn hình bất cứ lúc nào, và giao diện tự thay đổi mà người dùng không phải tải lại trang.

Ngoài ra, một cơ chế theo dõi tín hiệu “còn sống” cũng được bổ sung. Nếu một tủ im lặng quá một khoảng thời gian quy định, hệ thống sẽ tự động đánh dấu tủ đó là mất kết nối, để quản trị viên biết mà kiểm tra, thay vì cứ tưởng tủ vẫn đang bình thường.

5.2.3 Kết quả

Màn hình quản trị phản ánh tình trạng thật của từng ô tủ gần như ngay tại thời điểm sự việc xảy ra, mà không tiêu tốn tài nguyên cho việc hỏi đáp liên tục. Quản trị viên có thể giám sát toàn bộ hệ thống từ xa, mở khoá khi cần và phát hiện sớm những tủ gặp sự cố mất kết nối. Đây là nền tảng giúp việc vận hành hệ thống trở nên chủ động thay vì bị động chờ người dùng báo lỗi.

5.3 Tự động ghép nối kiosk với hệ thống

5.3.1 Giới thiệu bài toán

Một hệ thống tủ thông minh sẽ chỉ thực sự hữu ích nếu việc nhân rộng nó ra nhiều địa điểm là đơn giản. Trong thực tế, mỗi khi lắp đặt một kiosk mới, ta phải gắn nó với đúng một bản ghi tủ trong hệ thống, đồng thời cấp cho nó những thông tin cần thiết để kết nối và hoạt động.

Nếu phải nhập tay các thông tin này vào từng thiết bị, công việc sẽ vừa mất thời gian vừa dễ sai sót, nhất là khi số lượng kiosk tăng lên. Quan trọng hơn, việc cài sẵn các thông tin nhạy cảm vào thiết bị ngay từ lúc xuất xưởng cũng tiềm ẩn rủi

ro, bởi thiết bị được đặt ở nơi công cộng và có thể trải qua nhiều khâu vận chuyển. Bài toán đặt ra là tìm một quy trình lắp đặt vừa đơn giản cho người quản trị, vừa an toàn cho thông tin của hệ thống.

5.3.2 Giải pháp

Quy trình ghép nối được thiết kế theo hướng để chính kiosk chủ động khởi xướng, với một bước duyệt nhẹ nhàng từ phía quản trị viên.

Khi một kiosk mới được bật lên, nó sẽ tự gửi yêu cầu ghép nối về máy chủ. Máy chủ tạo ra một phiên ghép nối đang ở trạng thái chờ, còn kiosk thì hiển thị một mã QR kèm một mã số ngắn để người lắp đặt đối chiếu. Trên màn hình quản trị, yêu cầu này hiện ra như một lời đề nghị đang chờ xử lý. Quản trị viên chỉ cần xác nhận và chọn tử tương ứng để gắn vào. Ngay sau khi được duyệt, kiosk (vẫn vẫn đang chờ và hỏi máy chủ một cách định kỳ) sẽ nhận được cấu hình cùng những thông tin kết nối cần thiết để bắt đầu hoạt động.

Có hai điểm thiết kế đáng chú ý ở đây. Thứ nhất, quy trình luôn được khởi xướng từ phía kiosk, nghĩa là máy chủ không cần “vớ tay” vào mạng nội bộ của thiết bị, một việc thường khó thực hiện và tiềm ẩn rủi ro. Thứ hai, các thông tin nhạy cảm chỉ được trao cho kiosk *sau khi* một người có thẩm quyền đã duyệt, chứ không nằm sẵn trong thiết bị từ trước. Nhờ đó, ngay cả khi thiết bị thất lạc trên đường vận chuyển thì cũng không kèm theo thông tin bí mật nào.

5.3.3 Kết quả

Việc triển khai một tử mới được rút gọn xuống còn vài thao tác đơn giản: cắm điện, bật máy, rồi duyệt một lần trên màn hình quản trị. Người lắp đặt không phải nhập tay bất kỳ thông tin kỹ thuật phức tạp nào, nhờ đó giảm đáng kể sai sót và rút ngắn thời gian triển khai. Đây cũng chính là tiền đề giúp hệ thống được nhân rộng ra nhiều địa điểm một cách thuận lợi.

5.4 Tự động thu hồi tử khi hết hạn thuê

5.4.1 Giới thiệu bài toán

Mỗi lượt thuê tử đều gắn với một khoảng thời gian nhất định. Khi thời hạn kết thúc mà người dùng chưa lấy đồ, hệ thống cần xử lý tình huống này một cách hợp lý: vừa nhắc nhở người thuê, vừa giải phóng ô tử để sẵn sàng phục vụ người tiếp theo. Nếu mọi việc đều phải chờ quản trị viên kiểm tra và thao tác bằng tay, hệ thống sẽ khó vận hành trơn tru, nhất là khi có nhiều tử ở nhiều nơi cùng lúc.

Bên cạnh đó còn một rủi ro tinh vi hơn về tính nhất quán của dữ liệu. Khi nhiều việc cùng diễn ra song song, hoàn toàn có khả năng hai tiến trình khác nhau cùng lúc cố gắng giải phóng hoặc gán cùng một ô tử, dẫn tới việc hai người được xếp

vào chung một ô. Tình huống tranh chấp tài nguyên kiểu này cần được phòng ngừa ngay từ trong thiết kế.

5.4.2 Giải pháp

Bài toán được giải quyết bằng một tác vụ chạy nền theo lịch định kỳ. Cứ sau mỗi khoảng thời gian ngắn, tác vụ này lại tự động rà soát toàn bộ các lượt thuê, tìm ra những lượt đã quá hạn, rồi lần lượt: đánh dấu lượt thuê là đã hết hạn, trả ô tử về trạng thái còn trống, ghi lại sự việc vào nhật ký, và gửi thông báo cho người dùng. Toàn bộ chuỗi việc này diễn ra mà không cần ai khởi động bằng tay.

Để phòng ngừa tình huống hai người bị xếp chung một ô, giải pháp kết hợp với một quyết định thiết kế đã được thống nhất từ trước: việc gán ô tử luôn được thực hiện một cách tự động và tuần tự. Nghĩa là tại mỗi thời điểm, hệ thống chỉ cấp phát ô tử theo một thứ tự rõ ràng, nhờ đó loại bỏ được khả năng hai yêu cầu cùng nhận về một ô. Cách làm này giúp dữ liệu luôn nhất quán mà không cần đến những cơ chế xử lý xung đột phức tạp.

5.4.3 Kết quả

Những ô tử hết hạn được tự động thu hồi và quay vòng để phục vụ người dùng kế tiếp, còn người thuê thì nhận được thông báo kịp thời. Quan trọng không kém, hệ thống luôn giữ được trạng thái nhất quán: mỗi ô tử tại một thời điểm chỉ thuộc về đúng một lượt thuê. Nhờ vậy, hệ thống có thể vận hành liên tục và ổn định mà gần như không cần đến sự can thiệp thủ công của người quản trị.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết những gì đồ án đã đạt được, thẳng thắn nhìn nhận những phần còn dang dở, và đề xuất một vài hướng đi để hoàn thiện hệ thống trong tương lai.

6.1 Kết luận

Sau quá trình thực hiện đồ án tốt nghiệp, một hệ thống gửi đồ thông minh tự phục vụ đã được xây dựng, hướng tới các không gian công cộng như khuôn viên trường học, ký túc xá và khu chung cư. Hệ thống gồm bốn thành phần phối hợp với nhau: máy chủ trung tâm, ứng dụng kiosk đặt tại tủ, trang quản trị dành cho người vận hành và ứng dụng di động dành cho người dùng. Về cơ bản, hệ thống đã đáp ứng được các mục tiêu đặt ra ban đầu: người dùng có thể thuê tủ qua điện thoại hoặc trực tiếp tại kiosk, mở tủ bằng mã PIN hoặc mã QR; trong khi đó, người quản trị có thể theo dõi và điều khiển toàn bộ hệ thống từ xa theo thời gian thực.

So với một số sản phẩm tương tự đã được khảo sát ở Chương 2, chẳng hạn như SantaPocket và SmartBox của ViettelPost, hệ thống trong đề tài có một số điểm khác biệt đáng lưu ý. Các sản phẩm thương mại nói trên chủ yếu phục vụ nhu cầu giao nhận hàng hoá và bưu kiện, được vận hành ở quy mô lớn và đã được tối ưu qua nhiều năm. Trong khi đó, hệ thống gửi đồ trong đề tài tập trung vào nhu cầu cho thuê tủ tự phục vụ tại các không gian sinh hoạt hằng ngày. Quan trọng hơn, hệ thống được thiết kế như một giải pháp mà đơn vị sở hữu có thể tự triển khai và vận hành: chỉ với vài thao tác, một tủ mới có thể được lắp đặt và đưa vào hoạt động mà không cần phụ thuộc vào nhà cung cấp dịch vụ bên ngoài. Ngoài ra, trải nghiệm người dùng cuối cũng được chú trọng thông qua việc hỗ trợ cả hai phương thức mở tủ: mã PIN trong trường hợp người dùng không có điện thoại, và mã QR khi cần thao tác nhanh gọn.

Tất nhiên, đây là một sản phẩm trong khuôn khổ đồ án tốt nghiệp, nên chưa thể so sánh về độ trưởng thành và quy mô với các hệ thống thương mại đã vận hành thực tế. Một số phần vẫn còn dang dở. Cụ thể, việc tích hợp thanh toán trực tuyến qua PayOS mới ở giai đoạn ban đầu và chưa hoàn thiện trọn vẹn quy trình; bộ kiểm thử tự động của hệ thống cũng cần được hoàn chỉnh để đảm bảo độ tin cậy lâu dài. Những hạn chế này sẽ được đề cập kỹ hơn ở phần hướng phát triển.

Những đóng góp kỹ thuật nổi bật của đồ án tập trung vào bốn nội dung đã trình bày ở chương trước: cơ chế mở tủ an toàn bằng mã PIN và mã QR, khả năng đồng bộ trạng thái tủ theo thời gian thực, quy trình tự động ghép nối kiosk khi lắp đặt, và

cơ chế tự động thu hồi tử khi hết hạn thuê. Đây là những vấn đề được đầu tư nhiều công sức giải quyết và cũng là những điểm mang lại giá trị thực sự cho hệ thống.

Qua quá trình thực hiện đồ án, tôi rút ra được một số bài học kinh nghiệm quan trọng. Trước hết, việc thiết kế cẩn thận ngay từ đầu có ý nghĩa rất lớn; một quyết định nhỏ về kiến trúc hoặc luồng dữ liệu, nếu được cân nhắc kỹ, có thể giúp hạn chế nhiều vấn đề phát sinh trong các giai đoạn sau. Bên cạnh đó, cần hiểu rõ bản chất của vấn đề trước khi bắt tay vào triển khai, bởi cách tiếp cận vội vàng thường dẫn đến việc phải chỉnh sửa và hoàn thiện lại nhiều lần.

6.2 Hướng phát triển

Dựa trên kết quả đã đạt được, một số hướng phát triển tiếp theo được đề xuất nhằm hoàn thiện hệ thống như sau:

- Bổ sung thêm các hình thức mở tử bên cạnh mã PIN và mã QR, chẳng hạn như thẻ sinh viên hoặc thẻ từ, để phù hợp hơn với từng môi trường triển khai.
- Tích hợp thêm các kênh thông báo như tin nhắn, Zalo hoặc email, giúp nhắc nhở người dùng kịp thời trước khi lượt thuê hết hạn.
- Phát triển chức năng thống kê và phân tích dữ liệu sử dụng cho người quản trị, từ đó hỗ trợ việc nắm bắt thói quen thuê tử, tối ưu bố trí tử và điều chỉnh chính sách giá.
- Mở rộng ứng dụng di động sang nền tảng iOS, bên cạnh phiên bản Android hiện có, nhằm phục vụ nhiều nhóm người dùng hơn.

Dưới sự hướng dẫn tận tình của thầy Lê Bá Vui, đồ án đã được hoàn thành và hệ thống cơ bản đáp ứng được các mục tiêu ban đầu đề ra. Quá trình thực hiện đồ án đã mang lại nhiều kiến thức và kinh nghiệm quý báu. Hy vọng rằng hệ thống tử gửi đồ thông minh sẽ tiếp tục được hoàn thiện và có cơ hội mang lại sự tiện lợi thiết thực cho người dùng trong thực tế.

TÀI LIỆU THAM KHẢO

- [1] Raspberry Pi Ltd., *Raspberry Pi 3 Model B*, Product specifications, Available: <https://www.raspberrypi.com/products/raspberry-pi-3-model-b/>, 2016.
- [2] OASIS, *MQTT version 5.0*, OASIS Standard, Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>, 2019.
- [3] OpenJS Foundation, *The Node.js Event Loop*, Node.js Documentation, Available: <https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick>, 2024.
- [4] OpenJS Foundation, *Express — Node.js web application framework*, Express Documentation, Available: <https://expressjs.com/>, 2024.
- [5] Prisma Data, Inc., *Prisma Documentation*, Official documentation, Available: <https://www.prisma.io/docs>, 2024.
- [6] The Qt Company, *Qt for Python Documentation*, PySide6 Documentation, Available: <https://doc.qt.io/qtforpython-6/>, 2024.
- [7] Meta Platforms, Inc., *React Native Documentation*, Official documentation, Available: <https://reactnative.dev/>, 2024.
- [8] Meta Platforms, Inc., *React Documentation*, Official documentation, Available: <https://react.dev/>, 2024.
- [9] Socket.IO, *Socket.IO Documentation*, Official documentation, Available: <https://socket.io/docs/>, 2024.

A. PHỤ LỤC

A.1 Khai báo sử dụng công cụ trí tuệ nhân tạo

Để đảm bảo tính minh bạch trong quá trình thực hiện đồ án tốt nghiệp, mục này trình bày rõ việc sử dụng các công cụ hỗ trợ trí tuệ nhân tạo (AI). Trong suốt quá trình thực hiện, bốn công cụ AI dưới đây được sử dụng ở vai trò hỗ trợ, giúp tôi tra cứu kiến thức, thiết kế giao diện, tham khảo mã nguồn và soạn thảo tài liệu, chứ không thay thế quá trình tư duy và ra quyết định. Toàn bộ nội dung cuối cùng đều được tôi trực tiếp kiểm tra, chỉnh sửa và chịu trách nhiệm về mặt học thuật.

Công cụ	Mục đích sử dụng	Mức độ sử dụng
Google Gemini	Tra cứu kiến thức, giải thích khái niệm kỹ thuật và gợi ý hướng xử lý khi gặp khó khăn trong lập trình.	Thường xuyên
Google Stitch	Phác thảo giao diện (wireframe) cho ứng dụng trong giai đoạn thiết kế ban đầu.	Giai đoạn thiết kế
OpenAI Prism	Hỗ trợ soạn thảo và định dạng nội dung theo cú pháp LaTeX trong quá trình viết báo cáo.	Thường xuyên
ChatGPT	Tìm hiểu và giải thích các công nghệ sử dụng trong hệ thống, đồng thời tham khảo các đoạn mã nguồn mẫu trong quá trình lập trình.	Thường xuyên

Bảng A.1: Khai báo sử dụng các công cụ trí tuệ nhân tạo trong đồ án